RTL Design Sherpa

Bridge Hardware Architecture Specification 1.4

September 11, 2026

Table of Contents

1 Bridge Has Index....23

2 Bridge Micro-Architecture Specification Index....24

2.1 Overview....25

2.1.1 Key Features....25

2.1.2 Protocol Conversion Matrix....25

2.1.3 Design Philosophy....25

2.2 Design Notes....26

2.2.1 Version History....26

2.3 References....27

2.3.1 Visual Assets....27

2.3.2 Companion Specifications....27

2.3.3 Project-Level....27

2.3.4 Generator....27

2.4 Navigation....29

2.4.1 Document Organization....29

2.4.2 Quick Navigation....30

3 Bridge: AXI4 Full Crossbar Generator - Product Requirements Document....31

3.1 Executive Summary....32

3.2 Design Philosophy....33

3.2.1 Core Principles....33

3.2.2 Architecture Philosophy....34

3.3 CRITICAL: RTL Regeneration Requirements....36

3.4 1. Product Overview..... 37
3.4.1 1.1 Purpose..... 37
3.4.2 1.2 Target Audience..... 37
3.4.3 1.3 Success Criteria..... 37
3.4.4 1.4 Implementation Status and Phases..... 38
3.5 2. Architecture Overview..... 40
3.5.1 2.1 AXI4 vs AXIS vs APB..... 40
3.5.2 2.2 Block Diagram..... 41
3.5.3 2.3 Key Components..... 42
3.6 3. Functional Requirements..... 43
3.6.1 3.1 AXI4 Protocol Compliance..... 43
3.6.2 3.2 Address Decoding..... 43
3.6.3 3.3 Arbitration Strategy..... 44
3.6.4 3.4 Burst Handling..... 44
3.7 4. Non-Functional Requirements..... 45
3.7.1 4.1 Performance..... 45
3.7.2 4.2 Resource Usage (Estimated)..... 45
3.7.3 4.3 Quality Requirements..... 46
3.8 5. Interface Specifications..... 47
3.8.1 5.1 AXI4 Master Interfaces (M × 5 channels)..... 47
3.8.2 5.2 AXI4 Slave Interfaces (S × 5 channels)..... 48
3.8.3 5.3 Configuration Is Generation-Time, Not Parameters..... 48
3.9 6. Performance Modeling..... 50
3.9.1 6.1 Analytical Model..... 50

3.9.2 6.2 Resource Scaling....50
3.9.3 6.3 Comparison with Other Crossbars....51
3.10 7. Generator Architecture....52
3.10.1 7.1 Python Generator Structure....52
3.10.2 7.2 Address Map Configuration....53
3.11 8. Comparison with APB and Delta....55
3.11.1 8.1 Code Reuse from APB Generator....55
3.11.2 8.2 Code Reuse from Delta Generator....55
3.11.3 8.3 Complexity Comparison....55
3.12 9. Use Cases....57
3.12.1 9.1 Multi-Core Processor Interconnect....57
3.12.2 9.2 DMA + Accelerator System....57
3.12.3 9.3 FPGA System Integration....57
3.13 10. Testing Strategy....58
3.13.1 10.1 FUB (Functional Unit Block) Tests....58
3.13.2 10.2 Integration Tests....58
3.13.3 10.3 Performance Validation....58
3.14 11. Documentation Plan....59
3.14.1 11.1 Specifications (Before Code)....59
3.14.2 11.2 Performance Analysis (Before Implementation)....59
3.14.3 11.3 Code Generation....59
3.14.4 11.4 Verification....59
3.15 12. Success Metrics....60
3.15.1 12.1 Functional Completeness....60

3.15.2 12.2 Performance Targets....60
3.15.3 12.3 Educational Value....60
3.15.4 12.4 Reusability....60
3.16 12. Attribution and Contribution Guidelines....61
3.16.1 12.1 Git Commit Attribution....61
3.17 12.2 PDF Generation Location....62
3.18 13. Future Enhancements....63
3.18.1 13.1 Short-Term (Post-Initial Release)....63
3.18.2 13.2 Long-Term....63
3.19 14. Risk Assessment....64
3.20 15. Project Timeline (Estimated)....65
3.21 16. References....66
3.22 17. Glossary....67
4 Claude Code Guide: Bridge Subsystem....68
4.1 CRITICAL: Read Architecture Documents First....69
4.2 Quick Context....70
4.3 Target Architecture: Intelligent Width-Aware Routing....71
4.3.1 Efficient Multi-Width Design....71
4.3.2 Why This Architecture....71
4.3.3 Per-Master Output Paths....71
4.3.4 Benefits....72
4.3.5 Implementation Status....72
4.4 CRITICAL RULE #0: RTL Regeneration Requirements....73
4.4.1 The Golden Rule....73

4.4.2 Why This Is Non-Negotiable....73
4.4.3 Real Example (Historical Session)....73
4.4.4 Generator Files That Trigger Full Regeneration....73
4.4.5 The Regeneration Workflow....73
4.4.6 Symptoms of Version Mismatch....74
4.4.7 Think Like a Compiler Developer....74
4.4.8 Exception: Hand-Written RTL....74
4.5 Critical Pitfalls: slave_select_* Reversion After Handshake....76
4.5.1 The Problem: Combinational Address Decode....76
4.5.2 Visible Symptoms....76
4.5.3 The Solution (MANDATORY in All Generators)....76
4.5.4 When This Bug Appears....77
4.6 MANDATORY: Project Organization Pattern....78
4.6.1 Required Directory Structure....78
4.6.2 Testbench Class Location (MANDATORY)....78
4.6.3 Test File Pattern (MANDATORY)....79
4.7 Critical Rules for This Subsystem....81
4.7.1 Rule #0: Attribution Format for Git Commits....81
4.7.2 Rule #0.1: Testbench Architecture - MANDATORY SEPARATION....81
4.7.3 Rule #1: All Testbenches Inherit from TBBase....82
4.7.4 Rule #2: Mandatory Testbench Methods....82
4.7.5 Rule #3: Use GAXI Components for Protocol Handling....83
4.7.6 Rule #4: Queue-Based Verification....83
4.8 TOML/CSV-Based Bridge Generator (Phase 2 Complete)....85

4.8.1 Overview....85
4.8.2 Quick Start....85
4.8.3 Configuration Format Details....86
4.8.4 Channel-Specific Masters (Phase 2 Feature)....87
4.8.5 Example: RAPIDS-Style Configuration....87
4.8.6 Common User Questions....88
4.8.7 Generator Output Structure....90
4.8.8 Testing Generated Bridges....91
4.9 Bridge Architecture Quick Reference....93
4.9.1 Generated Bridge Crossbar Structure....93
4.9.2 Key Features....93
4.9.3 Address Map....93
4.10 Test Organization....95
4.10.1 Test Hierarchy....95
4.10.2 Test Levels....95
4.11 Common User Questions and Responses....96
4.11.1 Q: “How does the Bridge work?”....96
4.11.2 Q: “How do I generate a Bridge?”....96
4.11.3 Q: “How do I test a Bridge?”....97
4.12 Anti-Patterns to Avoid....98
4.12.1 Anti-Pattern 1: Embedded Testbench Classes....98
4.12.2 Anti-Pattern 2: Manual AXI4 Handshaking....98
4.12.3 Anti-Pattern 3: Memory Models for Simple Tests....98
4.13 Quick Reference....99

4.13.1 Finding Existing Components....99
4.13.2 Common Commands....99
4.14 Remember....100
4.15 PDF Generation Location....101
5 Document Information....102
5.1 Bridge Hardware Architecture Specification....102
5.2 Revision History....103
5.3 Document Purpose....106
5.4 Intended Audience....107
5.5 Related Documents....108
6 Purpose and Scope....109
6.1 What is Bridge?....109
6.2 The Problem Bridge Solves....110
6.3 Scope....111
6.4 Out of Scope....112
7 Document Conventions....113
7.1 Terminology....113
7.2 Signal Naming....114
7.3 Diagrams....115
7.3.1 Figure 1.1: Block Diagram Legend....115
7.4 Code Examples....116
7.5 Parameter Notation....117
8 Definitions and Acronyms....118
8.1 Acronyms....118

8.2 Definitions....120
9 Use Cases....121
9.1 Primary Applications....121
9.1.1 SoC Interconnects....121
9.1.2 Memory Systems....121
9.1.3 Accelerator Systems....121
9.2 Example: RAPIDS Accelerator....122
9.2.1 Figure 2.1: RAPIDS System Topology....122
9.2.2 Configuration....122
9.2.3 Benefits....122
9.3 Rapid Prototyping....124
10 Key Features....125
10.1 Configuration-Driven Generation....125
10.1.1 CSV/TOML Configuration....125
10.1.2 Example Configuration....125
10.2 Multi-Protocol Support....126
10.2.1 Protocol Features....127
10.3 Channel-Specific Masters....128
10.3.1 Write-Only Masters (wr)....128
10.3.2 Read-Only Masters (rd)....128
10.3.3 Full Masters (rw)....128
10.3.4 Resource Savings....128
10.4 Automatic Converters....129
10.4.1 Width Conversion....129

10.4.2 Protocol Conversion....129
10.5 Flexible Topology....130
10.5.1 Scalability....130
10.5.2 Mixed Configurations....130
11 System Context....131
11.1 Bridge in SoC Architecture....131
11.1.1 Figure 2.2: Bridge System Context....131
11.2 Interface Boundaries....132
11.2.1 Master-Side (Upstream)....132
11.2.2 Slave-Side (Downstream)....132
11.3 Address Space....133
11.3.1 Address Map Organization....133
11.3.2 Address Decode....133
11.4 Clock and Reset....134
11.4.1 Clock Domain....134
11.4.2 Reset....134
11.5 Dependencies....135
11.5.1 Required Infrastructure....135
11.5.2 Optional Features....135
12 Block Diagram....136
12.1 Bridge Architecture Overview....136
12.1.1 Figure 3.1: Bridge Block Diagram....136
12.2 Functional Blocks....137
12.2.1 Master Adapter....137

12.2.2 Crossbar Core....137
12.2.3 Slave Router....137
12.3 Data Flow....138
12.3.1 Figure 3.2: Write Path....138
12.3.2 Figure 3.3: Read Path....138
12.4 Scalability....139
12.4.1 Topology Parameters....139
12.4.2 Resource Scaling....139
13 Data Flow....140
13.1 Transaction Flow Overview....140
13.1.1 Write Transaction Flow....140
13.1.2 Read Transaction Flow....140
13.2 Channel Independence....141
13.2.1 AXI4 Channel Separation....141
13.2.2 Parallel Operation....141
13.3 Bridge ID (there is no ID extension)....142
13.3.1 Bridge ID (BID) Concept....142
13.3.2 IDs are pass-through, not extended....142
13.4 Response Routing....143
13.4.1 B Channel (Write Response)....143
13.4.2 R Channel (Read Data)....143
13.4.3 The requirement this creates....143
13.5 Ordering: what the fabric actually guarantees....144
14 Protocol Support....145

14.1 Supported Protocols....145
14.1.1 AXI4 Full....145
14.1.2 AXI4-Lite....145
14.1.3 APB (Advanced Peripheral Bus)....145
14.2 Protocol Conversion Matrix....146
14.3 Automatic Conversion Shims at Slave Boundary....148
14.4 Automatic Conversion at the Master Boundary....149
14.5 AXI4 to APB Conversion....151
14.5.1 Conversion Process....151
14.5.2 Figure 3.1: AXI4 to APB Conversion Sequence....151
14.5.3 Burst Handling....151
14.5.4 Timing Considerations....151
14.6 Width Conversion....152
14.6.1 Upsize (Narrow to Wide)....152
14.6.2 Figure 3.2: Width Upsize Conversion....152
14.6.3 Downsize (Wide to Narrow)....153
14.6.4 Figure 3.3: Width Downsize Conversion....153
14.7 Channel-Specific Protocol....154
14.7.1 Write-Only Masters....154
14.7.2 Read-Only Masters....154
15 AXI4 Interface Overview....155
15.1 Interface Organization....155
15.1.1 Master-Side Interfaces....155
15.1.2 Slave-Side Interfaces....155

15.2 Signal Naming Convention....157
15.2.1 Custom Prefixes....157
15.2.2 ID Width Extension....157
15.3 Channel-Specific Interfaces....158
15.3.1 Write-Only Master (wr)....158
15.3.2 Read-Only Master (rd)....158
15.4 Monitor Bus Output (when variants includes mon)....159
15.5 Handshake Protocol....160
15.5.1 Valid/Ready Handshake....160
15.5.2 Backpressure....160
15.6 Burst Support....161
15.6.1 Supported Burst Types....161
15.6.2 Burst Length....161
16 APB Interface....162
16.1 Overview....162
16.2 Ports....163
16.2.1 Signal Definition....163
16.2.2 Signal Naming....163
16.3 Functional Description....164
16.3.1 AXI4 to APB Conversion....164
16.3.2 Burst Handling....164
16.3.3 Error Mapping....164
16.4 Timing....165
16.5 Waveforms....166

16.5.1 Write Transaction....166
16.5.2 Read Transaction....166
16.6 APB Requester Ports....167
16.7 Design Notes....168
17 Clock and Reset....169
17.1 Overview....169
17.2 Ports....170
17.2.1 Clock....170
17.2.2 Reset....170
17.3 Functional Description....171
17.3.1 Single Clock Domain....171
17.3.2 No Clock Domain Crossing....171
17.3.3 Reset Behavior....171
17.3.4 Reset Effects....172
17.3.5 Outstanding Transactions....172
17.3.6 Reset Release....172
17.4 Timing....173
17.4.1 Setup and Hold....173
17.4.2 Clock-to-Output....173
17.5 Usage Example....174
17.5.1 Recommended Constraints....174
17.6 Design Notes....175
17.6.1 External CDC Required....175
17.6.2 CDC Recommendations....175

18 AXI5 and APB5 Interfaces (AMBA5 Support)....176
18.1 Overview....176
18.2 Parameters....177
18.2.1 Declaring AMBA5 Ports....177
18.3 Ports....178
18.3.1 AXI5 Port Surface....178
18.3.2 APB5 Slave Surface....178
18.3.3 AXI5-Lite Master Surface....179
18.3.4 APB5 Master Surface....179
18.3.5 AXI5-Lite Slave Surface....180
18.4 Functional Description....181
18.4.1 Interop Matrix....181
19 Unmapped-Address Handling....183
19.1 Overview....183
19.1.1 Why the bridge answers an address no slave owns....183
19.2 Ports....184
19.2.1 Status, interrupt and clearing....184
19.3 Functional Description....185
19.3.1 Behaviour....185
19.3.2 Register access (cfg regblock builds)....185
19.3.3 When it can never fire....186
19.3.4 What this does not do....186
19.4 Navigation....187
20 Wishbone B4 Interface....188

20.1 Overview....188
20.2 Ports....189
20.2.1 Signal Definition....189
20.2.2 Signal Naming....190
20.3 Functional Description....191
20.3.1 Slave port: AXI4 to Wishbone....191
20.3.2 Master port: Wishbone to AXI4....191
20.3.3 Rules the validator enforces....191
20.4 Design Notes....192
21 Throughput Characteristics....193
21.1 Overview....193
21.2 Functional Description....194
21.2.1 Peak Throughput....194
21.2.2 Formula....194
21.2.3 Factors Affecting Throughput....194
21.2.4 Multi-Master Scaling....195
21.2.5 Burst Efficiency....195
21.2.6 Width Conversion Impact....195
21.2.7 Protocol Conversion Impact....196
21.3 Design Notes....197
22 Latency Analysis....198
22.1 Overview....198
22.2 Timing....199
22.2.1 Address Path Latency....199

22.2.2 Data Path Latency....199
22.2.3 Response Path Latency....199
22.2.4 End-to-End Latency....200
22.2.5 Width Conversion Latency....201
22.2.6 Protocol Conversion Latency....202
22.3 Design Notes....203
22.3.1 Design Recommendations....203
23 Resource Estimates....204
23.1 Overview....204
23.2 Functional Description....205
23.2.1 Baseline Configuration (2x2, 64-bit)....205
23.2.2 Scaling Factors....205
23.2.3 Per-Master Resources....205
23.2.4 Per-Slave Resources....206
23.2.5 Crossbar Core....206
23.2.6 Width Converter Resources....206
23.2.7 Protocol Converter Resources....206
23.2.8 Example Configurations....207
23.3 Design Notes....208
23.3.1 Resource Reduction....208
23.3.2 Performance/Resource Trade-off....208
24 System Requirements....209
24.1 Overview....209
24.1.1 Clock Infrastructure....209

24.1.2 Reset Infrastructure..... 209
24.1.3 Power..... 209
24.2 Parameters..... 210
24.2.1 Address Map..... 210
24.2.2 Connectivity..... 210
24.3 Ports..... 211
24.3.1 Master Requirements..... 211
24.3.2 Slave Requirements..... 211
24.4 Timing..... 212
24.4.1 Setup/Hold..... 212
24.4.2 Recommended Margins..... 212
24.5 Testing..... 213
24.5.1 Pre-Integration Checks..... 213
24.5.2 Integration Checks..... 213
24.5.3 Post-Integration Checks..... 213
25 Parameter Configuration..... 214
25.1 Overview..... 214
25.2 Parameters..... 215
25.2.1 Core Parameters..... 215
25.2.2 Derived Parameters..... 215
25.2.3 Per-Port Configuration..... 215
25.2.4 Top-Level Monitor Configuration..... 217
25.2.5 Parameter Validation..... 218
25.3 Usage Example..... 219

25.3.1 TOML Configuration..... 219
25.3.2 CSV Connectivity..... 219
25.3.3 Simple 2x2..... 219
25.3.4 Mixed Protocol..... 220
26 Verification Strategy..... 221
26.1 Overview..... 221
26.2 Testing..... 222
26.2.1 Verification Levels..... 222
26.2.2 Test Infrastructure..... 223
26.2.3 Test Execution Model..... 224
26.2.4 Coverage Goals..... 224
26.2.5 Debug Features..... 225
26.2.6 Regression Testing..... 226
26.2.7 Known Limitations..... 226

List of Figures

Figure 1.1: Block Diagram Legend....115
Figure 2.1: RAPIDS System Topology....122
Figure 2.2: Bridge System Context....131
Figure 3.1: Bridge Block Diagram....136
Figure 3.2: Write Path....138
Figure 3.3: Read Path....138
Figure 3.1: AXI4 to APB Conversion Sequence....151
Figure 3.2: Width Upsize Conversion....152
Figure 3.3: Width Downsize Conversion....153
Figure 4.1: Clock Distribution....171
Figure 4.3: Reset Timing....171
Figure 4.2: Multi-Clock CDC....175

List of Tables

Table 1: Table 1.1: Common Terminology....113
Table 2: Table 1.2: Signal Naming Prefixes....114
Table 3: Table 1.3: Block Diagram Symbols....115
Table 4: Table 1.4: Parameter Format Example....117
Table 5: Table 1.5: Acronyms and Abbreviations....118
Table 6: Table 2.1: Supported Protocols....126
Table 7: Table 2.2: Channel-Specific Resource Savings....128
Table 8: Table 2.3: Address Map Organization....133
Table 9: Table 3.1: Topology Parameter Scaling....139
Table 10: Table 3.2: AXI4 Channel Summary....141
Table 11: Table 3.3: Protocol Conversion Matrix....146
Table 12: Table 3.4: Slave-port protocol values....148
Table 13: Table 3.5: Master-port protocol values....149
Table 14: Table 4.9: Master-Side Interface Channels....155
Table 15: Table 4.10: Slave-Side Interface Channels....156
Table 16: Table 4.11: Supported Burst Types....161
Table 17: Table 4.1: APB Signal Definitions....163
Table 18: Table 4.2: AXI4 to APB Burst Conversion....164
Table 19: Table 4.3: APB to AXI4 Error Mapping....164
Table 20: Table 4.4: Clock Requirements....170
Table 21: Table 4.5: Reset Signal....170
Table 22: Table 4.6: Reset Effects....172
Table 23: Table 4.7: Timing Constraints....173
Table 24: Table 4.8: Clock-to-Output Delays....173
Table 25: AXI5-Lite sideband groups at an axil5 master boundary....179
Table 26: AXI5-Lite sideband groups at an axil5 slave boundary....180
Table 27: Unmapped-address status registers....185
Table 28: Table 4.7: Wishbone B4 Signal Definitions....189
Table 29: Table 4.8: Wishbone termination to AXI4 response....191
Table 30: Table 5.1: Peak Throughput by Data Width....194
Table 31: Table 5.2: Factors Affecting Throughput....194
Table 32: Table 5.3: Burst Efficiency Comparison....195
Table 33: Table 5.4: AXI4 vs APB Protocol Impact....196
Table 34: Table 5.5: Address Path Latency....199
Table 35: Table 5.6: Data Path Latency....199
Table 36: Table 5.7: Response Path Latency....199

Table 37: Table 5.8: Upsize Latency by Ratio....201
Table 38: Table 5.9: Downsize Latency by Ratio....201
Table 39: Table 5.10: AXI4 to APB Conversion Latency....202
Table 40: Table 5.11: Baseline Resource Requirements....205
Table 41: Table 5.12: Resource Scaling Factors....205
Table 42: Table 5.13: Per-Master Resource Breakdown....205
Table 43: Table 5.14: Per-Slave Resource Breakdown....206
Table 44: Table 5.15: Crossbar Core Resources....206
Table 45: Table 5.16: Width Converter Resources....206
Table 46: Table 5.17: AXI4 to APB Converter Resources....206
Table 47: Table 5.18: Complete Bridge Resource Estimates....207
Table 48: Table 5.19: Performance/Resource Trade-offs....208
Table 49: Table 6.1: Clock Requirements....209
Table 50: Table 6.2: Reset Requirements....209
Table 51: Table 6.3: Power Requirements....209
Table 52: Table 6.4: Address Map Requirements....210
Table 53: Table 6.5: Connectivity Requirements....210
Table 54: Table 6.6: Timing Requirements....212
Table 55: Table 6.7: Recommended Timing Margins....212
Table 56: Table 6.8: Bridge Core Parameters....215
Table 57: Table 6.9: Derived Parameters....215
Table 58: Table 6.10: Master Port Configuration....216
Table 59: Table 6.11: Slave Port Configuration....216
Table 60: Table 6.12: Generator Validation Checks....218
Table 61: Table 6.13: Unit Level Test Focus....222
Table 62: Table 6.14: Integration Test Coverage....222
Table 63: Table 6.15: System Level Tests....222
Table 64: Table 6.16: Test Categories....223
Table 65: Table 6.17: Functional Coverage Goals....224
Table 66: Table 6.18: Code Coverage Goals....225
Table 67: Table 6.19: Debug Signals....225
Table 68: Table 6.20: CI Test Schedule....226
Table 69: Table 6.21: Configuration Test Matrix....226
Table 70: Table 6.22: Known Test Gaps....226
Table 71: Table 6.23: Simulator Support Status....226

1 Bridge Has Index

Generated: 2026-09-11

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Bridge Micro-Architecture Specification Index

Component: Bridge (Multi-Protocol Crossbar Generator) **Version:** 1.2 **Date:** 2026-09-07 **Purpose:** Detailed micro-architecture specification for Bridge component

---

2.1 Overview

Bridge is a CSV-driven generator that produces AXI4 crossbars with automatic width conversion, protocol conversion (AXI4, AXI4-Lite, APB), and channel-specific masters (read-only, write-only, or full). This is the internals document — the one you open when you need to know what the generated RTL actually does, block by block, rather than what the brochure says.

2.1.1 Key Features

- **Multi-Protocol Support:** AXI4, AXI4-Lite, and APB slave conversion
- **CSV-Driven Configuration:** Human-readable port and connectivity definitions
- **Channel-Specific Masters:** Write-only (wr), read-only (rd), or full (rw) support
- **Automatic Width Conversion:** Upsize/downsize for data width mismatches
- **Out-of-Order Support:** NONE. Responses route by in-order bridge_id FIFO position; a slave that reorders between IDs misroutes (BRIDGE-010)
- **Custom Signal Prefixes:** Unique prefixes per port for clean integration

That out-of-order bullet is the one people skim past and regret. The bridge does not sort responses back into order — it trusts the FIFO position, so the trust has to be mutual.

2.1.2 Protocol Conversion Matrix

What happens to a transaction depends on the master/slave protocol pairing:

Master Protocol	Slave Protocol	Conversion
AXI4	AXI4	Direct or width convert
AXI4	AXI4-Lite	Protocol downgrade
AXI4	APB	Full protocol conversion

2.1.3 Design Philosophy

Efficient Multi-Width Routing: - Direct connections where widths match (zero conversion overhead) - Width converters only where needed - No fixed internal crossbar width

Resource Optimization: - Channel-specific masters reduce port count by 40-60% - Per-path width converters instead of global conversion - Minimal logic for matching-width connections

2.2 Design Notes

2.2.1 Version History

Version 1.2 (2026-09-07): qc round_1/round_2 correctness pass. Retired the ID-extension architecture from every page that claimed it (IDs are pass-through; responses route by an in-order `bridge_id` FIFO – see BRIDGE-010 for the ordering requirement that creates). Corrected the out-of-range responder against `axi4_subtractive_slave` (RLAST on the final beat only, DECERR, 0xDEADBEEF), the monbus group instantiation, the transaction timeout that does not exist, and the strobe-mapping example. Documented the subtractive catch-all and its status/IRQ path.

Version 1.1 (2026-06-04): content sync to RTL state at that date.

Version 1.0 (2026-01-03): - Initial specification release - Restructured from single spec to HAS/MAS format - Complete block-level documentation - FSM and ID management details

2.3 References

2.3.1 Visual Assets

Diagram sources and their renders live in two directories:

- **Source Files:**
  - `assets/graphviz/*.gv` - Graphviz source diagrams
  - `assets/puml/*.puml` - PlantUML FSM diagrams
- **Rendered Files:**
  - `assets/graphviz/*.png` - Rendered block diagrams
  - `assets/puml/*.png` - Rendered FSM diagrams

2.3.1.1 Architecture Diagrams

1. **Master Adapter** - assets/graphviz/master_adapter.png
2. **Slave Router** - assets/graphviz/slave_router.png
3. **Crossbar Core** - assets/graphviz/crossbar_core.png
4. **ID Management** - assets/graphviz/id_management.png
5. **Width Conversion** - assets/graphviz/width_conversion.png
6. **Protocol Conversion** - assets/graphviz/protocol_conversion_apb.png
7. **Response Routing** - assets/graphviz/response_routing.png
8. **Error Handling** - assets/graphviz/error_handling.png

2.3.1.2 FSM Diagrams

1. **AW Arbiter FSM** - assets/puml/aw_arbiter_fsm.png
2. **AR Arbiter FSM** - assets/puml/ar_arbiter_fsm.png

2.3.2 Companion Specifications

- **Bridge HAS** - Hardware Architecture Specification (high-level)

2.3.3 Project-Level

- **PRD.md:** ../../PRD.md - Complete product requirements document
- **CLAUDE.md:** ../../CLAUDE.md - AI assistant integration guide

2.3.4 Generator

- **Generator:** `../../bin/bridge_generator.py` - CSV/TOML-based generator script (with `../../bin/bridge_pkg/`)

- **Test Configs:** `../../bin/test_configs/` - Example TOML/CSV configurations

2.4 Navigation

2.4.1 Document Organization

Each chapter below is one source file.

2.4.1.1 Front Matter

- Document Information

2.4.1.2 Chapter 1: Introduction

- Overview

2.4.1.3 Chapter 2: Block Descriptions

- Master Adapter
- Slave Router
- Crossbar Core
- Arbitration
- ID Management
- Width Conversion
- Protocol Conversion
- Response Routing
- Error Handling
- AMBA5 Boundary and Native Sideband

2.4.1.4 Chapter 3: FSM Design

- Arbiter FSMs
- Transaction Tracking

2.4.1.5 Chapter 4: ID Management

- CAM Architecture – describes a design that was NEVER BUILT; see ID Tracking for the mechanism actually generated
- ID Tracking Tables

2.4.1.6 Chapter 5: Converters

- Width Converters
- APB Converters

2.4.1.7 Chapter 6: Generated RTL

- Module Structure

- Signal Naming

2.4.1.8 Chapter 7: Verification

- Test Strategy
- Debug Guide

2.4.2 Quick Navigation

2.4.2.1 For New Users

1. Start with Overview for a tour of what Bridge can do
2. Read Block Diagram for the architecture
3. Study Arbitration for operational details
4. Reference Module Structure for generated RTL

2.4.2.2 For Integration

- **Protocol support:** See Protocol Conversion for AXI4/APB/AXI-Lite
- **Width handling:** See Width Conversion for data width mismatches
- **ID management:** See ID Management for transaction tracking
- **Error handling:** See Error Handling for OOR and timeout

Last Updated: 2026-09-07 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 Bridge: AXI4 Full Crossbar Generator - Product Requirements Document

Project: Bridge **Version:** 2.1 **Status:** Phase 2 Complete - Simplified Architecture with Hard Limits
Created: 2025-10-18 **Last Updated:** 2025-11-02

3.1 Executive Summary

Bridge is a Python-based AXI4 crossbar generator that produces simple, performant SystemVerilog RTL for connecting multiple AXI4 masters to multiple AXI4 slaves. The name follows the infrastructure theme - bridges connect different regions, enabling communication across divides, just like crossbars connect masters and slaves.

Design Philosophy: A simple AMBA fabric that is performant, but makes no attempt to support all features. ID and address widths are PER PORT, set in the TOML (`id_width`, `addr_width`) – see §5.3, “Configuration Is Generation-Time, Not Parameters”. An earlier revision of this line claimed hard 8-bit/64-bit limits; the generator has never enforced them, and the shipped configs use 2-, 4- and 8-bit IDs.

Key Differentiator from Delta: - **Delta:** AXI-Stream crossbar (streaming data, single channel, simple routing) - **Bridge:** AXI4 full crossbar (memory-mapped, 5 channels, burst support, `bridge_id` sideband routing)

Key Differentiator from Commercial IP: - **Commercial AXI4 Crossbars:** Feature-complete, support every AXI4 edge case, complex configurability - **Bridge:** Simple and performant, supports common use cases, hard limits for simplicity

Target Use Case: High-performance memory-mapped interconnects for multi-core processors, accelerators, and memory controllers where simplicity and performance matter more than spec compliance.

3.2 Design Philosophy

Vision: A simple AMBA fabric that is performant, but makes no attempt to support all features.

3.2.1 Core Principles

1. Simplicity Over Completeness - Focus on common use cases, not edge cases - Implement what's needed for real hardware, not AXI4 spec compliance theater - Prefer straightforward implementations over complex feature sets

2. Performance First - Direct connections where possible (matching widths = zero overhead) - Minimal conversion logic in critical paths - Optimize for throughput and latency, not feature count

3. Hard Limits for Simplicity

We enforce uniform widths on certain parameters to eliminate complexity:

Parameter	Hard Limit	Rationale
ID Width	Per port, set at generation	NOT fixed at 8. The TOML sets `id_width` per port and the generated bridge bakes it in – `bridge_2x2_rw` has 4-bit IDs. This row previously said "8 bits (fixed)", which the generator has never enforced.
Address Width	Per port, set at generation	NOT fixed at 64. `bridge_2x2_rw` is a 32-bit-address bridge. Same correction as the row above.
Data Width	Variable (32b-512b)	Width converters ONLY for data. Commonly needed for bandwidth optimization.

Why This Works: - ID width conversion adds complexity with minimal benefit (nobody needs >256 outstanding transactions per master) - Address width conversion is rarely needed (peripheral addresses fit in 32-bit, but using 64-bit everywhere is simpler) - Data width conversion is the ONLY width conversion that provides real value (bandwidth matching)

4. What We DON'T Support (By Design)

Features intentionally excluded for simplicity: - (Both of these WERE excluded once and no longer are: ID width and address width are per-port generation inputs. Left here as history because the "design differentiator" argument above still cites them.) - ~~No AXI4-Lite protocol variant~~ – `axil` and `axil5` ARE supported, as slave protocols (converted at the boundary by `axi4_to_axil4_{rd,wr}` / `axi4_to_axil5_{rd,wr}`) and, since BRIDGE-014, as master protocols (tied-off promotion plus the AXI5-Lite sideband surface). `apb` / `apb5` likewise work on both sides (`axi4_to_apb{4,5}_shim` at a slave, `apb{4,5}_to_axi4` at a master), and so does Wishbone B4 since BRIDGE-019 (`axi4_to_wb4` / `wb4_to_axi4`). - No ACE protocol extensions (cache coherency) - ~~No AXI5 features~~ – interop-mode AXI5 sideband, native sideband through the structs, atomics of every class and APB5/AXI5-Lite ports were delivered by BRIDGE-002; what is NOT built is a fabric that is itself AXI5 (BRIDGE-018) - QoS, Region and REQUEST-side User (AWUSER/ARUSER/WUSER) are routed since `c2955863`/`2b229516`. RESPONSE-side User is not: `BUSER`/`RUSER` exist as ports but the master adapters tie them to zero (`.fub_axi_buser(1'b0)`, `.fub_axi_ruser(1'b0)`), so a slave's response User never reaches its master. An earlier correction of this line said "end to end", which was true only of the request side.

5. What We DO Support

Core AXI4 features that matter: - Full 5-channel AXI4 protocol (AW, W, B, AR, R) - Burst transactions (INCR, WRAP, FIXED) - In-order completion. Responses are routed by a per-master `bridge_id` sideband and an in-order FIFO, NOT by matching AXI IDs – see FR-2. - Multiple outstanding transactions per master, **to one slave at a time**. The generated master adapters carry a single-outstanding-target gate (`aw_gate_ok` / `ar_gate_ok`): a new AW/AR to a DIFFERENT slave is held off until the outstanding ones drain. - Channel-specific masters (write-only, read-only, read-write) - Data width conversion (32b ↔️ 64b ↔️ 128b ↔️ 256b ↔️ 512b) - Configurable M×N topology (1-32 masters, 1-256 slaves) - Fair round-robin arbitration per slave

3.2.2 Architecture Philosophy

Delivered: Intelligent width-aware routing, not a fixed-width crossbar. The generated adapters carry per-width paths and convert only at a mismatch.

OLD Approach (What We're Moving Away From):

```
Master (64b) → Upsize(64→256) → Fixed 256b Crossbar → Downsize(256→64)
→ Slave (64b)
```

- Two conversions for same-width connections (wasteful!)
- Artificial bandwidth bottleneck
- Unnecessary logic and latency

NEW Approach (Target Architecture):

```
Master (64b) → Direct Connection → Slave (64b)    [0 conversions!]
Master (512b) → Conv(512→64) → Slave (64b)         [1 conversion]
```

- Per-master paths to each unique slave width it connects to
- Direct paths where widths match (zero overhead)
- Converters only where actually needed
- Router selects appropriate path based on address decode

Result: Minimal conversion logic, maximum performance, pragmatic simplicity.

3.3 CRITICAL: RTL Regeneration Requirements

ALL generated RTL files MUST be deleted and regenerated together whenever ANY generator code changes.

Why This Matters: - Generated RTL files may have interdependencies (bridges, wrappers, integrators) - Generator code changes can create version mismatches between files - Partial regeneration creates subtle incompatibilities that cause test failures - Even “small innocuous” generator changes can have cascading effects

The Rule:

```
# WRONG - Partial regeneration
./bridge_generator.py --masters 5 --slaves 3 --output ../rtl/
# Only regenerates bridge_axi4_flat_5x3.sv
# Other files (wrappers, integrators) now mismatched!

# CORRECT - Full regeneration
cd bin
make clean                                  # Delete ALL generated
bridges (rtl/generated/)
make all                                    # Regenerate everything from
bridge_batch.csv
```

Generator Files That Trigger Full Regeneration: - `bin/bridge_generator.py` - Main bridge generator (CSV/TOML driven) - Any Python file in `bin/bridge_pkg/` (components/, generators/, config, csv_parser, ...) - Any Jinja template in `bin/bridge_pkg/jinja_templates/` - `bridge_wrapper_generator.py` - Wrapper generation - **Any** Python file in `projects/components/bridge/bin/`

Symptoms of Version Mismatch: - Tests that previously passed now fail - Simulation errors about missing signals - Mismatched port widths or counts - Address decode routing to wrong slaves

Think of Generated RTL Like Compiled Code: When you update a compiler, you don’t selectively recompile - you rebuild everything. When you update a generator, you don’t selectively regenerate - you regenerate everything.

3.4 1. Product Overview

3.4.1 1.1 Purpose

Bridge automates generation of AXI4 crossbar infrastructure: - **Python code generation** - Parameterized RTL generation (similar to APB/Delta) - **Performance modeling** - Analytical + simulation validation - **Flat topology** - Full M×N interconnect matrix - **`bridge_id` sideband routing** - in-order response return (see FR-2) - **Burst optimization** - Pipelined burst transfers

3.4.2 1.2 Target Audience

Primary Users: - RTL designers building SoC interconnect - System architects evaluating interconnect topologies - Verification engineers needing crossbar testbenches - Students learning AXI4 protocol and interconnects

Educational Focus: - Demonstrates AXI4 protocol complexity - Shows arbitration strategies for memory-mapped busses - Teaches ID-based transaction tracking - Illustrates burst optimization techniques

3.4.3 1.3 Success Criteria

Functional: - [x] Generates working AXI4 crossbar RTL (bridge_generator.py) - [x] Generates CSV/TOML-configured bridges (bridge_generator.py + bridge_pkg/; the earlier separate bridge_csv_generator.py was merged in) - [x] Passes Verilator lint - [x] Supports 1-32 masters, 1-256 slaves - [ ] Handles out-of-order completion via IDs. NOT IMPLEMENTED: `bridge_cam.sv` exists but is instantiated by no generated module. The slave adapters run "Bridge ID Tracking - FIFO Mode (In-Order)" and 93 generated files still declare `cam_wr_allocate`/`cam_rd_allocate` without driving them. - [x] Supports burst lengths 1-256 beats - [x] Channel-specific masters (wr/rd/rw) for resource optimization (Phase 2) - [x] APB converter integration – DELIVERED. `axi4_to_apb4_shim` is instantiated by `apb_periph_adapter.sv` in every mixed config, and `bridge_1x2_rw_apb5`, `bridge_1x2_rw_apb5_mon` and the four `mix_*` bridges ship with APB slaves.

Performance: - [ ] Latency ≤ 3 cycles for single-beat transactions. **Unverified for the generated bridge.** The generated path crosses four skid buffers on the round trip (adapter timing wrappers on both sides), and gaxi_skid_buffer registers its output side, so an ideal zero-wait slave still costs >=4 cycles of bridge-added latency before arbitration. The 2-3 cycle figure in 6.1 describes the wrapperless flat crossbar, not what the generator emits. - [ ] Throughput: M×N paths do NOT all transfer concurrently. Each master is held to ONE target slave at a time (`aw_gate_ok`/`ar_gate_ok`, "single-outstanding-target"), a deadlock fix, not an oversight. Concurrency is across MASTERS, not across a master's targets. - [x] Performance models implemented (bridge_model.py - V1 Flat) - [ ] Fmax ≥ 300 MHz on UltraScale+ FPGAs (pending synthesis validation)

Quality: - [x] Complete specifications (PRD) before code - [x] Performance models validate requirements (bridge_model.py) - [x] All generated RTL Verilator verified - [x] Integration

examples provided (CSV examples) - [x] Comprehensive documentation (GENERATOR_ARCHITECTURE.md, docs/bridge_has/, docs/bridge_mas/)

3.4.4 1.4 Implementation Status and Phases

Unified Generator (bridge_generator.py):

The bridge generator now supports both TOML/CSV configuration and legacy array-indexed modes:

Configuration Modes: 1. **TOML/CSV Mode (Preferred)** - TOML port configuration (`bridge_name.toml`) - CSV connectivity matrix (`bridge_name_connectivity.csv`) - Custom port prefixes (`rapids_m_axi_`, `cpu_m_axi_`, etc.) - Interface wrapper integration (timing isolation) - Mixed protocols (AXI4 + APB + AXI4-Lite slaves) - Channel-specific masters (wr/rd/rw) - Status: Phase 2 complete, Phase 3 pending

2. **Legacy CSV Mode (Backwards Compatible)**
   - Separate `ports.csv` and `connectivity.csv` files
   - Migration path to TOML format
   - See `bin/test_configs/README.md` for conversion guide

Phase Status:

Phase 1: CSV Configuration COMPLETE - CSV parser (ports.csv, connectivity.csv) - Port generation with custom prefixes - Converter identification logic - Basic crossbar instantiation

Phase 2: Channel-Specific Masters COMPLETE (2025-10-26) - Added `channels` field to PortSpec (rw/wr/rd) - Conditional port generation based on channels - **Resource Optimization:** - wr (write-only): AW, W, B channels only → 39% port reduction - rd (read-only): AR, R channels only → 61% port reduction - rw (full): All 5 channels - Width converter awareness (only generate needed converters) - Example: 4-master bridge saves 35% signals with optimized channels

Phase 3: APB Converter Integration – DELIVERED - AXI2APB converter module: `axi4_to_apb4_shim`, emitted by `axi4_to_apb4_shim_component.py` - APB signal packing/unpacking: in the shim - APB converter instantiation in generated bridges: `apb_periph_adapter.sv` and the `periph5_adapter.sv` variants - Width converter + APB converter chaining: the `*_mon` builds chain the AXI4 timing wrapper into the shim - End-to-end testing with APB slaves: `test_bridge_1x2_rw_apb5*` and the `mix_*` suites, all in the 72-test FULL regression - The “placeholders with TODO comments” status was stale. This block said PENDING while the shim it describes was shipping in six configurations.

Additional Resources: - `models/bridge_model/bridge_model.py` - Performance modeling (V1 Flat implemented) - `rtl/bridge_cam.sv` - CAM for ID tracking. **Not instantiated by any generated module**; kept for the OOO work FR-2 describes as not implemented. - See `GENERATOR_ARCHITECTURE.md` for the generator/build architecture - See `docs/bridge_has/` and `docs/bridge_mas/` for architecture diagrams and specs (the older BRIDGE_CURRENT_STATE.md / BRIDGE_ARCHITECTURE_DIAGRAMS.md were superseded)

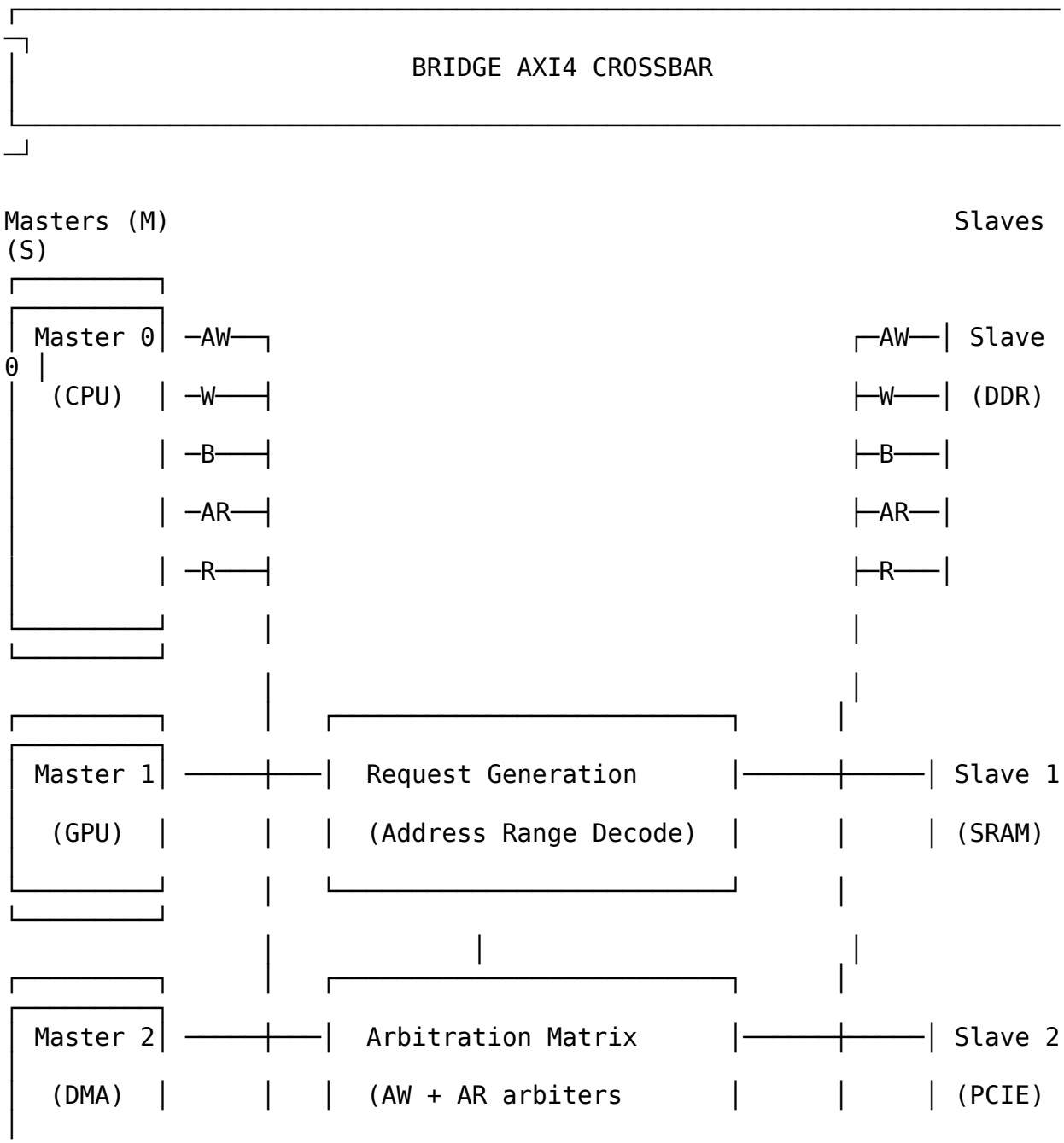
3.5 2. Architecture Overview

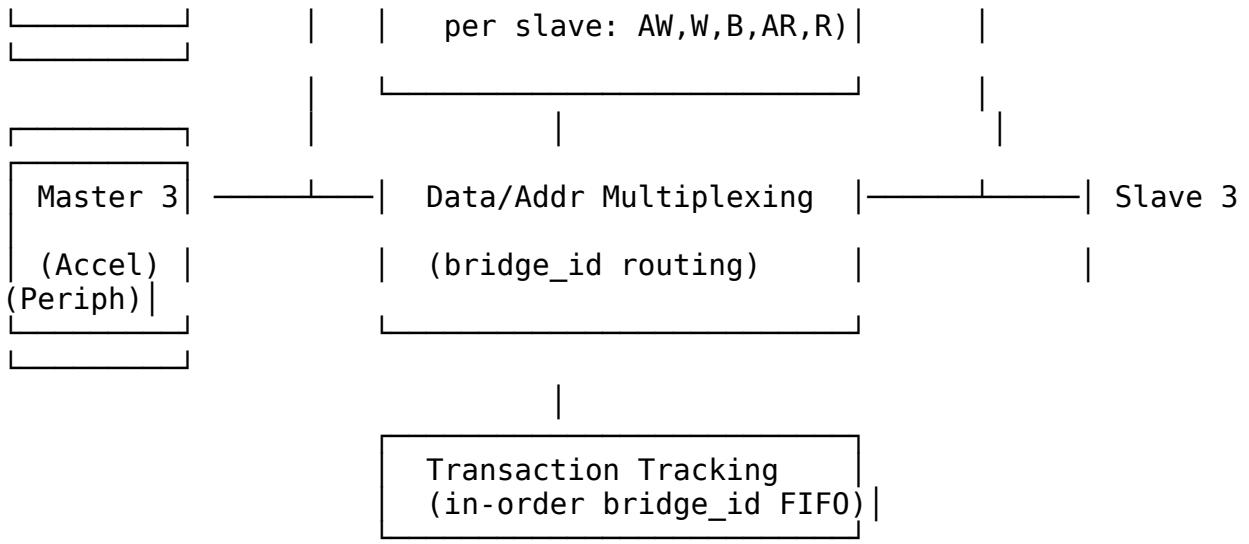
3.5.1 2.1 AXI4 vs AXIS vs APB

Feature	APB	AXI-Stream (Delta)	AXI4 (Bridge)
Protocol Type	Simple register bus	Streaming data	Memory-mapped burst
Channels	1 (address + data)	1 (data stream)	5 (AW, W, B, AR, R)
Request Generation	Address range decode	TDEST decode	Address range decode
Arbitration	Per-slave, per-cycle	Per-slave, packet-locked	Per-slave, per-address-phase
Out-of-Order	No (sequential)	No (streaming order)	No (in-order, `bridge_id` routed)
Burst Support	No	Packet (via TLAST)	Yes (AWLEN/ARLEN)
Complexity	Low	Medium	**High**
Latency	1-2 cycles	2 cycles	2 cycles each way (measured; HAS Table 5.7)
Use Case	Control registers	Data streaming	Memory-mapped I/O

Bridge Complexity Sources: 1. **5 independent channels**, of which TWO are arbitrated. The crossbar instantiates a round-robin arbiter per slave for AW and AR only; W, R and B follow the selection recorded at the address handshake, so they are routed, not arbitrated. 2. **`bridge_id` sideband routing**, in-order (the AXI ID is not used for routing) 3. **Burst handling** with interleaving constraints 4. **Write response tracking** (match AW with B channel) 5. **Address decode** + **ID muxing** for response routing

3.5.2 2.2 Block Diagram





3.5.3 2.3 Key Components

1. Request Generation - Address range decode for each slave - Generates M×S request matrix per channel (AW, AR) - Similar to APB but more complex (2 address channels)

2. Per-Slave Arbitration - **2 arbiters per slave**, not five: - AW channel arbiter (write address) - AR channel arbiter (read address) - W has no arbiter – a W-owner FIFO gives the channel to whichever master's AW the slave accepted first. - B and R have no arbiter – they are muxed combinationally on the `bridge_id` carried alongside the response. - Round-robin, grant held to the ADDRESS handshake – NOT to xlast. The generated arbiters read `lock until handshake`. - Separate read/write paths (no head-of-line blocking)

3. Data Multiplexing - Mux master signals to selected slave - ID-based response routing (B, R channels) - Burst tracking (grant held to the ADDRESS handshake, not to xlast – the generated arbiters read `lock until handshake`; see FR-6)

4. Transaction Tracking - In-order `bridge_id` FIFO per slave adapter (`Bridge ID Tracking - FIFO Mode (In-Order)`), popped in AW/AR order - Track the originating master in a per-slave in-order FIFO, popped on the response. There is no {Master ID, Transaction ID} map – routing is by FIFO POSITION and the AXI ID is not consulted. - Required for routing B/R channels back to correct master - ID tables for OUT-OF-ORDER support are not implemented: `bridge_cam.sv` exists but is instantiated in zero generated bridges

5. ~~Optional Performance Counters~~ – NOT implemented, and no config key enables them - Transaction counts per master/slave - Arbitration conflict counts - Latency histograms

---

3.6 3. Functional Requirements

3.6.1 3.1 AXI4 Protocol Compliance

FR-1: Full AXI4 Protocol Support - Support all 5 AXI4 channels: AW, W, B, AR, R - Comply with AMBA AXI4 specification (ARM IHI 0022) - Support burst lengths: 1-256 beats (AWLEN/ARLEN = 0-255) - Support burst types: INCR, WRAP, FIXED - Support burst sizes: 1-128 bytes (AWSIZE/ARSIZE = 0-7)

FR-2: Response Routing (in-order) - Responses are routed by the `bridge_id` sideband, a static per-master tag, popped from an in-order FIFO. The AXI ID plays no part in routing. - Master IDs pass through unchanged – there is no ID extension. `cpu_adapter` drives `cpu_32b_aw.id = fub_axi_awid`, 4 bits in, 4 bits out. - **Constraint this imposes:** the slave must return responses in the order its requests were accepted. A reordering slave misroutes – the FIFO head names the wrong master, so one master receives another's beats and the second starves. Nothing detects this. - Two masters never alias an ID at a slave: inside the fabric every ID is `{master index, master id}` (BRIDGE-016), so a slave port carries the widest master's id_width plus `$clog2(NUM_MASTERS)` bits – 8-bit masters behind a 16-master bridge give 12-bit IDs at the slaves. The `bridge_id` sideband still routes responses; the prefix is what makes per-ID tracking sound across masters, and every real AXI slave of a multi-master bridge is tracked by ID in `bridge_cam` (BRIDGE-015). Single-master bridges add no prefix and keep the in-order FIFO.

FR-3: Atomic Operations (the bridge does NOT implement a monitor – see below) - Pass AWLOCK/ARLOCK through to the slave. **The bridge implements no exclusive monitor**: there is no per-slave exclusive tracking and no EXOKAY generation anywhere in the generated RTL – `awlock` is muxed to the granted slave and nothing more. Exclusives work only if the attached slave implements the monitor itself. 13.1 correctly lists the monitor as future work; this requirement previously read as though the bridge provided it. - ~~Track exclusive monitor per slave~~ – NOT implemented; see the corrected bullet above - ~~Generate BRESP/RRESP errors for failed exclusives~~ – NOT implemented

3.6.2 3.2 Address Decoding

FR-4: Configurable Address Map - Support M masters × S slaves - Each slave has base address and size - Non-overlapping address ranges (verified at generation) - Default slave for unmapped addresses (optional)

FR-5: Address Range Configuration

```
# Example address map
address_map = {
    0: {'base': 0x00000000, 'size': 0x40000000, 'name': 'DDR'},      #
1GB
    1: {'base': 0x40000000, 'size': 0x00100000, 'name': 'SRAM'},     #
1MB
    2: {'base': 0x50000000, 'size': 0x01000000, 'name': 'PCIE'},     #
16MB
    3: {'base': 0x60000000, 'size': 0x00010000, 'name': 'Peripherals'}
```

```
# 64KB
}
```

3.6.3 3.3 Arbitration Strategy

FR-6: Round-Robin Arbitration - Fair bandwidth allocation (no starvation) - Separate arbiters for AW and AR channels - Grant is locked only until the ADDRESS handshake (`awvalid && awready`), not until xlast. The RTL comment reads “lock until handshake”. Back-to-back AWs from different masters can therefore be accepted and their W bursts sequenced by the W-owner FIFO. - Round-robin arbitration, HARD-CODED. There is no TOML key and no parameter to change it; “configurable” was aspirational.

FR-7: Read/Write Independence - Separate read and write paths - No head-of-line blocking between read/write - Concurrent read and write to same slave (if slave supports)

3.6.4 3.4 Burst Handling

FR-8: Burst Optimization - Pipelined burst transfers (overlap address and data) - No artificial burst splitting - Full AXI4 burst protocol support

FR-9: Interleaving Constraints - W channel follows the AW owner via a per-slave W-owner FIFO (not a held grant) - R channel routed by in-order `bridge_id` FIFO position, NOT by transaction ID. IDs are pass-through: the slave port is the same width as the master port. `bridge_cam.sv` exists but is instantiated in zero generated bridges. - ID-based interleaving is NOT supported. Each slave port must return B/R in request order across ALL IDs – see BRIDGE-010.

3.7 4. Non-Functional Requirements

3.7.1 4.1 Performance

NFR-1: Latency - **Single-beat read:** ≤ 3 cycles (address decode + arbitration + mux) - **Single-beat write:** ≤ 3 cycles (address decode + arbitration + mux) - **Burst transfer:** No additional latency per beat (pipelined)

NFR-2: Throughput - **Concurrent transfers:** different masters to different slaves, yes; ONE master to several slaves, no – see 1.3 - **Burst efficiency:** Line-rate data transfer after address phase - **No artificial stalls:** Crossbar adds no wait states beyond arbitration

NFR-3: Fmax - **Target:** 300-400 MHz on Xilinx UltraScale+ - **Registered outputs:** All slave outputs registered for timing closure - **Pipelineable:** Optional pipeline stages for >400 MHz

3.7.2 4.2 Resource Usage (Estimated)

M = 4 masters, S = 4 slaves, DATA_WIDTH = 512, ADDR_WIDTH = 64, ID_WIDTH = 4:

Resource	Flat Crossbar	Notes
LUTs	~2,500	Address decode + arbiters + mux
FFs	~3,000 (hand estimate, unverified)	Registered outputs + the per-slave bridge_id FIFOs
BRAM	0	No ID tables exist. Tracking is a small in-order FIFO per direction in each slave adapter.
DSP	0	No arithmetic operations

Scaling: ~150 LUTs per M×S connection

3.7.3 4.3 Quality Requirements

NFR-4: Code Generation Quality - Lint-clean SystemVerilog (Verilator) - Synthesizable (Vivado, Yosys, Design Compiler) - No vendor-specific primitives (technology-agnostic) - Clear structure (commented, readable)

NFR-5: Verification - CocoTB testbench framework - Transaction-level verification - ~~Out-of-order test scenarios~~ – not applicable; ordering is structural (see FR-9) - Burst interleaving tests - >95% functional coverage

---

3.8 5. Interface Specifications

3.8.1 5.1 AXI4 Master Interfaces (M × 5 channels)

Write Address Channel (AW):

```
input  logic [ADDR_WIDTH-1:0]   s_axi_awaddr  [NUM_MASTERS];
input  logic [ID_WIDTH-1:0]     s_axi_awid    [NUM_MASTERS];
input  logic [7:0]              s_axi_awlen   [NUM_MASTERS];  // Burst
length-1
input  logic [2:0]              s_axi_awsize  [NUM_MASTERS];  // Burst
size
input  logic [1:0]              s_axi_awburst [NUM_MASTERS];  //
INCR/WRAP/FIXED
input  logic                    s_axi_awlock  [NUM_MASTERS];  //
Exclusive access
input  logic [3:0]              s_axi_awcache [NUM_MASTERS];  // Cache
attributes
input  logic [2:0]              s_axi_awprot  [NUM_MASTERS];  //
Protection type
input  logic [3:0]              s_axi_awqos   [NUM_MASTERS];  // QoS
-- routed, not arbitrated on
input  logic [3:0]              s_axi_awregion[NUM_MASTERS];  //
Region -- passed through
input  logic [UW-1:0]           s_axi_awuser  [NUM_MASTERS];  // USER
-- passed through
input  logic                    s_axi_awvalid [NUM_MASTERS];
output logic                    s_axi_awready [NUM_MASTERS];
```

Write Data Channel (W):

```
input  logic [DATA_WIDTH-1:0]   s_axi_wdata  [NUM_MASTERS];
input  logic [DATA_WIDTH/8-1:0] s_axi_wstrb  [NUM_MASTERS];  // Byte
strobes
input  logic                    s_axi_wlast  [NUM_MASTERS];  // Last
beat
input  logic [UW-1:0]           s_axi_wuser  [NUM_MASTERS];  // USER
-- passed through
input  logic                    s_axi_wvalid [NUM_MASTERS];
output logic                    s_axi_wready [NUM_MASTERS];
```

Write Response Channel (B):

```
output logic [ID_WIDTH-1:0]     s_axi_bid    [NUM_MASTERS];
output logic [1:0]              s_axi_bresp  [NUM_MASTERS];  //
OKAY/SLVERR/DECERR (EXOKAY never generated -- no exclusive monitor)
output logic [UW-1:0]           s_axi_buser  [NUM_MASTERS];  // USER
-- returned to the master
```

```
output logic                    s_axi_bvalid [NUM_MASTERS];
input  logic                    s_axi_bready [NUM_MASTERS];
```

Read Address Channel (AR):

```
input  logic [ADDR_WIDTH-1:0]   s_axi_araddr  [NUM_MASTERS];
input  logic [ID_WIDTH-1:0]     s_axi_arid    [NUM_MASTERS];
input  logic [7:0]              s_axi_arlen   [NUM_MASTERS];
input  logic [2:0]              s_axi_arsize  [NUM_MASTERS];
input  logic [1:0]              s_axi_arburst [NUM_MASTERS];
input  logic                    s_axi_arlock  [NUM_MASTERS];
input  logic [3:0]              s_axi_arcache [NUM_MASTERS];
input  logic [2:0]              s_axi_arprot  [NUM_MASTERS];
input  logic [3:0]              s_axi_arqos   [NUM_MASTERS];  // QoS
-- routed, not arbitrated on
input  logic [3:0]              s_axi_arregion[NUM_MASTERS];  //
Region -- passed through
input  logic [UW-1:0]           s_axi_aruser  [NUM_MASTERS];  // USER
-- passed through
input  logic                    s_axi_arvalid [NUM_MASTERS];
output logic                    s_axi_arready [NUM_MASTERS];
```

Read Data Channel (R):

```
output logic [DATA_WIDTH-1:0]   s_axi_rdata  [NUM_MASTERS];
output logic [ID_WIDTH-1:0]     s_axi_rid    [NUM_MASTERS];
output logic [1:0]              s_axi_rresp  [NUM_MASTERS];
output logic                    s_axi_rlast  [NUM_MASTERS];
output logic [UW-1:0]           s_axi_ruser  [NUM_MASTERS];  // USER
-- returned to the master
output logic                    s_axi_rvalid [NUM_MASTERS];
input  logic                    s_axi_rready [NUM_MASTERS];
```

3.8.2 5.2 AXI4 Slave Interfaces (S × 5 channels)

Mirror of master interfaces, with M → S direction reversed.

3.8.3 5.3 Configuration Is Generation-Time, Not Parameters

The generated bridge exposes no parameters. Everything below is fixed when the generator runs and reaches the module through its package:

```
module bridge_2x2_rw
    import bridge_2x2_rw_pkg::*;
(
    input logic aclk, ...
```

Geometry and widths come from the TOML, per port, and appear inside the generated modules as localparams (`localparam ADDR_WIDTH = 32;`, `localparam ID_WIDTH = 4;` in `cpu_adapter`). To change any of them, regenerate – there is nothing to override at elaboration.

An earlier revision of this section listed a parameter block including `MAX_BURST_LEN`, `PIPELINE_OUTPUTS` and `ENABLE_COUNTERS`. **None of those three exist in any generated module**, and neither do the performance counters that `ENABLE_COUNTERS` implied – there is no counter logic anywhere in the generated set.

3.9 6. Performance Modeling

3.9.1 6.1 Analytical Model

Latency Components (Flat Crossbar):

```
Single-Beat Read Latency:
  1. Address Decode:      0 cycles (combinatorial)
  2. Arbitration:         1 cycle  (AR arbiter decision)
  3. Address Mux:         0 cycles (combinatorial)
  4. Slave Access:        (slave-dependent, not crossbar)
  5. Response Mux:        1 cycle  (R channel ID lookup + mux)
  6. Output Register:     1 cycle  (optional, for timing closure)

  Total (no pipeline):    2 cycles
  Total (pipelined):      3 cycles
```

Burst Transfer Throughput:

```
After address phase completes, data transfer is line-rate:
  - W channel: 1 beat/cycle (routed by the W-owner FIFO)
  - R channel: 1 beat/cycle (ID-routed from slave)

Example: 256-beat burst
  Address phase:  2 cycles (measured, one-time)
  Data phase:     256 cycles (line-rate)
  Total:          258-259 cycles
  Efficiency:     98.8%
```

Concurrent Throughput:

```
Maximum concurrent transfers (4×4 crossbar):
  - Read:  4 concurrent (one per master-slave pair)
  - Write: 4 concurrent (one per master-slave pair)
  - Total: 8 concurrent read+write (separate paths)

Throughput @ 100 MHz, 512-bit data:
  Per path:   100 MHz × 512 bits = 51.2 Gbps
  Total:      8 paths × 51.2 Gbps = 409.6 Gbps theoretical
```

3.9.2 6.2 Resource Scaling

LUT Usage Formula (empirical):

```
LUTs ≈ 500 (base) + 150 × M × S + 20 × FIFO_DEPTH × BRIDGE_ID_WIDTH
```

Example (4×4 crossbar, ID_WIDTH=4, bridge_id FIFO depth 16 per slave):

```
LUTs ≈ 500 + 150 × 16 + 20 × 16 × 4
     ≈ 500 + 2,400 + 1,280
     ≈ 4,180 LUTs
```

3.9.3 6.3 Comparison with Other Crossbars

Crossbar Type	Latency	Throughput	Complexity	Use Case
APB	1-2 cycles	Low (serialized)	Low	Control registers
AXI-Stream (Delta)	2 cycles	High (streaming)	Medium	Data streaming
AXI4 (Bridge)	2 cycles each way	High (burst)	**High**	Memory-mapped I/O
AXI4 + Slices	4-6 cycles	High (burst)	Very High	>400 MHz designs

Bridge Sweet Spot: memory-mapped interconnects that need burst efficiency and mixed protocols. NOT a fit where out-of-order completion matters – the fabric is in-order by construction (FR-9, BRIDGE-010).

3.10 7. Generator Architecture

3.10.1 7.1 Python Generator Structure

```
class BridgeGenerator:
    """AXI4 crossbar RTL generator"""

    def __init__(self, config):
        self.num_masters = config.num_masters
        self.num_slaves = config.num_slaves
        self.data_width = config.data_width
        self.addr_width = config.addr_width
        self.id_width = config.id_width
        self.address_map = config.address_map

    def generate_address_decode(self) -> str:
        """Generate address range decode logic"""
        # For each master × slave, check if address in range
        # More complex than AXIS (uses address), simpler than full
decode

    def generate_aw_arbiter(self, slave_idx) -> str:
        """Generate write address channel arbiter for one slave"""
        # Round-robin arbiter
        # Grant held to the ADDRESS handshake, not to the B response.
        # (This pseudocode described an abandoned lock-until-response
design.)

    def generate_ar_arbiter(self, slave_idx) -> str:
        """Generate read address channel arbiter for one slave"""
        # Round-robin arbiter
        # Grant held to the ADDRESS handshake, not to RLAST.

    def generate_w_channel_mux(self, slave_idx) -> str:
        """Generate write data channel multiplexer"""
        # W channel follows the AW owner recorded in the per-slave W-
owner
        # FIFO. The arbiter grant itself released at the ADDRESS
handshake.

    def generate_b_channel_demux(self, slave_idx) -> str:
        """Generate write response channel demultiplexer"""
        # Route by FIFO POSITION: pop the per-slave bridge_id FIFO.
        # The returned BID is NOT consulted (see 4.2).

    def generate_r_channel_demux(self, slave_idx) -> str:
```

```
        """Generate read data channel demultiplexer"""
        # Route by FIFO POSITION: pop the per-slave bridge_id FIFO on
RLAST.
        # The returned RID is NOT consulted (see 4.2).

    def generate_bridge_id_fifo(self, slave_idx) -> str:
        """Generate transaction ID tracking table"""
        # Maps {slave, transaction_id} → master_id
        # Indexed on AW/AR grant, looked up on B/R response

    def generate_crossbar(self) -> str:
        """Generate complete crossbar module"""
        # Instantiate all arbiters, muxes, demuxes and the per-slave
        # bridge_id FIFOs (there are no ID tables)
```

3.10.2 7.2 Address Map Configuration

Python Configuration:

```
bridge_config = {
    'num_masters': 4,
    'num_slaves': 4,
    'data_width': 512,
    'addr_width': 64,
    'id_width': 4,
    'address_map': {
        0: {'base': 0x00000000, 'size': 0x40000000, 'name': 'DDR'},
        1: {'base': 0x40000000, 'size': 0x00100000, 'name': 'SRAM'},
        2: {'base': 0x50000000, 'size': 0x01000000, 'name': 'PCIE'},
        3: {'base': 0x60000000, 'size': 0x00010000, 'name':
'Peripherals'}
    },
    # NOTE: 'pipeline_outputs' and 'enable_counters' are NOT real
config
    # keys -- config_validator rejects unknown keys, and 5.3 says the
    # counters do not exist. Removed from this example rather than
left
    # for someone to copy.
}
```

Generated Address Decode:

```
// Address decode for each master
always_comb begin
    for (int s = 0; s < NUM_SLAVES; s++)
        aw_request_matrix[s] = '0;

    for (int m = 0; m < NUM_MASTERS; m++) begin
```

```
        if (s_axi_awvalid[m]) begin
            // Slave 0: DDR (0x00000000 - 0x3FFFFFFF)
            if (s_axi_awaddr[m] >= 64'h00000000 &&
                s_axi_awaddr[m] < 64'h40000000)
                aw_request_matrix[0][m] = 1'b1;

            // Slave 1: SRAM (0x40000000 - 0x400FFFFF)
            if (s_axi_awaddr[m] >= 64'h40000000 &&
                s_axi_awaddr[m] < 64'h40100000)
                aw_request_matrix[1][m] = 1'b1;

            // ... slaves 2-3 ...
        end
    end
end
```

3.11 8. Comparison with APB and Delta

3.11.1 8.1 Code Reuse from APB Generator

Similar Components (~70% reuse): - Address range decode logic (same pattern, different signals) - Round-robin arbiter (same algorithm) - Data multiplexing pattern (same approach) - Backpressure handling (similar to PREADY)

New Components for Bridge: - **2 arbiters per slave** (AW and AR). W is owned via a FIFO, B and R are muxed combinationally – no arbiter on any of the three. - **`bridge_id` sideband routing** (B and R channel demuxing) - **In-order transaction tracking** (a FIFO per direction, not an ID table) - **Burst handling** (grant locked to the ADDRESS handshake, not xlast)

Migration Effort from APB: - ~120 minutes (vs ~75 min for AXIS, due to higher complexity) - Most time: response demuxing and the bridge_id FIFOs

3.11.2 8.2 Code Reuse from Delta Generator

Similar Components (~60% reuse): - Python generation framework - Command-line interface - Performance modeling structure - Arbitration pattern (round-robin with locking)

Key Differences: - **5 channels** vs 1 channel (Delta only has TDATA/TVALID/TREADY/TLAST) - **`bridge_id` sideband routing** vs TDEST-based routing - **Address decode** vs TDEST decode (Bridge more complex) - **Transaction tracking** vs packet atomicity (different mechanisms)

3.11.3 8.3 Complexity Comparison

Metric	APB Crossbar	Delta (AXIS)	Bridge (AXI4)
Channels to arbitrate	1	1	5 ★
Request generation	Address ranges	TDEST decode	Address ranges
Response routing	Grant-based	Grant-based	ID-based ★
Burst support	No	Packet (TLAST)	Yes (AWLEN/ARLEN) ★
Out-of-order	No	No	No – in-order `bridge_id` FIFO (`bridge_cam` unused)
Transaction	No	No	Yes ★

Metric	APB Crossbar	Delta (AXIS)	Bridge (AXI4)
tracking			
Lines of Python	~500	~697	**~900** (est.)
Lines of generated SV	~200 (4×4)	~250 (4×4)	**~400** (4×4) (est.)

★ = Additional complexity in Bridge

3.12 9. Use Cases

3.12.1 9.1 Multi-Core Processor Interconnect

Scenario: 4 CPU cores + GPU accessing DDR + SRAM + Peripherals

```
Configuration:
  Masters:  5 (4 CPUs, 1 GPU)
  Slaves:   3 (DDR, SRAM, Peripherals)
  Data:     512-bit (cache line width)
  Address:  64-bit (large memory space)
  ID:       4-bit (up to 16 outstanding per master)
```

Benefits: - Concurrent access to all slaves - in-order completion (see FR-9 – out-of-order is NOT implemented) - Burst transfers for cache line fills - Separate read/write paths (no head-of-line blocking)

3.12.2 9.2 DMA + Accelerator System

Scenario: DMA engine + 4 accelerators accessing shared memory

```
Configuration:
  Masters:  5 (1 DMA, 4 accelerators)
  Slaves:   2 (DDR, Control registers)
  Data:     512-bit (high-bandwidth DMA)
  Address:  32-bit (limited address space)
  ID:       2-bit (simple ID space)
```

Benefits: - High-bandwidth memory access for DMA - Fair arbitration prevents accelerator starvation - Control register access doesn't block data transfers

3.12.3 9.3 FPGA System Integration

Scenario: MicroBlaze CPU + custom accelerators + memory controllers

```
Configuration:
  Masters:  8 (1 CPU, 7 accelerators)
  Slaves:   4 (DDR, BRAM, AXI GPIO, AXI DMA)
  Data:     128-bit (AXI4 standard width)
  Address:  32-bit (standard FPGA address space)
  ID:       4-bit (moderate outstanding transactions)
```

Benefits: - Standard AXI4 interfaces (Xilinx IP compatibility) - Scalable to many masters/slaves - Performance counters for profiling. **NOT IMPLEMENTED** – no counter logic exists in any generated module.

3.13 10. Testing Strategy

3.13.1 10.1 FUB (Functional Unit Block) Tests

Address Decode Tests: - Verify all address ranges correctly decoded - Test boundary conditions (base, base+size-1) - Test unmapped addresses (error response)

Arbiter Tests: - Round-robin fairness (all masters get turns) - Grant locking to the ADDRESS handshake (not xlast – see FR-6) - Starvation prevention

bridge_id FIFO Tests: - Correct ID → master mapping - ~~Out-of-order transaction handling~~ – not implemented; see FR-9 - Table full condition

Mux/Demux Tests: - Data integrity through crossbar - Response routing to correct master - Concurrent transfers don't interfere

3.13.2 10.2 Integration Tests

Single-Master, Single-Slave: - Basic read/write transactions - Burst transfers (various lengths) - ~~Out-of-order completions~~ – not applicable (in-order fabric)

Multi-Master, Single-Slave: - Arbitration correctness - Fairness verification - Burst interleaving (if supported)

Multi-Master, Multi-Slave: - Concurrent transfers - No crosstalk between paths - Full M×S matrix coverage

Stress Tests: - All masters active simultaneously - Maximum burst lengths - Full ID space utilization - Back-to-back bursts

3.13.3 10.3 Performance Validation

Latency Measurement: - Single-beat read: measure actual vs analytical - Single-beat write: measure actual vs analytical - Compare with specification (≤3 cycles)

Throughput Measurement: - Burst transfer efficiency (should be ~98%) - Concurrent path throughput (should be line-rate × M×S) - Compare with theoretical maximum

Resource Validation: - Synthesize for Xilinx UltraScale+ - Compare LUT/FF usage with estimates - Verify Fmax ≥ 300 MHz

---

3.14 11. Documentation Plan

3.14.1 11.1 Specifications (Before Code)

- ☒ **PRD.md** - This document (complete requirements)
- ☐ **README.md** - User guide with quick start
- ☐ **BRIDGE_VS_APB_GENERATOR.md** - Migration guide from APB
- ☐ **BRIDGE_VS_DELTA_GENERATOR.md** - Comparison with Delta
- ☐ **AXI4_PROTOCOL_GUIDE.md** - AXI4 primer for students

3.14.2 11.2 Performance Analysis (Before Implementation)

- ☒ **models/bridge_model/bridge_model.py** - Analytical model (V1 Flat implemented)
  - Latency calculations (single-beat, burst)
  - Throughput estimates (concurrent paths)
  - Resource estimates (LUT/FF scaling)
- ☐ **bridge simulator** - Discrete event simulation (optional, not implemented)
  - Cycle-accurate modeling
  - Traffic pattern support
  - Validation against RTL

3.14.3 11.3 Code Generation

- ☐ **bin/bridge_generator.py** - Main RTL generator
  - Command-line interface
  - Address map configuration
  - Generated RTL output

3.14.4 11.4 Verification

- ☐ **dv/tests/** - CocoTB testbenches
  - FUB tests (individual blocks)
  - Integration tests (multi-block)
  - Stress tests (corner cases)

3.15 12. Success Metrics

3.15.1 12.1 Functional Completeness

- ☐ Generates working AXI4 crossbar RTL
- ☐ Passes all CocoTB tests
- ☐ Verilator lint clean
- ☐ Xilinx Vivado synthesis clean

3.15.2 12.2 Performance Targets

- ☐ Latency ≤ 3 cycles (measured in simulation)
- ☐ Throughput = line-rate × M×S paths (measured)
- ☐ Fmax ≥ 300 MHz (post-synthesis)

3.15.3 12.3 Educational Value

- ☐ Complete specifications demonstrating rigor
- ☐ Performance models validate requirements
- ☐ Clear code structure (readable generated RTL)
- ☐ Integration examples provided

3.15.4 12.4 Reusability

- ☐ ~70% code reuse from APB generator (measured by LOC)
- ☐ Similar patterns to Delta generator
- ☐ Can be extended (weighted arbitration, QoS, etc.)

3.16 12. Attribution and Contribution Guidelines

3.16.1 12.1 Git Commit Attribution

When creating git commits for Bridge documentation or implementation:

Use:

```
Documentation and implementation support by Claude.
```

Do NOT use:

```
Co-Authored-By: Claude <noreply@anthropic.com>
```

Rationale: Bridge documentation and organization receives AI assistance for structure and clarity, while design concepts and architectural decisions remain human-authored.

3.17 12.2 PDF Generation Location

IMPORTANT: PDF files should be generated in the docs directory:

`projects/components/bridge/docs/`

Quick Commands: Use the provided shell scripts:

```
cd projects/components/bridge/docs
./generate_has_pdf.sh   # Bridge_HAS_v<rev>.docx/.pdf
./generate_mas_pdf.sh   # Bridge_MAS_v<rev>.docx/.pdf
```

The shell scripts will automatically: 1. Use the md_to_docx.py tool from bin/ 2. Process the bridge_has/bridge_mas index files 3. Generate both DOCX and PDF files in the docs/ directory 4. Create table of contents and title page

See: `bin/md_to_docx.py` for complete implementation details

3.18 13. Future Enhancements

3.18.1 13.1 Short-Term (Post-Initial Release)

- ☐ **Optional pipeline stages** - For Fmax >400 MHz
- ☐ **Weighted arbitration** - QoS support
- ☒ **Default slave** - unmapped address handling. Built 2026-09-07: a subtractive catch-all answers DECERR + 0xDEADBEEF, with a sticky status/IRQ cleared by the `unmapped_clear` INPUT PIN. There is no APB access to it – the pin is the whole interface, and wiring it to a register is the integrator's job. See HAS 4.5 and BRIDGE-009.
- ☐ **Exclusive monitor** - Full atomic operation support

3.18.2 13.2 Long-Term

- ☐ **Tree topology** - Hierarchical crossbar (like Delta)
- ☐ **AXI4-Lite variant** - Simplified for control registers
- ☐ **ACE support** - Coherent cache interconnect
- ☐ **GUI configurator** - Visual address map setup

3.19 14. Risk Assessment

Risk	Probability	Impact	Mitigation
~~ID table complexity~~	–	–	Moot: no ID tables were built. The realised risk was the opposite – the DOCS described tables for months while the RTL used an in-order FIFO.
Response-ordering assumption	High	High	The fabric REQUIRES each slave to return B/R in request order across IDs and does not check it. Assert on a returned BID/RID that does not match the FIFO head – see BRIDGE-010.
Fmax below target	Low	Medium	Optional pipeline stages for timing closure
Resource usage exceeds	Low	Low	Empirical formulas guide expectations
Burst interleaving bugs	Medium	High	Separate test suite for burst scenarios

3.20 15. Project Timeline (Estimated)

Week 1: Specifications and Models - [x] Day 1-2: PRD.md (complete) - [ ] Day 3-4: Performance modeling (analytical) - [ ] Day 5-7: README.md, migration guides

Week 2-3: Core Implementation - [ ] Day 1-3: Address decode + arbiter generation - [ ] Day 4-5: Data mux/demux generation - [x] Day 6-8: response tracking (delivered as a per-slave in-order bridge_id FIFO, not ID tables) - [ ] Day 9-10: Integration and testing

Week 4: Verification and Examples - [ ] Day 1-5: CocoTB testbenches - [ ] Day 6-7: Integration examples

---

3.21 16. References

AXI4 Specifications: - ARM IHI 0022 - AMBA AXI and ACE Protocol Specification - Xilinx UG1037 - Vivado AXI Reference Guide

Related Projects: - **APB Crossbar** - Simple register bus crossbar (existing) - **Delta (AXIS Crossbar)** - Streaming data crossbar (projects/components/delta/) - **RAPIDS** - DMA engine with AXI4 masters (projects/components/dmas/rapids/)

Tools: - Verilator - RTL linting and simulation - CocoTB - Python-based verification - Xilinx Vivado - FPGA synthesis

---

3.22 17. Glossary

- **AXI4:** Advanced eXtensible Interface version 4 (AMBA standard)
- **Burst:** Multi-beat transaction (AWLEN/ARLEN > 0)
- **Crossbar:** Full M×N interconnect matrix
- **ID:** AXI transaction identifier. Passed through this bridge unchanged; it is NOT used for response routing here.
- **Out-of-order:** responses returning in a different order than requested. AXI4 permits it between IDs; **this bridge does not support it** and does not detect a slave that does (BRIDGE-010).
- **xlast:** WLAST (write) or RLAST (read) - last beat indicator

---

Version: 2.1 **Status:** Specification Complete - Ready for Implementation **Next Steps:** Create performance models, then implement generator

Project Bridge - Connecting masters and slaves across the divide

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

4 Claude Code Guide: Bridge Subsystem

Version: 2.2 **Last Updated:** 2026-09-07 **Purpose:** AI-specific guidance for working with Bridge subsystem

---

4.1 CRITICAL: Read Architecture Documents First

Before making ANY changes to bridge generator or understanding signal flow:

READ: `projects/components/bridge/GENERATOR_ARCHITECTURE.md` (generator structure) and `projects/components/bridge/docs/bridge_mas/` (micro-architecture spec; rendered as `docs/Bridge_MAS_v1.1.pdf`)

These documents contain the **definitive bridge architecture** including: - Correct signal flow (wrappers → decoder → converters → crossbar → slaves) - Component purposes and placement - Common misconceptions that previous agents made - Why there's NO fixed crossbar width

If you skip these documents, you WILL make incorrect assumptions.

4.2 Quick Context

What: Bridge - one AXI4/AXI5 crossbar generator, `bin/bridge_generator.py`, taking TOML or CSV configuration. There is no second generator: the separate `bridge_csv_generator.py` was merged into it and no longer exists. **Status:** Phase 2 Complete - CSV generator with channel-specific masters (wr/rd/rw) **Your Role:** Help users configure CSV files, generate bridges, understand architecture, and create tests

Complete Documentation (Read in This Order): 1. `projects/components/bridge/GENERATOR_ARCHITECTURE.md` ← **START HERE** (generator architecture reference) 2. `projects/components/bridge/PRD.md` ← Product requirements 3. `vault/Tasks/bridge/INDEX.md` ← Task tracking (open / closed / dropped; the old TASKS.md was folded in 2026-09-10) 4. `projects/components/bridge/docs/bridge_has/bridge_has_index.md` ← Hardware Architecture Spec (rendered: `docs/Bridge_HAS_v1.1.pdf`) 5. `projects/components/bridge/docs/bridge_mas/bridge_mas_index.md` ← Micro-Architecture Spec (rendered: `docs/Bridge_MAS_v1.1.pdf`)

(The older BRIDGE_ARCHITECTURE.md, BRIDGE_CURRENT_STATE.md, BRIDGE_ARCHITECTURE_DIAGRAMS.md, CSV_BRIDGE_STATUS.md, and docs/bridge_spec/ documents were superseded by the HAS/MAS above.)

4.3 Target Architecture: Intelligent Width-Aware Routing

Core Principle: Direct connections where possible, converters only where needed, no fixed crossbar width.

4.3.1 Efficient Multi-Width Design

```
Master_A (64b)
  ├─ Direct → Slave_0 (64b)           [0 conversions, minimal latency]
  ├─ Conv(64→128) → Slave_1 (128b)    [1 conversion]
  └─ Conv(64→512) → Slave_2 (512b)    [1 conversion]

Master_B (512b)
  ├─ Conv(512→64) → Slave_0 (64b)     [1 conversion]
  ├─ Conv(512→128) → Slave_1 (128b)   [1 conversion]
  └─ Direct → Slave_2 (512b)          [0 conversions, full bandwidth]
```

Router Logic: Address decoder determines target slave, selects correctly-sized path for that master-slave pair.

4.3.2 Why This Architecture

Naive Fixed-Width Approach (Don't Do This):

```
Master (64b) → Upsize(64→256) → Fixed 256b Crossbar → Downsize(256→64)
→ Slave (64b)
```

- TWO conversions for same-width connections (wasteful!)
- Reduced bandwidth on narrow paths
- Unnecessary logic and area
- Higher latency

Intelligent Routing (Target):

```
Master (64b) → Router → Direct Connection → Slave (64b)
```

- ZERO conversions for matching widths
- Full native bandwidth
- Minimal logic
- Lowest latency

4.3.3 Per-Master Output Paths

Each master has N output paths (one per unique slave width it connects to):

```
// Master_A connects to slaves at 64b, 128b, 512b
// Generate 3 output paths:
```

```
logic [63:0]  master_a_64b_wdata;   // For 64b slaves (direct)
logic [127:0] master_a_128b_wdata;  // For 128b slaves (via converter)
logic [511:0] master_a_512b_wdata;  // For 512b slaves (via converter)

// Router selects based on address decode:
always_comb begin
    case (decoded_slave_id)
        SLAVE_0: select master_a_64b_wdata;   // Slave_0 is 64b
        SLAVE_1: select master_a_128b_wdata;  // Slave_1 is 128b
        SLAVE_2: select master_a_512b_wdata;  // Slave_2 is 512b
    endcase
end
```

4.3.4 Benefits

1. **Resource Efficient** - Only instantiate converters actually needed
2. **Maximum Performance** - Direct paths have zero conversion overhead
3. **Optimal Bandwidth** - No artificial width bottlenecks
4. **Lower Latency** - Minimal logic in critical path for matching widths
5. **Scalable** - Works for any combination of master/slave widths

4.3.5 Implementation Status

Current State: Intelligent per-master multi-width routing – DELIVERED. `cpu_master_adapter.sv` in `bridge_4x4_rw` carries separate 32b, 64b, 128b and 256b paths and converts only where a master and its slave disagree. There is no fixed internal crossbar width.

This block described the Phase-1 fixed-width crossbar as current and the per-width routing as a future migration “requiring generator architecture rework”. That rework has happened; the block outlived it.

4.4 CRITICAL RULE #0: RTL Regeneration Requirements

READ THIS FIRST - FAILURE TO FOLLOW CAUSES TEST FAILURES

4.4.1 The Golden Rule

ALL generated RTL files MUST be deleted and regenerated together whenever ANY generator code changes.

4.4.2 Why This Is Non-Negotiable

Generated RTL files have interdependencies: - Each `rtl/generated/<bridge_name>/` directory holds a matched set (top module, crossbar, adapters, package) - Generator changes affect signal names, port widths, interface structure - **Partial regeneration creates version mismatches that cause silent failures**

4.4.3 Real Example (Historical Session)

```
# WHAT WENT WRONG:
1. Updated the address decode logic in the generator
2. Regenerated ONLY one bridge configuration
3. Did NOT regenerate the wrapper that instantiated it
4. Result: All tests that were passing now FAIL
5. Cause: Version mismatch between wrapper and core bridge

# WHAT SHOULD HAVE BEEN DONE:
1. Update the generator
2. Delete ALL generated bridge directories (rtl/generated/bridge_*/)
3. Regenerate ALL bridges from scratch (make all in bin/)
4. Run ALL tests to verify
```

4.4.4 Generator Files That Trigger Full Regeneration

Any change to these files requires regenerating ALL bridges:

- `bin/bridge_generator.py` - Core bridge generator (CSV/TOML driven)
- **ANY** Python file in `bin/bridge_pkg/` (components/, generators/, config, csv_parser, signal_naming, ...)
- Any Jinja template in `bin/bridge_pkg/jinja_templates/`

4.4.5 The Regeneration Workflow

```
# Step 1: Make generator code changes
vim bin/bridge_pkg/generators/crossbar_generator.py

# Step 2: Delete ALL generated RTL (be aggressive!)
cd projects/components/bridge/bin
make clean   # Removes rtl/generated/ output
```

```
# Step 3: Regenerate everything from bridge_batch.csv
make all     # Runs bridge_generator.py --bulk bridge_batch.csv

# Step 4: Run ALL tests
cd ../dv/tests
pytest -v  # ALL tests, not just the one you think changed

# Step 5: Verify git diff makes sense
git diff ../rtl/  # Review all changes
```

4.4.6 Symptoms of Version Mismatch

If you see these symptoms, you probably did partial regeneration:

- Tests that previously passed now fail
- “Signal not found” errors in simulation
- Port width mismatches in instantiation
- Address decode routing to wrong slaves
- Missing debug signals (dbg_*)
- Unexpected compile errors in working code

4.4.7 Think Like a Compiler Developer

Generated RTL = Compiled Object Files

When you update a compiler, you don’t selectively recompile - you `make clean && make all`.

When you update a generator, you don’t selectively regenerate - you **delete all and regenerate all**.

4.4.8 Exception: Hand-Written RTL

These files are **never** regenerated: - `bridge_cam.sv` - CAM module (hand-written; instantiated in ZERO generated bridges) - Any file in `rtl/` that is NOT generated

Check file headers. Each generated file names the class that emitted it, so there are FIVE strings, not one:

File	Header
`<name>.sv`	`// Generated by: BridgeModuleGenerator`
`<name>_pkg.sv`	`// Generated by: PackageGenerator`
`<name>_xbar.sv`	`// Generated by: CrossbarGenerator`

File	Header
`<master>_adapter.sv`	`// Generated by: AdapterGenerator`
`<slave>_adapter.sv`	`// Generated by: SlaveAdapterGenerator`

“bridge_generator.py” appears in no generated file; matching on it finds nothing, and matching only on BridgeModuleGenerator finds 1 file in 5.

4.5 Critical Pitfalls: slave_select_* Reversion After Handshake

4.5.1 The Problem: Combinational Address Decode

The master adapter's R/B response MUX and the crossbar's AR/AW gating both relied on `<master>_slave_select_{ar,aw}` — a combinational decode of `fub_axi_{ar,aw}addr`. This signal reverts immediately when the address port is freed:

1. AR/AW handshake completes (arvalid && arready, awvalid && awready)
2. Wrapper pops skid buffer and releases the address port
3. fub_axi_araddr / fub_axi_awaddr reverts to next transaction or zero
4. Decode flips to different slave
5. Response MUX is now gated on WRONG slave
6. Response path closes, transaction hangs

This happens BEFORE the converter even finishes pushing to the crossbar, and ALWAYS before response arrives.

4.5.2 Visible Symptoms

- Fails: Multi-slave multi-master bridges hang (timeout)
- Fails: Multi-width masters fail basic connectivity
- Fails: APB/AXIL slaves consistently timeout (longer converter latency = more decode changes)
- Works: Single-slave or single-master systems (no decode change)

4.5.3 The Solution (MANDATORY in All Generators)

For Crossbar AR/AW Gating: - Replace `<master>_slave_select_ar/aw` gates with **inline address re-decode** on stable m_axi buses - Address on m_axi is held stable across the entire AXI handshake

For Master Adapter Response MUX: - Add **per-channel response-tracking FIFOs** that capture slave index at request time - Use FIFO head for response MUX instead of live combinational decode

Crossbar Code Pattern:

```
// WRONG (old implementation):
assign gate_ar_s0 = m0_slave_select_ar && m0_arvalid;
//                  ^^^^^^^^^^^^^^^^^^ Reverts after handshake!

// CORRECT (new implementation):
assign gate_ar_s0 = ((m0_m_axi_araddr >= S0_BASE) &&
                     (m0_m_axi_araddr < S0_END) &&
```

```
                        m0_arvalid);  // Re-decode on stable m_axi
address
```

Master Adapter Code Pattern:

```
// WRONG (old implementation):
assign r_slave_select = comb_slave_select_ar;  // Stale after pop!

// CORRECT (new implementation):
// Capture at request time
always_ff @(posedge aclk) begin
    if (fub_axi_arvalid && fub_axi_arready) begin
        ar_trk_fifo <= comb_slave_select_ar;
    end
end

// Use stable value for response MUX
assign r_slave_select = ar_trk_fifo_out;
```

4.5.4 When This Bug Appears

- Works: Single-master, single-slave systems (no address re-evaluation)
- Fails: Multi-master to same slave (multiple addresses decode differently)
- Fails: Multi-slave same master (address decode changes per transaction)
- Fails: Multi-width paths (W beats re-evaluate AW decode mid-burst)

4.6 MANDATORY: Project Organization Pattern

THIS SUBSYSTEM MUST FOLLOW THE RAPIDS/AMBA ORGANIZATIONAL PATTERN - NO EXCEPTIONS

4.6.1 Required Directory Structure

```
projects/components/bridge/
├── bin/                            # Bridge-specific tools
(generators, scripts)
│   ├── bridge_generator.py        # AXI4 crossbar generator (CSV/TOML
driven)
│   ├── bridge_pkg/                 # Generator implementation package
│   ├── test_configs/               # TOML port configs + CSV
connectivity
│   └── Makefile                    # make all / make single / make
clean
├── docs/                           # Design documentation
(bridge_has/, bridge_mas/, PDFs)
├── dv/                             # Design Verification (MANDATORY
structure)
│   ├── tbclasses/                  # Testbench classes (MANDATORY - TB
classes here!)
│   │   ├── __init__.py
│   │   └── bridge2x2_rw_tb.py      # Per-config generated TB classes
│   ├── components/                 # Bridge-specific BFMs (if needed)
│   ├── scoreboards/                # Bridge-specific scoreboards (if
needed)
│   ├── testplans/                  # Test plans
│   └── tests/                      # All test files (test runners
only, flat)
│       ├── conftest.py             # Pytest configuration (MANDATORY)
│       └── test_bridge_*.py        # Per-config tests
(test_bridge_2x2_rw.py, ...)
├── rtl/                            # RTL source files
│   ├── bridge_cam.sv               # Hand-written CAM (unused by every
generated bridge)
│   ├── filelists/                  # Generated filelists
│   └── generated/                  # Generated bridges (one subdir per
config)
├── CLAUDE.md                       # This file
├── PRD.md                          # Product requirements
└── TASKS.md                        # Task history and status
```

4.6.2 Testbench Class Location (MANDATORY)

WRONG: Testbench class in test file

```
# projects/components/bridge/dv/tests/test_bridge_2x2_rw.py
class Bridge2x2RwTB:  # WRONG LOCATION!
    """Embedded TB - NOT REUSABLE"""
```

CORRECT: Testbench class in PROJECT AREA dv/tbclasses/

```
# projects/components/bridge/dv/tbclasses/bridge2x2_rw_tb.py
class Bridge2x2RwTB(TBBase):  # CORRECT LOCATION!
    """Reusable TB class - imported by the matching test runner"""
```

CRITICAL: TB classes are PROJECT-SPECIFIC and MUST be in the project area (`projects/components/{name}/dv/tbclasses/`), NOT in the framework (`bin/TBClasses/`).

4.6.3 Test File Pattern (MANDATORY)

Test files MUST follow this structure:

```
# projects/components/bridge/dv/tests/test_bridge_2x2_rw.py

import os
import pytest
import cocotb
from cocotb_test.simulator import run

# IMPORT testbench class from PROJECT AREA
import sys
from TBClasses.shared.utilities import get_repo_root
sys.path.insert(0, get_repo_root())
from projects.components.bridge.dv.tbclasses.bridge2x2_rw_tb import
Bridge2x2RwTB
from TBClasses.shared.utilities import get_paths, get_wave_config
from TBClasses.shared.filelist_utils import get_sources_from_filelist

#
==========================================================================
=====
# COCOTB TEST FUNCTIONS - Prefix with "cocotb_" to prevent pytest
collection
#
==========================================================================
=====

@cocotb.test(timeout_time=100, timeout_unit='us')
async def cocotb_test_basic_routing(dut):
    """Test basic address routing"""
    tb = Bridge2x2RwTB(dut)  # Imported TB
    await tb.setup_clocks_and_reset()
    result = await tb.test_basic_routing()
```

```
    assert result, "Basic routing test failed"

# More cocotb test functions...

#
================================================================================
=====
# PYTEST WRAPPER FUNCTIONS - At bottom of file
#
================================================================================
=====

@pytest.mark.bridge
def test_bridge_2x2_rw(request):
    """Pytest wrapper for the generated 2x2 rw bridge"""
    module, repo_root, tests_dir, log_dir, rtl_dict = get_paths({
        'rtl_bridge': '../../rtl'
    })

    # ... verilog_sources from the bridge's filelist, parameters,
etc ...

    run(
        verilog_sources=verilog_sources,
        toplevel="bridge_2x2_rw",
        module=module,
        testcase="cocotb_test_basic_routing",  # ← cocotb function
name
        parameters=rtl_parameters,
        sim_build=sim_build,
        # ...
    )
```

See `dv/tests/test_bridge_2x2_rw.py` for the real, complete pattern (tests and TB classes are generated per bridge configuration alongside the RTL).

4.7 Critical Rules for This Subsystem

4.7.1 Rule #0: Attribution Format for Git Commits

IMPORTANT: When creating git commit messages for Bridge documentation or code:

Use:

```
Documentation and implementation support by Claude.
```

Do NOT use:

```
Co-Authored-By: Claude <noreply@anthropic.com>
```

Rationale: Bridge receives AI assistance for structure and clarity, while design concepts and architectural decisions remain human-authored.

4.7.2 Rule #0.1: Testbench Architecture - MANDATORY SEPARATION

THIS IS A HARD REQUIREMENT - NO EXCEPTIONS

NEVER embed testbench classes inside test runner files!

The same testbench logic will be reused across multiple test scenarios. Having testbench code only in test files makes it COMPLETELY WORTHLESS for reuse.

MANDATORY Structure:

```
projects/components/bridge/
├── dv/
│   ├── tbclasses/                  # TB classes HERE (not in
framework!)
│   │   ├── __init__.py
│   │   ├── bridge2x2_rw_tb.py      ← REUSABLE TB CLASS (per config)
│   │   └── bridge5x3_channels_tb.py
│   ├── components/                 # Bridge-specific BFMs (if
needed)
│   │   └── __init__.py
│   ├── scoreboards/                # Bridge-specific scoreboards
│   │   └── __init__.py
│   └── tests/                      # Test runners (flat, one per
config)
│       ├── conftest.py             ← MANDATORY pytest config
│       ├── test_bridge_2x2_rw.py   ← TEST RUNNER ONLY (imports TB)
│       └── test_bridge_5x3_channels.py
```

Why This Matters:

1. **Reusability**: Same TB class used in:

- Per-config functional tests
- Monitor stress tests (`test_bridge_*_mon_monitor.py`)
- User projects (external imports)

2. **Maintainability**: Fix bug once in TB class, all tests benefit
3. **Composition**: TB classes can inherit/compose for complex scenarios

---

4.7.3 Rule #1: All Testbenches Inherit from TBBase

Every testbench class MUST inherit from TBBase:

```python
from TBClasses.shared.tbbase import TBBase

class BridgeAXI4FlatTB(TBBase):
    """Testbench for Bridge crossbar - inherits base functionality"""

    def __init__(self, dut, num_masters=2, num_slaves=2, **kwargs):
        super().__init__(dut)
        # Bridge-specific initialization
```

TBBase Provides: - Clock management (`start_clock`, `wait_clocks`) - Reset utilities (`assert_reset`, `deassert_reset`) - Logging (`self.log`) - Progress tracking (`mark_progress`) - Safety monitoring (timeouts, memory limits)

4.7.4 Rule #2: Mandatory Testbench Methods

Every testbench class MUST implement these three methods:

```python
async def setup_clocks_and_reset(self):
    """Complete initialization - starts clocks and performs reset"""
    await self.start_clock('aclk', freq=10, units='ns')

    # Set config signals before reset (if needed)
    # self.dut.cfg_param.value = initial_value

    # Reset sequence
    await self.assert_reset()
    await self.wait_clocks('aclk', 10)
    await self.deassert_reset()
    await self.wait_clocks('aclk', 5)

async def assert_reset(self):
```

```
    """Assert reset signal (active-low for AXI4)"""
    self.dut.aresetn.value = 0

async def deassert_reset(self):
    """Deassert reset signal"""
    self.dut.aresetn.value = 1
```

Why Required: - Consistency across all testbenches - Reusability for mid-test resets - Clear test structure and intent

---

4.7.5 Rule #3: Use GAXI Components for Protocol Handling

For Bridge testing, use GAXI Master/Slave components for AXI4 channel handling:

```
from CocoTBFramework.components.gaxi.gaxi_master import GAXIMaster
from CocoTBFramework.components.gaxi.gaxi_slave import GAXISlave
from CocoTBFramework.components.axi4.axi4_field_configs import
AXI4FieldConfigHelper

# CORRECT: Use GAXI for AXI4 channels
self.aw_master = GAXIMaster(
    dut=dut,
    title="AW_M0",
    prefix="s0_axi4_",
    clock=clock,

field_config=AXI4FieldConfigHelper.create_aw_field_config(id_width,
addr_width, 1),
    pkt_prefix="aw",
    multi_sig=True,
    log=log
)
```

Never manually drive AXI4 valid/ready handshakes - Use GAXI components.

4.7.6 Rule #4: Queue-Based Verification

For simple in-order verification, use direct monitor queue access:

```
# CORRECT: Direct queue access
aw_pkt = self.aw_slave._recvQ.popleft()
w_pkt = self.w_slave._recvQ.popleft()

# Verify
assert aw_pkt.addr == expected_addr
```

```
assert w_pkt.data == expected_data

# WRONG: Memory model for simple test
memory_model = MemoryModel()  # Unnecessary complexity
```

When to Use Memory Models: - Simple in-order tests → Use queue access - Single-master systems → Use queue access - Complex out-of-order scenarios → not supported; slaves must respond in request order - Multi-master with address overlap → Memory model tracks state

---

4.8 TOML/CSV-Based Bridge Generator (Phase 2 Complete)

4.8.1 Overview

The Bridge generator creates parameterized SystemVerilog crossbars from TOML configuration files with CSV connectivity matrices, eliminating manual RTL editing for complex interconnects.

Key Benefits: - **Human-readable configuration** - TOML for ports, CSV for connectivity - **Custom signal prefixes** - Each port has unique prefix (rapids_m_axi_, apb0_, etc.) - **Channel-specific masters** - Write-only (wr), read-only (rd), or full (rw) - **Interface modules** - Timing isolation via axi4_master/slave wrappers with configurable skid depths - **Automatic converters** - Width and protocol conversion inserted automatically - **Resource efficient** - Only generates needed channels and converters

4.8.2 Quick Start

1. Create bridge_mybridge.toml:

```
[bridge]
  name = "bridge_mybridge"
  description = "Custom bridge example"

  # Default skid buffer depths (can be overridden per port)
  defaults.skid_depths = {ar = 2, r = 4, aw = 2, w = 4, b = 2}

  masters = [
    {name = "cpu", prefix = "cpu_m_axi", id_width = 4, addr_width =
32, data_width = 64, user_width = 1,
     interface = {type = "axi4_master"}},
    {name = "dma", prefix = "dma_m_axi", id_width = 4, addr_width =
32, data_width = 512, user_width = 1,
     interface = {type = "axi4_master", skid_depths = {ar = 4, r = 8,
aw = 4, w = 8, b = 4}}}
  ]

  slaves = [
    {name = "ddr", prefix = "ddr_s_axi", id_width = 4, addr_width =
32, data_width = 512, user_width = 1,
     base_addr = 0x00000000, addr_range = 0x80000000, interface =
{type = "axi4_slave"}},
    {name = "sram", prefix = "sram_s_axi", id_width = 4, addr_width =
32, data_width = 256, user_width = 1,
     base_addr = 0x80000000, addr_range = 0x80000000, interface =
{type = "axi4_slave"}}
  ]
```

2. Create bridge_mybridge_connectivity.csv:

```
master\slave,ddr,sram
cpu,1,1
dma,1,1
```

3. Generate bridge:

```
cd projects/components/bridge/bin
python3 bridge_generator.py --ports test_configs/bridge_mybridge.toml
# Auto-finds bridge_mybridge_connectivity.csv

# Or use bulk generation
python3 bridge_generator.py --bulk bridge_batch.csv
```

Result: Complete SystemVerilog module with: - Custom port prefixes per port - Timing isolation via interface wrappers - Only needed AXI4 channels (wr/rd/rw optimized) - Width converters for data mismatches - Internal crossbar instantiation - APB/AXI4-Lite converter integration points

4.8.3 Configuration Format Details

TOML Port Configuration:

The primary format is now **TOML** (preferred over CSV for better structure and interface configuration):

Port Specifications: - `name` - Unique identifier (cpu, dma, ddr, etc.) - `prefix` - Signal prefix (cpu_m_axi_, ddr_s_axi_, etc.) - `id_width` - AXI4 ID width in bits - `addr_width` - Address width in bits - `data_width` - Data width in bits (32, 64, 128, 256, 512) - `user_width` - AXI4 user signal width - `base_addr` - Slave base address (slaves only) - `addr_range` - Slave address range (slaves only) - `interface` - Interface wrapper configuration (optional) - `type` - “axi4_master”, “axi4_slave”, “axi4_master_mon”, “axi4_slave_mon”, or omit for direct connection - `skid_depths` - Per-channel buffer depths: {ar, r, aw, w, b}. Valid range is **2..8 inclusive, any integer** – odd depths are legal. `gaxi_skid_buffer` says so at its parameter: “Must be 2..8 inclusive”. An earlier `{2, 4, 6, 8}` here was an inference, not the contract.

CSV Connectivity Matrix:

```
master\slave,slave0,slave1,slave2
master0,1,0,1
master1,0,1,1
```

- `1` = connected, `0` = not connected
- Partial connectivity supported (not all masters to all slaves)
- Auto-detected based on TOML filename: `bridge_name.toml` → `bridge_name_connectivity.csv`

Legacy CSV Format:

For backwards compatibility, the generator still supports CSV port files: - See `bin/test_configs/README.md` for migration guide from CSV to TOML - TOML is now preferred for new bridges (better structure, interface config support)

4.8.4 Channel-Specific Masters (Phase 2 Feature)

Why Channel-Specific? Real hardware often has dedicated read or write masters. Generating all 5 AXI4 channels wastes resources:

Traditional (wasteful):

```
// Write-only master gets unused read channels
input  logic [63:0]  rapids_descr_m_axi_awaddr,  // USED
input  logic [511:0] rapids_descr_m_axi_wdata,   // USED
output logic [7:0]   rapids_descr_m_axi_bid,     // USED
input  logic [63:0]  rapids_descr_m_axi_araddr,  // UNUSED (50%
waste!)
output logic [511:0] rapids_descr_m_axi_rdata,   // UNUSED
```

Channel-Specific (optimized):

```
rapids_descr_wr,master,axi4,wr,rapids_descr_m_axi_,512,64,8,N/A,N/A
```

Generated:

```
// Write-only master - only AW, W, B channels
input  logic [63:0]  rapids_descr_m_axi_awaddr,
input  logic [511:0] rapids_descr_m_axi_wdata,
output logic [7:0]   rapids_descr_m_axi_bid,
// NO READ CHANNELS (araddr, rdata, etc.)
```

Resource Savings: - 40-60% fewer ports for dedicated masters - Only necessary width converters instantiated - Channel-aware direct connection wiring - Faster synthesis, smaller netlists

4.8.5 Example: RAPIDS-Style Configuration

RAPIDS Architecture: - Descriptor write master (wr) - Writes descriptors to memory - Sink write master (wr) - Writes incoming packets to memory - Source read master (rd) - Reads outgoing packets from memory - CPU master (rw) - Full access for configuration

TOML Configuration (bridge_rapids.toml):

```
[bridge]
  name = "bridge_rapids"
  description = "RAPIDS accelerator bridge"
  defaults.skid_depths = {ar = 2, r = 4, aw = 2, w = 4, b = 2}

  masters = [
    {name = "rapids_descr_wr", prefix = "rapids_descr_m_axi", channels
= "wr",
```

```
     id_width = 8, addr_width = 64, data_width = 512, user_width = 1,
     interface = {type = "axi4_master"}},
    {name = "rapids_sink_wr", prefix = "rapids_sink_m_axi", channels =
"wr",
     id_width = 8, addr_width = 64, data_width = 512, user_width = 1,
     interface = {type = "axi4_master"}},
    {name = "rapids_src_rd", prefix = "rapids_src_m_axi", channels =
"rd",
     id_width = 8, addr_width = 64, data_width = 512, user_width = 1,
     interface = {type = "axi4_master"}},
    {name = "cpu", prefix = "cpu_m_axi", channels = "rw",
     id_width = 4, addr_width = 32, data_width = 64, user_width = 1,
     interface = {type = "axi4_master"}}
  ]

  slaves = [
    {name = "ddr", prefix = "ddr_s_axi", protocol = "axi4",
     id_width = 8, addr_width = 64, data_width = 512, user_width = 1,
     base_addr = 0x80000000, addr_range = 0x80000000,
     interface = {type = "axi4_slave"}},
    {name = "apb_periph", prefix = "apb0_", protocol = "apb",
     addr_width = 32, data_width = 32,
     base_addr = 0x00000000, addr_range = 0x00010000}
  ]
```

Connectivity CSV (bridge_rapids_connectivity.csv):

```
master\slave,ddr,apb_periph
rapids_descr_wr,1,0
rapids_sink_wr,1,0
rapids_src_rd,1,0
cpu,1,1
```

Generated RTL Features: - rapids_descr_wr: write channels only - rapids_sink_wr: write channels only - rapids_src_rd: read channels only

The specific counts this list used to give (37 wr / 24 rd against 61 full, for 39% and 61% reductions) reproduce from no port structure in the tree and are not recoverable – the RAPIDS bridges they describe are not in the generated set here. For scale, counted on the shipped variants: an AXI4 `rw` master port is 44 signals, a write-only master 24, a read-only master 20. Channel trimming is real and worth doing; the old percentages were not measurements. - cpu_master: Width converters for 64b→512b upsize (both wr and rd converters) - ddr_controller: Direct 512b connection (no conversion) - apb_periph0: APB slave – `axi4_to_apb4_shim`, a real conversion

4.8.6 Common User Questions

Q: “How do I generate a bridge?”

A: Three steps:

1. Create TOML port configuration file
2. Create CSV connectivity matrix
3. Run bridge_generator.py

```
# Create bridge_mybridge.toml (port configuration)
# Create bridge_mybridge_connectivity.csv (connectivity matrix)

cd projects/components/bridge/bin
python3 bridge_generator.py --ports test_configs/bridge_mybridge.toml
# Auto-finds bridge_mybridge_connectivity.csv

# Or use bulk generation for multiple bridges
python3 bridge_generator.py --bulk bridge_batch.csv
```

Q: "What's the difference between wr/rd/rw channels?"

A: Number of AXI4 channels generated:

- **rw** (read/write) - All 5 channels: AW, W, B, AR, R
- **wr** (write-only) - 3 channels: AW, W, B (no read channels)
- **rd** (read-only) - 2 channels: AR, R (no write channels)

Benefits: 40-60% fewer ports for dedicated masters, less logic, faster synthesis

Q: "Can I add timing isolation to ports?"

A: Yes, use interface configuration:

```
masters = [
  {name = "cpu", prefix = "cpu_m_axi", ...,
   interface = {type = "axi4_master", skid_depths = {ar = 2, r = 4, aw
= 2, w = 4, b = 2}}}
]
```

Available interface types: - `"axi4_master"` - Timing isolation on master port - `"axi4_slave"` - Timing isolation on slave port - `"axi4_master_mon"` - Timing + monitoring - `"axi4_slave_mon"` - Timing + monitoring - Omit `interface` field for direct connection

Q: "Can I mix AXI4 and APB slaves?"

A: Yes, use protocol field in TOML:

```
slaves = [
  {name = "ddr", prefix = "ddr_s_axi", protocol = "axi4", ...},
  {name = "apb0", prefix = "apb0_", protocol = "apb", ...}
]
```

Generator inserts `axi4_to_apb4_shim` automatically. This is a real, complete conversion – burst decomposition, width slicing and response mapping – not a Phase-3 placeholder.

Q: "Can ports be AMBA5 (AXI5 / APB5)?"

A: Yes — per port, while the fabric stays AXI4 (BRIDGE-002):

```
[[bridge.masters]]
protocol = "axi5"
axi5_features = ["trace", "atomic"]  # nsaid/trace/mpam/mecid/unique
are
                                     # droppable sideband;
poison/atomic are
                                     # connectivity-gated (every
connected path
                                     # must be AXI5 + feature + width-
matched);
                                     # mte/chunking rejected

[[bridge.slaves]]
protocol = "apb5"   # APB4 rules apply (rw-only, 32-bit); adds the
APB5 pins
```

Sideband passes natively on width-matched AXI5<->AXI5 paths and terminates with a generation-time warning elsewhere. Atomics are store-class only: read-return classes DECERR at the boundary (`axi5_atomic_filter`). Details: `docs/bridge_has/ch04_interfaces/04_axi5_apb5_interfaces.md` and `docs/bridge_mas/ch02_blocks/10_amba5_boundary.md`.

Q: "What if data widths don't match?"

A: Generator inserts width converters automatically:

```
masters = [
  {name = "cpu", data_width = 64, ...},  # 64b master
]
slaves = [
  {name = "ddr", data_width = 512, ...}  # 512b slave
]
# Generator creates width converter: 64b → 512b
```

Q: "Do all masters need to connect to all slaves?"

A: No, use partial connectivity in CSV:

```
master\slave,ddr,sram,apb
rapids_descr,1,1,0     # Connects to ddr and sram only
cpu,1,1,1              # Connects to all three
```

4.8.7 Generator Output Structure

Generated File Contains:

1. **Module Header** - NOT parameterized. The generator resolves every width and count at emission time, so the top module has no NUM_MASTERS / NUM_SLAVES parameters to override.
2. **Port Declarations** - custom prefix per port, channel-specific signals
3. **Internal Signals** - crossbar interface nets
4. **Width Converters** - master-side upsize instances (channel-aware)
5. **Crossbar Instance** - internal AXI4 crossbar
6. **Slave Adapters** - one per slave, carrying the in-order `bridge_id` FIFO
7. **Subtractive slave** - `axi4_subtractive_slave`, ALWAYS emitted, wired to the decode `else`; answers unmapped addresses with DECERR (HAS 4.5)
8. **Protocol shims** - `axi4_to_apb4_shim` / `axi4_to_axil4_*` for apb/axil slaves. These are real conversions, not the Phase-3 TODO placeholders this list used to claim.
9. **Monitor subsystem** (monitored variants) - per-port wrappers, a `monbus_arbiter` tree and one `monbus_axil4_axil4_group`

Example Output Size: - Simple 2x2 bridge: 1,237 lines in the top file; 4,651 across all generated files for that variant (measured, not estimated). Formerly "~400" (plus per-master and per-slave adapters, a crossbar and a package). The "~400" this line used to claim predates monitors, the cfg subsystem and sideband routing. - Complex 5x3 with mixed protocols: 1,776 in the top file; 7,556 across all files. The old "~900" was smaller than the 2x2 figure beside it, which alone showed both were guesses. - Includes comprehensive comments and structure

4.8.8 Testing Generated Bridges

For Pure AXI4 Bridges (Working Now):

```
# Generate bridge (outputs to rtl/generated/<bridge_name>/)
python3 bridge_generator.py --ports test_configs/bridge_2x2_rw.toml

# Run its test (per-config TB classes live in dv/tbclasses/, e.g.
bridge2x2_rw_tb.py)
cd ../dv/tests
make clean-all && make run-bridge_2x2_rw-gate-parallel   # or -func /
-full, [-waves]
```

Tests are GENERATED too. `make regen` in `bin/` regenerates RTL only; the tests and TB classes come from `bridge_generator.py --bulk bridge_batch.csv --generate-tests`, which writes into `dv/tests` and `dv/tbclasses` per the manifest's own output columns (the `--output-*` flags do not override a bulk run). Regenerate them the same way you regenerate RTL: delete every file marked "Generated by bridge test generator", regenerate all, never hand-edit one. Hand-written tests for a generated DUT live in their own file beside it

(`test_bridge_2x2_rw_tracking.py`, `test_bridge_*_sideband*.py`, `*_atomics.py`, `*_bfm5.py`).

For Mixed AXI4/APB: the APB path is complete: `axi4_to_apb4_shim` does real burst decomposition, width slicing and response mapping, and six configurations ship APB slaves with tests in the FULL regression. This line claimed placeholders long after the converter shipped.

4.9 Bridge Architecture Quick Reference

4.9.1 Generated Bridge Crossbar Structure

Illustrative shape (see `rtl/generated/bridge_2x2_rw/bridge_2x2_rw.sv` for a real example):

The generated top takes **no parameters at all** – every width is fixed at generation time and reaches the module through its package:

```
module bridge_2x2_rw
    import bridge_2x2_rw_pkg::*;
(
    input  logic aclk,
    input  logic aresetn,

    // Master-side interfaces (slave ports on bridge)
    // AW, W, B, AR, R channels for each master

    // Slave-side interfaces (master ports on bridge)
    // AW, W, B, AR, R channels for each slave
);
```

4.9.2 Key Features

1. **5-Channel Implementation:** Complete AW, W, B, AR, R routing
2. **Round-Robin Arbitration:** Per-slave arbitration with grant locking
3. **Response routing:** a small in-order FIFO per direction in each slave adapter, holding the `bridge_id` of the master whose request was accepted. Not a RAM, and not indexed by AXI ID.
4. **`bridge_id` sideband routing:** responses return IN ORDER. The slave must complete in the order its requests were accepted; a reordering slave misroutes silently (see FR-2 in the PRD).
5. **Configurable Parameters:** NxM topology, data/addr/ID widths

4.9.3 Address Map

There is no default address map and no 0x10000000 stride. Windows come from each slave's `base_addr` / `addr_range` in the TOML and are baked into the decode. `bridge_2x2_rw` splits the space in half:

- Slave 0 (ddr): 0x00000000 - 0x7FFFFFFF
- Slave 1 (sram): 0x80000000 - 0xFFFFFFFF

Probing 0x1000_0000 expecting "slave 1" hits slave 0.

4.10 Test Organization

4.10.1 Test Hierarchy

```
projects/components/bridge/dv/tests/
├── conftest.py                          # Pytest configuration with
fixtures
├── test_bridge_1x2_rd.py                # Per-config functional tests
├── test_bridge_2x2_rw.py
├── test_bridge_4x4_rw.py
├── test_bridge_5x3_channels.py
├── test_bridge_mix_a_mon_monitor.py     # Monitor stress tests (mon
configs)
└── monitor_stress_common.py             # Shared monitor-stress helpers
```

4.10.2 Test Levels

Three real levels, both mechanisms, every test (45 of 45 under `bin/review/check_test_levels.py`): each wrapper's `generate_bridge_levels()` branches on `REG_LEVEL` (GATE 72 / FUNC 144 / FULL 216 cells) and exports `TEST_LEVEL` + `SEED` per cell; the TB reads the depth profile from `dv/tbclasses/bridge_levels.py`. Do NOT stamp `TEST_LEVEL` into `os.environ` from a conftest – cocotb_test lets the environment override `extra_env`, and that ran all 216 cells at one depth once (TOOL-016).

Per-Config Functional Tests (generated): basic connectivity, boundary probe, arbitration (multi-master). Every AXI5 master port carries an `AXI5ComplianceChecker`; every test asserts zero violations.

Monitor Stress Tests (`*_mon_monitor.py`) (generated): bridges built with monitor interfaces, monbus traffic / WRITE_BP / ERR_BP phases scaled by level.

Hand-written: BRIDGE-011 tracking + latency on 2x2; the AMBA5 sign-off tests (sideband values, sideband through arbitration, sideband across a width converter, atomics incl. Compare, AXI5 read BFM sign-off).

4.11 Common User Questions and Responses

4.11.1 Q: "How does the Bridge work?"

A: Direct answer:

The Bridge AXI4 crossbar connects multiple AXI4 masters to multiple slaves:

1. **Address Decode (AW/AR):** Routes master requests to appropriate slave based on address
2. **Write Path:** AW → arbitration → slave, W follows locked grant
3. **Read Path:** AR → arbitration → slave, R returns via the slave adapter's in-order `bridge_id` FIFO
4. **Arbitration:** Per-slave round-robin, AW and AR only, with the grant locked until the ADDRESS handshake – not until burst complete. W is owned via a FIFO; B and R have no arbiter.
5. **Response Routing:** each slave adapter keeps an IN-ORDER FIFO of the originating master's `bridge_id`, pushed on the address handshake and popped on the response. Routing is keyed on FIFO POSITION, not on the returned BID/RID – IDs are pass-through and never widened. There are no ID lookup tables; `bridge_cam.sv` is instantiated in zero generated bridges.

Key Features: - Configurable NxM topology - Single-clock AXI fabric (an `apb`/`apb5` slave crosses domains inside its own shim, which contains two async FIFOs) - Subtractive catch-all: an unmapped address gets DECERR + 0xDEADBEEF and a sticky status/IRQ instead of hanging the master (HAS 4.5)

Deliberately NOT supported – these were claimed here for months and none of them is built: - Out-of-order responses. Ordering is structural: one target slave per master at a time (`aw_gate_ok`), and each slave port must return B/R in request order across all IDs (BRIDGE-010). - Burst-locked arbitration. The grant releases at the address handshake, as item 4 above already said – this bullet contradicted it directly. - Configurable arbitration policy. Round-robin is hard-coded.

See: - `projects/components/bridge/PRD.md` - Complete specification - `projects/components/bridge/bin/bridge_generator.py` - Generator implementation

4.11.2 Q: "How do I generate a Bridge?"

A: Use the bridge_generator.py tool:

```
cd projects/components/bridge/bin
python3 bridge_generator.py --ports test_configs/bridge_2x2_rw.toml
# Auto-finds test_configs/bridge_2x2_rw_connectivity.csv
# Or regenerate everything: make all  (bulk from bridge_batch.csv)
```

Generated files: - RTL: `projects/components/bridge/rtl/generated/<bridge_name>/` (top module, crossbar, adapters, package) - Contains complete 5-channel crossbar with ID tracking

4.11.3 Q: "How do I test a Bridge?"

A: Use the Bridge testbench class:

```
# Import from project area
import sys
from TBClasses.shared.utilities import get_repo_root
sys.path.insert(0, get_repo_root())
from projects.components.bridge.dv.tbclasses.bridge2x2_rw_tb import
Bridge2x2RwTB

@cocotb.test()
async def cocotb_test_basic(dut):
    tb = Bridge2x2RwTB(dut)
    await tb.setup_clocks_and_reset()

    # Send write to slave 0
    await tb.write_transaction(master_idx=0, address=0x00001000,
data=0xDEADBEEF)

    # Verify routing
    assert len(tb.aw_slaves[0]._recvQ) > 0, "Slave 0 should receive
AW"
```

Run tests:

```
cd projects/components/bridge/dv/tests
pytest test_bridge_2x2_rw.py -v
```

4.12 Anti-Patterns to Avoid

4.12.1 Anti-Pattern 1: Embedded Testbench Classes

```
WRONG: TB class in test file
class BridgeTB:
    """NOT REUSABLE - WRONG LOCATION"""

CORRECT: Import from project area
import sys
from TBClasses.shared.utilities import get_repo_root
sys.path.insert(0, get_repo_root())
from projects.components.bridge.dv.tbclasses.bridge2x2_rw_tb import
Bridge2x2RwTB
```

4.12.2 Anti-Pattern 2: Manual AXI4 Handshaking

```
WRONG: Manual signal driving
self.dut.s0_axi4_awvalid.value = 1
while self.dut.s0_axi4_awready.value == 0:
    await RisingEdge(self.clock)

CORRECT: Use GAXI components
await self.aw_master.send(aw_pkt)
```

4.12.3 Anti-Pattern 3: Memory Models for Simple Tests

```
WRONG: Unnecessary complexity
memory = MemoryModel()
memory.write(addr, data)
result = memory.read(addr)

CORRECT: Direct queue verification
aw_pkt = self.aw_slave._recvQ.popleft()
assert aw_pkt.addr == expected_addr
```

4.13 Quick Reference

4.13.1 Finding Existing Components

```
# Bridge TB classes (in PROJECT AREA, not framework!)
ls projects/components/bridge/dv/tbclasses/

# Bridge generator
cat projects/components/bridge/bin/bridge_generator.py

# Test examples
ls projects/components/bridge/dv/tests/
```

4.13.2 Common Commands

```
# Generate 2x2 bridge (from bin/, outputs to
rtl/generated/bridge_2x2_rw/)
cd projects/components/bridge/bin
python3 bridge_generator.py --ports test_configs/bridge_2x2_rw.toml

# Run tests
pytest projects/components/bridge/dv/tests/test_bridge_2x2_rw.py -v

# Lint generated RTL
verilator --lint-only
projects/components/bridge/rtl/generated/bridge_2x2_rw/bridge_2x2_rw.s
v
```

4.14 Remember

1. **MANDATORY: Testbench architecture** - TB classes in the PROJECT area (`projects/components/bridge/dv/tbclasses/`), NOT the framework; tests import them. (This line used to say “in framework”, contradicting the hard rule above.)
2. **MANDATORY: Directory structure** - Follow RAPIDS/AMBA pattern exactly
3. **MANDATORY: conftest.py** - Must exist in dv/tests/
4. **Use GAXI components** - Never manually drive AXI4 handshakes
5. **Queue-based verification** - Simple tests use direct queue access
6. **Three-layer architecture** - TB (PROJECT area, not the framework) + Test (runner) + Scoreboard (verification)
7. **Three mandatory methods** - setup_clocks_and_reset, assert_reset, deassert_reset
8. **Search first** - Use existing components before creating new ones
9. **Test scalability** - Support basic/medium/full test levels
10. **100% success** - All tests must achieve 100% success rate

4.15 PDF Generation Location

IMPORTANT: PDF files should be generated in the docs directory:

`projects/components/bridge/docs/`

Quick Commands: Use the provided shell scripts:

```
cd projects/components/bridge/docs
./generate_has_pdf.sh   # Builds Bridge_HAS_v<rev> from bridge_has/
./generate_mas_pdf.sh   # Builds Bridge_MAS_v<rev> from bridge_mas/
```

The shell scripts will automatically: 1. Use the md_to_docx.py tool from bin/ 2. Process the bridge_has/bridge_mas index files 3. Generate both DOCX and PDF files in the docs/ directory 4. Create table of contents and title page

See: `bin/md_to_docx.py` for complete implementation details

Maintained By: RTL Design Sherpa Project

(The version and date for this file are in the header at the top. A second stamp here said 1.0 / 2025-10-18 while the header said 2.1 / 2025-11-03 – two stamps in one file guarantee one of them is wrong.)

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 Document Information

5.1 Bridge Hardware Architecture Specification

Property	Value
Document Title	Bridge Hardware Architecture Specification
Version	1.2
Date	September 8, 2026
Status	Released
Classification	Open Source - MIT License

5.2 Revision History

Version	Date	Author	Description
1.0	2026-01-03	RTL Design Sherpa	Initial release - restructured from single spec
1.1	2026-06-04	RTL Design Sherpa	Content sync to RTL state at 2026-06-04. Documents (1) the 64→128-bit monbus packet migration with new 64-bit side-band timestamp wire — referenced via `docs/markdown/rtl-amba/includes/monitor_package_spec.md`; (2) per-port + global SV-parameter monitor methodology with mandatory `use_monitor` TOML field and `variants = ["no","mon"]` generator selector; (3) generator-emitted AXI4↔AXIL conversion at the slave boundary (was external); (4) new AXIL→wider-slave master-side alignment converters (`axil_to_axi4_wide_align_{rd,wr}`); (5) verification-strategy rewrite to protocol-BFM-only TBs with memory-backed slaves, serial-only execution, per-module cocotb function names, boundary_probe (renamed from address_decode) tests;

Version	Date	Author	Description
			(6) note of the independent W-tracker FIFO + split AW/W-ready MUX fix (commit `d24bd617`) that closed an interleaved-master deadlock.
1.2	2026-09-08	RTL Design Sherpa	Correctness and voice pass. Four external qc rounds (65/54/53/57 findings, all triaged against rtl/generated and the generator) removed the planned-but-never-built ID-table/CAM out-of-order scheme, the fixed 64-bit internal path, response arbitration/FIFOs and the AXI4-Lite-as-future-work status from the text; annotated the TOML keys the loader has never known and its three-tier unknown-key behaviour; replaced asserted latency figures with measured ones. Four RTL defects found by the rounds are fixed and mutation-checked (BRIDGE-011 response-FIFO overrun, axi4_subtractive_slave simultaneous AW+AR fault loss, axi5_atomic_filter duplicate B and B-before-WLAST). Prose then voice-passed with all correctness

Version	Date	Author	Description
			annotations preserved.

5.3 Document Purpose

This Hardware Architecture Specification (HAS) is the high-level view of the Bridge component — the map you read before you integrate, not the schematic. It covers:

- System-level architecture and data flow
- Protocol support (AXI4, AXI4-Lite, APB)
- Interface definitions and requirements
- Performance characteristics
- Integration guidelines

For micro-architecture, FSM designs, and signal-level detail, see the companion **Bridge Micro-Architecture Specification (MAS)**.

5.4 Intended Audience

- System architects defining interconnect topology
- Hardware engineers integrating Bridge into SoC designs
- Verification engineers planning test strategies
- Technical managers evaluating Bridge capabilities

5.5 Related Documents

- **Bridge MAS** - Micro-Architecture Specification (detailed block-level)
- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI development guide

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

6 Purpose and Scope

6.1 What is Bridge?

Bridge is a Python-based multi-protocol crossbar generator. You describe the interconnect in human-readable configuration files; it hands you back parameterized SystemVerilog RTL. The generated AXI4 crossbars come with automatic support for:

- **Protocol conversion** - AXI4, AXI4-Lite, and APB slave interfaces
- **Width conversion** - Automatic upsize/downsize for data width mismatches
- **Channel-specific masters** - Write-only, read-only, or full AXI4 masters

6.2 The Problem Bridge Solves

A hand-written AXI4 crossbar is error-prone. Building one means:

1. Writing 5 separate channel multiplexers (AW, W, B, AR, R)
2. Implementing per-slave arbitration for each channel
3. Routing responses by in-order bridge_id FIFO position (out-of-order is not supported)
4. Handling burst locking and interleaving constraints
5. Inserting width converters for data width mismatches
6. Inserting protocol converters for APB/AXI4-Lite mixed systems
7. Wiring hundreds of signals with consistent naming

Every item on that list is a place a bug can sit quietly until integration. I've chased most of them at least once.

Bridge automates all of it:

- Define ports and connectivity in configuration files
- Run the generator script
- Get production-ready SystemVerilog RTL

6.3 Scope

This specification covers:

- High-level architecture and block organization
- Supported protocols and conversion capabilities
- Interface requirements and signal conventions
- Performance characteristics and resource estimates
- Integration guidelines and verification strategy

6.4 Out of Scope

These live in the companion MAS:

- Detailed FSM state diagrams and transitions
- Signal-level timing diagrams
- Internal pipeline structures
- RTL implementation details

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

7 Document Conventions

7.1 Terminology

Table 1.1: Common Terminology

Term	Definition
Master	AXI4 initiator that issues transactions
Slave	AXI4 target that responds to transactions
Crossbar	NxM interconnect connecting masters to slaves
Channel	AXI4 communication path (AW, W, B, AR, R)
Converter	Logic that transforms width or protocol

7.2 Signal Naming

Bridge uses consistent signal prefixes — learn them once and every generated interface reads the same way:

Table 1.2: Signal Naming Prefixes

Prefix	Meaning
`s_*`	Slave-side interface (master port on bridge)
`m_*`	Master-side interface (slave port on bridge)
`axi4_*`	AXI4 protocol signals
`apb_*`	APB protocol signals
`cfg_*`	Configuration signals
`dbg_*`	Debug signals

7.3 Diagrams

7.3.1 Figure 1.1: Block Diagram Legend

Block diagrams in this specification use these symbols:

Table 1.3: Block Diagram Symbols

Symbol	Meaning
Rectangle	Functional block
Arrow	Data/signal flow
Dashed box	Optional/configurable block
Double line	Multi-signal bus

7.4 Code Examples

SystemVerilog examples are formatted like this:

```
// Module instantiation example
bridge_4x4 u_bridge (
    .aclk       (sys_clk),
    .aresetn    (sys_rst_n),
    // ... port connections
);
```

7.5 Parameter Notation

Parameter tables throughout the spec use this format:

Table 1.4: Parameter Format Example

Parameter	Type	Range	Default	Description
NUM_MASTERS	int	1-32	2	Number of master ports
NUM_SLAVES	int	1-256	2	Number of slave ports

8 Definitions and Acronyms

8.1 Acronyms

Table 1.5: Acronyms and Abbreviations

Acronym	Definition
AMBA	Advanced Microcontroller Bus Architecture
APB	Advanced Peripheral Bus
AR	AXI4 Read Address channel
AW	AXI4 Write Address channel
AXI	Advanced eXtensible Interface
AXI4	AXI Protocol Version 4
B	AXI4 Write Response channel
BID	Bridge ID (internal master identifier)
CAM	Content-Addressable Memory
CDC	Clock Domain Crossing
CSV	Comma-Separated Values
DECERR	Decode Error (AXI response)
DMA	Direct Memory Access
FIFO	First-In First-Out buffer
FSM	Finite State Machine
HAS	Hardware Architecture Specification
ID	Transaction Identifier
MAS	Micro-Architecture Specification
MUX	Multiplexer
NxM	N masters by M slaves (topology notation)
OOO	Out-of-Order
OOR	Out-of-Range
QoS	Quality of Service

Acronym	Definition
R	AXI4 Read Data channel
RTL	Register-Transfer Level
SLVERR	Slave Error (AXI response)
SoC	System-on-Chip
TOML	Tom’s Obvious Minimal Language
W	AXI4 Write Data channel

8.2 Definitions

Address Decode: Determining which slave receives a transaction, from its address.

Arbitration: Selecting which master gets a slave when several masters contend for it.

Bridge ID (BID): Internal routing tag carried ALONGSIDE a transaction inside the fabric, never prepended to the AXI ID. Width = clog2(NUM_MASTERS). Slave-side AXI IDs are the same width as master-side ones; the tag lives in the slave adapter's in-order FIFO, not on the bus.

Channel-Specific Master: A master that uses only a subset of AXI4 channels (write-only or read-only).

Crossbar: An NxM interconnect allowing any master to communicate with any connected slave.

Grant Locking: Mechanism that holds arbitration grant until transaction completes.

Out-of-Order Response: (defined for completeness; NOT supported by this bridge) Responses returned in different order than requests were issued.

Protocol Conversion: Transformation between different bus protocols (e.g., AXI4 to APB).

Response Routing: Directing B/R channel responses back to the originating master.

Round-Robin Arbitration: Fair arbitration where priority rotates among requesters.

Width Conversion: Transformation between different data bus widths (upsize or downsize).

9 Use Cases

9.1 Primary Applications

9.1.1 SoC Interconnects

In SoC designs, Bridge is the main interconnect fabric:

- **Multi-core processor memory subsystems** - Connect multiple CPU cores to shared memory
- **Accelerator integration** - Route GPU, DSP, or custom accelerator traffic
- **DMA engine routing** - Connect DMA controllers to memory and peripherals
- **Peripheral bus bridges** - Convert AXI4 to APB for low-speed peripherals

9.1.2 Memory Systems

Typical memory-system uses:

- **Multi-port DDR controllers** - Multiple masters accessing shared DDR
- **SRAM buffer sharing** - Shared packet buffers for networking
- **Cache coherency interconnects** - Connect cache controllers
- **Memory-mapped I/O** - Unified address space for peripherals

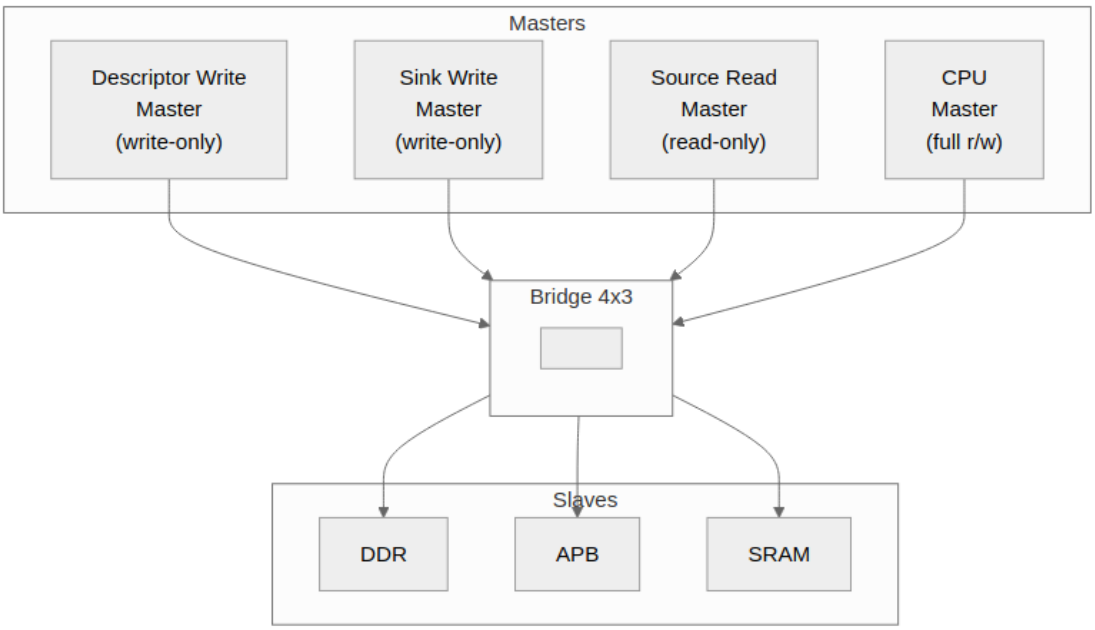
9.1.3 Accelerator Systems

Bridge supports custom accelerator architectures like RAPIDS:

- **Descriptor write masters** - Write control descriptors (write-only)
- **Sink write masters** - Write incoming data packets (write-only)
- **Source read masters** - Read outgoing data packets (read-only)
- **CPU configuration** - Full access for register configuration

9.2 Example: RAPIDS Accelerator

9.2.1 Figure 2.1: RAPIDS System Topology



RAPIDS System Topology

Bridge topology for RAPIDS accelerator with channel-specific masters.

9.2.2 Configuration

```
[bridge]
  name = "bridge_rapids"
  masters = [
    {name = "rapids_descr_wr", channels = "wr", data_width = 512},
    {name = "rapids_sink_wr", channels = "wr", data_width = 512},
    {name = "rapids_src_rd", channels = "rd", data_width = 512},
    {name = "cpu", channels = "rw", data_width = 64}
  ]
  slaves = [
    {name = "ddr", protocol = "axi4", data_width = 512},
    {name = "apb_periph", protocol = "apb", data_width = 32},
    {name = "sram", protocol = "axi4", data_width = 256}
  ]
```

9.2.3 Benefits

- **40% fewer channels** for write-only masters (3 of 5: AW, W, B)
- **60% fewer channels** for read-only masters (2 of 5: AR, R)

Those are CHANNEL counts, which is the honest way to state it – the saving in actual wires depends on the widths. At DW=64/AW=32/IW=4/UW=1 the five channels are roughly 295 bits, and dropping AR+R saves about 48% of them while dropping AW+W+B saves about 52%; at DW=512 it is nearer 47%/53%. Earlier revisions of this page quoted “39%” and “61%”, which match neither basis. - **Automatic width conversion** for 64b CPU to 512b DDR - **Protocol conversion** for APB peripheral access

9.3 Rapid Prototyping

Bridge accelerates design exploration:

- **Quick topology exploration** - Try different master/slave configurations
- **Performance analysis** - Evaluate arbitration and bandwidth
- **Design space exploration** - Compare resource usage
- **Educational projects** - Learn AXI4 protocol and interconnects

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

10 Key Features

10.1 Configuration-Driven Generation

10.1.1 CSV/TOML Configuration

Everything about a Bridge instance is set in human-readable text files:

- **ports configuration** - Define each master and slave port
- **connectivity matrix** - Specify which masters connect to which slaves
- **Version-control friendly** - Text-based, easy to diff and review

10.1.2 Example Configuration

Port Definition (TOML):

```
masters = [
  {name = "cpu", prefix = "cpu_m_axi", data_width = 64, channels =
"rw"},
  {name = "dma", prefix = "dma_m_axi", data_width = 256, channels =
"rw"}
]
slaves = [
  {name = "ddr", prefix = "ddr_s_axi", data_width = 512, protocol =
"axi4"},
  {name = "uart", prefix = "uart_apb", data_width = 32, protocol =
"apb"}
]
```

Connectivity Matrix (CSV):

```
master\slave,ddr,uart
cpu,1,1
dma,1,0
```

10.2 Multi-Protocol Support

Table 2.1: Supported Protocols

Master Protocol	Slave Protocol	Conversion Type
AXI4	AXI4	Direct or width convert
AXI4	AXI4-Lite	Protocol downgrade
AXI4	APB	Full protocol conversion
AXI5	AXI4 / AXI4-Lite / APB	Base-subset interop; sideband terminates at the boundary (warning)
AXI5	AXI5 (width-matched)	Native sideband pass-through; atomics (read-return classes need an rw port); poison
AXI4 / AXI5	APB5	APB4 conversion core + APB5 sideband surface
AXI4-Lite / AXI5-Lite	any	Lite requester: single-beat AXI4 inside; the AXI5-Lite `user` and `exclusive` groups ride the fabric, the rest terminate at the boundary
APB / APB5	any	APB requester: `apb4_to_axi4` / `apb5_to_axi4` front end, one AXI4 transaction per transfer, SLVERR and DECERR fold to PSLVERR
any	Wishbone B4	`axi4_to_wb4`: one Wishbone transfer per AXI4 beat, ERR/RTY fold to SLVERR
Wishbone B4	any	`wb4_to_axi4` front end: single-beat AXI4 inside, SLVERR/DECERR

Master Protocol	Slave Protocol	Conversion Type
		terminate ERR

AMBA5 ports are declared per port (`protocol = "axi5"` / `"axil5"` / `"apb5"` with an optional `axi5_features` list); the fabric stays AXI4 internally. Every protocol value is legal on a master port as well as a slave port (BRIDGE-014). See AXI5 and APB5 Interfaces.

10.2.1 Protocol Features

- **AXI4 Full** - Complete 5-channel implementation with bursts
- **AXI4-Lite** - Simplified single-beat transactions
- **APB** - Low-power peripheral access
- **Wishbone B4** - The open FPGA bus, pipelined mode, on either side (BRIDGE-019)

10.3 Channel-Specific Masters

10.3.1 Write-Only Masters (wr)

Generate only write channels: - AW (Write Address) - W (Write Data) - B (Write Response)

Use case: DMA write engines, descriptor writers

10.3.2 Read-Only Masters (rd)

Generate only read channels: - AR (Read Address) - R (Read Data)

Use case: DMA read engines, source data fetchers

10.3.3 Full Masters (rw)

Generate all five channels for complete read/write capability.

Use case: CPU interfaces, configuration masters

10.3.4 Resource Savings

Table 2.2: Channel-Specific Resource Savings

Master Type	Channels	Signal Reduction
Write-only (wr)	3 of 5	40% fewer channels (~48% fewer wires at DW=64)
Read-only (rd)	2 of 5	60% fewer channels (~52% fewer wires at DW=64)
Full (rw)	5 of 5	Baseline

10.4 Automatic Converters

10.4.1 Width Conversion

Bridge automatically inserts width converters:

- **Upsize** - Narrow master to wide slave (e.g., 64b to 512b)
- **Downsize** - Wide master to narrow slave (e.g., 512b to 32b)
- **Per-path** - Only where needed, not global conversion

10.4.2 Protocol Conversion

Bridge inserts protocol converters for mixed systems:

- **AXI4 to APB** - Full conversion with burst handling
- **AXI4 to AXI4-Lite** - Simplified conversion

10.5 Flexible Topology

10.5.1 Scalability

- **Masters:** 1-32 master ports
- **Slaves:** 1-256 slave ports
- **Partial connectivity** - Not all masters need access to all slaves

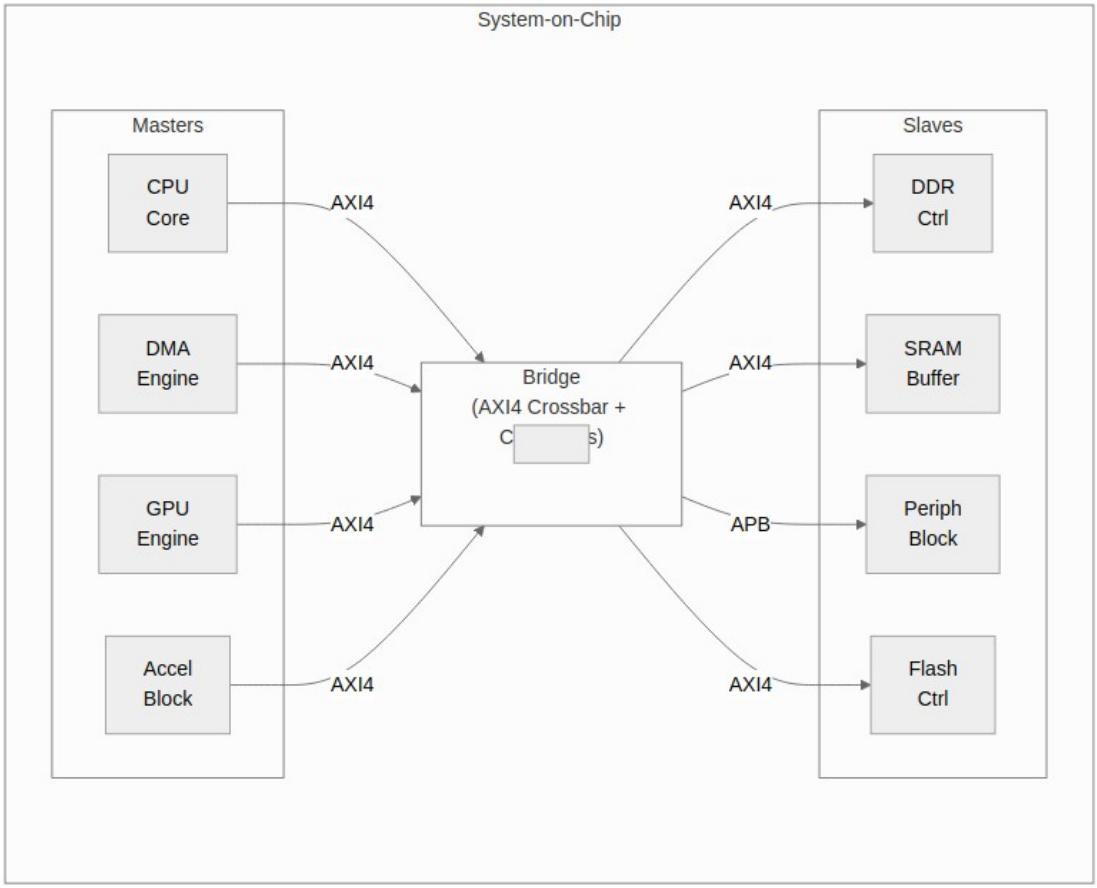
10.5.2 Mixed Configurations

- **Mixed protocols** - AXI4, AXI4-Lite, and APB slaves in same crossbar
- **Mixed data widths** - 32b, 64b, 128b, 256b, 512b
- **Mixed channels** - Write-only, read-only, and full masters

11 System Context

11.1 Bridge in SoC Architecture

11.1.1 Figure 2.2: Bridge System Context



Bridge System Context

Bridge as central interconnect connecting masters to slaves with protocol conversion.

11.2 Interface Boundaries

11.2.1 Master-Side (Upstream)

Bridge presents slave interfaces to upstream masters:

- Accepts AXI4 transactions from masters
- Provides flow control via ready signals
- Returns responses (B/R channels) to correct master

11.2.2 Slave-Side (Downstream)

Bridge presents master interfaces to downstream slaves:

- Issues AXI4/APB transactions to slaves
- Accepts responses from slaves
- Routes responses back through ID tracking

11.3 Address Space

11.3.1 Address Map Organization

Each slave occupies a configurable address region:

Table 2.3: Address Map Organization

Slave	Base Address	Address Range	Size
Slave 0	`base_addr` from its own TOML entry	`addr_range`	Variable
Slave 1	`base_addr` from its own TOML entry	`addr_range`	Variable
Slave N	`base_addr` from its own TOML entry	`addr_range`	Variable

Windows are not derived from a single BASE and are not contiguous. Each slave carries its own `base_addr`, subject only to 4 KB alignment and non-overlap (Parameters). An earlier revision of this table showed `BASE + addr_range_0` and "sum of previous ranges", which no shipped config follows – `bridge_mix_d` places a 1 GB `ddr` at `0x0000_0000`, a 4 KB `doorbell` at `0x4000_0000` and a 64 KB `apb_periph` at `0x4000_1000`, and gaps between windows are legal (they decode-miss).

11.3.2 Address Decode

- **Subtractive decode** - the ranges are tested in order and the chain ends in an `else`, so the one-hot result is *never* all-zero
- **One-hot result** - exactly one slave selected per transaction, always
- **Out-of-range** - an address matching no slave range selects the internal subtractive slave, which completes the transaction with **DECERR** and `0xDEADBEEF` read data rather than leaving it unanswered

Until 2026-09-07 the last bullet was aspirational: the decode chain had no `else`, so an unmapped address produced an all-zero select, no slave ever saw AWVALID/ARVALID, READY never rose, and **the master waited forever**. A hang is the worst available failure here because it destroys the evidence – there is no response to inspect, no error bit to read, and the offending address is whatever the master still has latched. See 4.5 for the status/interrupt path that now reports it.

11.4 Clock and Reset

11.4.1 Clock Domain

- **Single clock domain** - All Bridge logic synchronous to `aclk`
- **No CDC** - Masters and slaves must be in same clock domain

11.4.2 Reset

- **Active-low asynchronous** - Standard `aresetn` convention
- **Full reset** - All internal state cleared on reset
- **Transaction safety** - Outstanding transactions aborted on reset

11.5 Dependencies

11.5.1 Required Infrastructure

- AXI4-compliant masters and slaves
- Proper clock and reset distribution
- Address map configuration matching slave address ranges

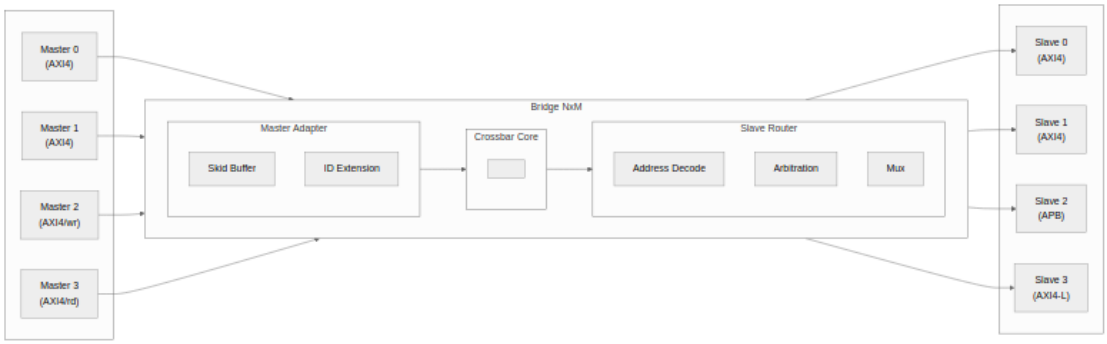
11.5.2 Optional Features

- Width converters (only if widths mismatch)
- Protocol converters (only if protocols differ)
- Per-slave in-order bridge_id FIFO (no CAM; OOO is not supported)

12 Block Diagram

12.1 Bridge Architecture Overview

12.1.1 Figure 3.1: Bridge Block Diagram



Bridge Block Diagram

Bridge high-level block diagram showing master adapters, crossbar core, and slave routers.

12.2 Functional Blocks

12.2.1 Master Adapter

Receives transactions from upstream masters:

- **Skid buffer** - Pipeline registration for timing
- **Response tracking** - the originating master is recorded in the slave adapter's in-order `bridge_id` FIFO; the AXI ID itself is passed through untouched
- **Channel separation** - Route AW/W/AR independently

12.2.2 Crossbar Core

Central switching fabric:

- **5-channel routing** - Independent AW, W, B, AR, R paths
- **Address decode** - Determine target slave
- **Arbitration** - Per-slave round-robin selection

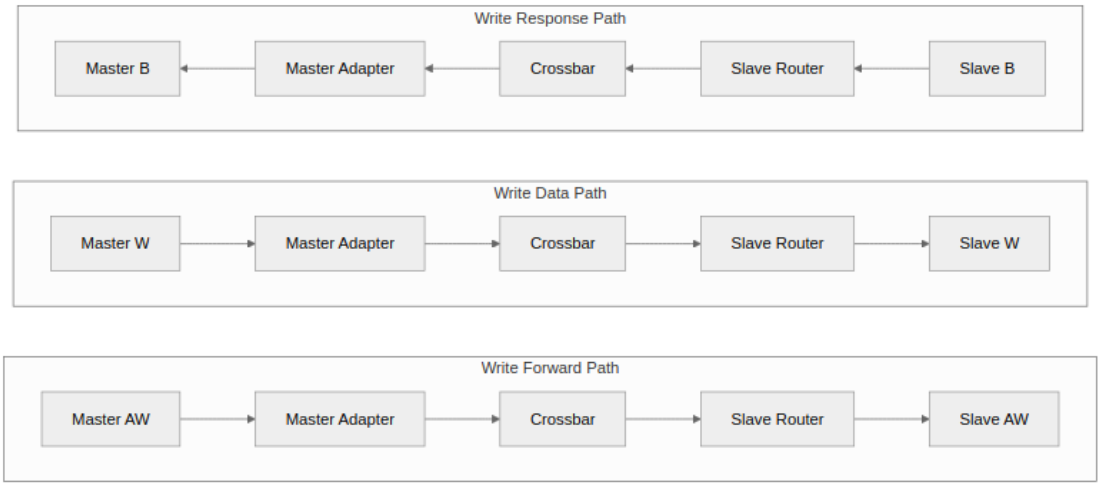
12.2.3 Slave Router

Delivers transactions to downstream slaves:

- **Protocol conversion** - AXI4 to APB/AXI4-Lite if needed
- **Width conversion** - Upsize/downsize data paths
- **Response collection** - Route B/R back to masters

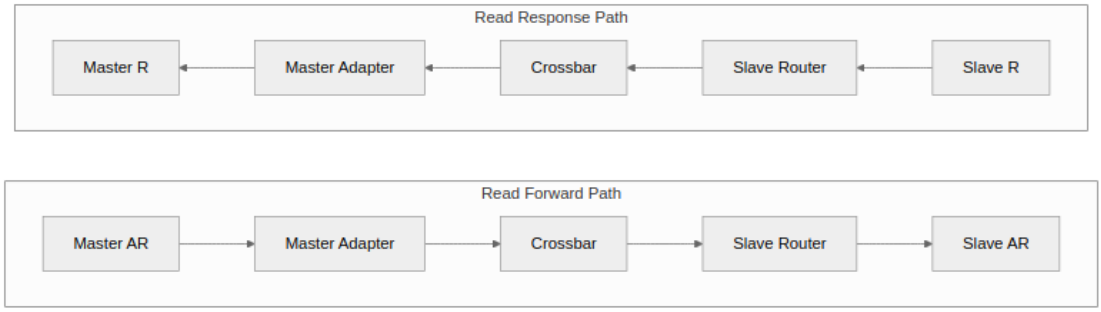
12.3 Data Flow

12.3.1 Figure 3.2: Write Path



Write Path Flow

12.3.2 Figure 3.3: Read Path



Read Path Flow

12.4 Scalability

12.4.1 Topology Parameters

Table 3.1: Topology Parameter Scaling

Parameter	Range	Impact
NUM_MASTERS	1-32	Linear MUX growth
NUM_SLAVES	1-256	Linear decoder growth
DATA_WIDTH	32-512	Linear data path growth
ID_WIDTH	1-16	Pass-through; scales the ID wires only (no CAM)

12.4.2 Resource Scaling

- **Logic:** O(M x N) for crossbar core
- **Routing:** O(M + N) for address/response paths
- **Memory:** O(M x outstanding) for ID tracking

13 Data Flow

13.1 Transaction Flow Overview

13.1.1 Write Transaction Flow

1. **Master issues AW** - Address and control information
2. **Bridge decodes address** - Determines target slave
3. **Arbitration** - Grants access if multiple masters contend
4. **AW forwarded to slave** - ID unchanged; the originating master is pushed onto the slave adapter's `bridge_id` FIFO
5. **Master sends W data** - Following the granted AW
6. **W forwarded to slave** - Using same grant
7. **Slave returns B response** - with the master's own ID; the FIFO head selects the destination
8. **Bridge routes B to master** - Using ID to find originator

13.1.2 Read Transaction Flow

1. **Master issues AR** - Address and control information
2. **Bridge decodes address** - Determines target slave
3. **Arbitration** - Grants access if multiple masters contend
4. **AR forwarded to slave** - ID unchanged; master recorded in the read `bridge_id` FIFO
5. **Slave returns R data** - Potentially multiple beats
6. **Bridge routes R to master** - Using ID to find originator

13.2 Channel Independence

13.2.1 AXI4 Channel Separation

Each AXI4 channel operates independently:

Table 3.2: AXI4 Channel Summary

Channel	Direction	Contents	Arbitration
AW	Master to Slave	Write address	Per-slave
W	Master to Slave	Write data	Follows AW grant
B	Slave to Master	Write response	ID-based routing
AR	Master to Slave	Read address	Per-slave
R	Slave to Master	Read data	ID-based routing

13.2.2 Parallel Operation

- Read and write transactions can proceed simultaneously
- Different masters can access different slaves in parallel
- Same slave serializes access via arbitration

13.3 Bridge ID (there is no ID extension)

13.3.1 Bridge ID (BID) Concept

```
Original Master ID: [ID_WIDTH-1:0]
AXI ID on the slave port: the master's original ID, unmodified
BID Width: clog2(NUM_MASTERS)
```

13.3.2 IDs are pass-through, not extended

The generated RTL does **not** widen or prepend anything. A master's ID reaches the slave unchanged, and the slave port is declared at the same width:

```
input  logic [3:0]  cpu_axi4_awid     // master side
output logic [3:0]  ddr_s_axi_awid    // slave side -- same width
```

The slave-side ID is `{master index, master id}` (BRIDGE-016): the widest master's id_width plus `$clog2(NUM_MASTERS)` bits, so masters never alias at a slave and an `enable_ooo` slave can track by ID across them. For one master the prefix is empty and the slave sees the master's width unchanged.

Earlier revisions of this page described a bridge-ID-prepend scheme, with `4'b0101` becoming `6'b00_0101` and the upper bits extracted on the response. That design was never built. It is documented here only because the mechanism that replaced it has a consequence worth understanding.

13.4 Response Routing

Each slave adapter keeps an **in-order FIFO** of the originating master's `bridge_id`: pushed on the address handshake, popped on the response.

```
wr_fifo[wr_ptr[...]] <= xbar_bridge_id_aw;      // push on AW accept
assign bid_bridge_id  = wr_fifo[rd_ptr[...]];   // route by FIFO HEAD
```

13.4.1 B Channel (Write Response)

1. Slave issues B carrying the master's own (unmodified) ID
2. The slave adapter pops its `bridge_id` FIFO
3. That FIFO entry – **not** anything in the BID – selects the master
4. The B beat is forwarded unchanged

13.4.2 R Channel (Read Data)

1. Slave issues R carrying the master's own (unmodified) ID
2. The slave adapter's read FIFO selects the master, popped on RLAST
3. RLAST indicates the final beat

13.4.3 The requirement this creates

Because routing is keyed on FIFO *position* rather than on the returned ID, **each slave port must return B/R in request order across ALL IDs.** AXI4 permits a slave to complete different-ID transactions out of order, and a multi-ported memory controller normally does; nothing in the fabric detects or prevents it. Two masters with writes outstanding at one slave are enough to expose it – the response goes to the wrong master, carrying an ID that master never issued. Tracked as BRIDGE-010.

13.5 Ordering: what the fabric actually guarantees

Out-of-order completion is not supported. There are no ID tracking tables; `bridge_cam.sv` exists in the tree but is instantiated in zero generated bridges. Ordering is enforced structurally instead, in two places:

Per master – one target at a time. A master may not have transactions outstanding to more than one slave simultaneously. `aw_gate_ok`/`ar_gate_ok` hold off a new address phase until every outstanding transaction targets the same slave. This is a deadlock fix: the response mux replays in address-issue order, slaves respond in their own order, and cross-slave outstanding transactions from several masters can wedge the heads against each other. Same-slave pipelining is unaffected.

Per slave – responses must come back in order. See "The requirement this creates" above, and BRIDGE-010.

The sequence an earlier revision of this page offered as a worked example – master 0 issuing to slave 0 and slave 1 concurrently, then the fast slave answering first – is exactly what `aw_gate_ok` prevents. It cannot occur.

14 Protocol Support

14.1 Supported Protocols

14.1.1 AXI4 Full

Complete AMBA AXI4 protocol support:

- **5 channels:** AW, W, B, AR, R
- **Burst transactions:** FIXED, INCR, WRAP
- **Transaction IDs:** Configurable width
- **Out-of-order:** NOT supported – responses must return in request order (BRIDGE-010)
- **Data widths:** 32, 64, 128, 256, 512 bits

14.1.2 AXI4-Lite

Simplified AXI4 for register access:

- **5 channels:** Same as AXI4
- **Single-beat only:** AWLEN/ARLEN = 0
- **No IDs:** Transaction ordering by channel
- **Fixed width:** Typically 32 or 64 bits

14.1.3 APB (Advanced Peripheral Bus)

Low-power peripheral protocol:

- **Single channel:** Combined address and data
- **Simple handshake:** PSEL, PENABLE, PREADY
- **No bursts:** Single transfer per transaction
- **Low overhead:** Minimal logic for slow peripherals

14.2 Protocol Conversion Matrix

Table 3.3: Protocol Conversion Matrix

Master	Slave	Conversion Type	Notes
AXI4	AXI4	Direct	Width conversion if needed
AXI4	AXI4-Lite	Downgrade	Burst split to single beats
AXI4	APB	Full convert	Channel and timing conversion
AXI4-Lite	AXI4	Upgrade	Single-beat pass-through
AXI4-Lite	AXI4-Lite	Direct	Width conversion if needed
AXI4-Lite	APB	Convert	Simplified conversion
AXI5-Lite	any	Upgrade	As AXI4-Lite; `user`/`exclusive` ride the fabric, other sideband terminates at the boundary
APB / APB5	any	Full convert	`apb{4,5}_to_axi4` front end, then as AXI4-Lite
any	Wishbone B4	Full convert	`axi4_to_wb4`: one transfer per

Master	Slave	Conversion Type	Notes
			beat, in-order termination
Wishbone B4	any	Full convert	`wb4_to_axi4` front end, then as AXI4-Lite

14.3 Automatic Conversion Shims at Slave Boundary

When a slave port's TOML entry names a protocol other than native AXI4 (e.g., `protocol = "axil"` or `protocol = "apb"`), the generator emits the conversion shims between the crossbar core and that slave port. This happens once, at generation time; there is nothing to configure at runtime.

Supported Slave Protocols – these are the exact TOML values the generator accepts (`config_validator.valid_protocols`); anything else is a generation error, so spelling matters:

Table 3.4: Slave-port protocol values

`protocol`	Meaning	Shim emitted at the slave boundary
`axi4`	Native AXI4	none
`axi5`	AXI5, interop mode	`axi5_master_{wr,rd}` boundary wrappers
`axil`	AXI4-Lite	`axi4_to_axil4_{rd,wr}`
`axil5`	AXI5-Lite	`axi4_to_axil5_{rd,wr}`
`apb`	APB3/APB4	`axi4_to_apb4_shim`
`apb5`	APB5	`axi4_to_apb5_shim`
`wb4`	Wishbone B4 (pipelined)	`axi4_to_wb4`

Note the spelling: it is `axil`, not `axi4lite`. Earlier revisions of this page used the latter, which the generator rejects.

The shim modules live in `projects/components/converters/rtl/`; the generator instantiates them into each generated top-level module. The slave port still presents the configured protocol (AXIL or APB) to the outside; inside, the crossbar core is uniformly AXI4.

14.4 Automatic Conversion at the Master Boundary

The same `protocol` values are legal on a master port (BRIDGE-014). The bridge presents the requester's own protocol on the boundary – the AXI4-Lite signal set, the AXI5-Lite set with its sideband, or the APB completer set – and the master adapter converts to the AXI4 the crossbar speaks:

Table 3.5: Master-port protocol values

`protocol`	Boundary presents	Conversion in the master adapter
`axi4`	AXI4	none (timing wrapper only)
`axi5`	AXI5 (minus REGION, plus enabled features)	`axi5_slave_{wr,rd}` boundary wrappers
`axil`	AXI4-Lite	AXI4 extras tied (`AxLEN=0`, INCR, `WLAST=1`, ID = placeholder); the wide-slave aligner toward wider slaves
`axil5`	AXI5-Lite, full sideband	as `axil`; `exclusive` -> `AxLOCK`, `user` -> the 1-bit USER fields; every other group terminated at the top
`apb`	APB4 completer	`apb4_to_axi4`, then the AXI4 timing wrapper
`apb5`	APB5 completer (+ `PAUSER/PWUSER/PWAKEUP` in, `PRUSER/PBUSER` out)	`apb5_to_axi4` (`PAUSER[0]/PWUSER[0]` -> USER), then the AXI4 timing wrapper
`wb4`	Wishbone B4 completer (pipelined)	`wb4_to_axi4`, then the AXI4 timing wrapper; SLVERR/DECERR terminate ERR

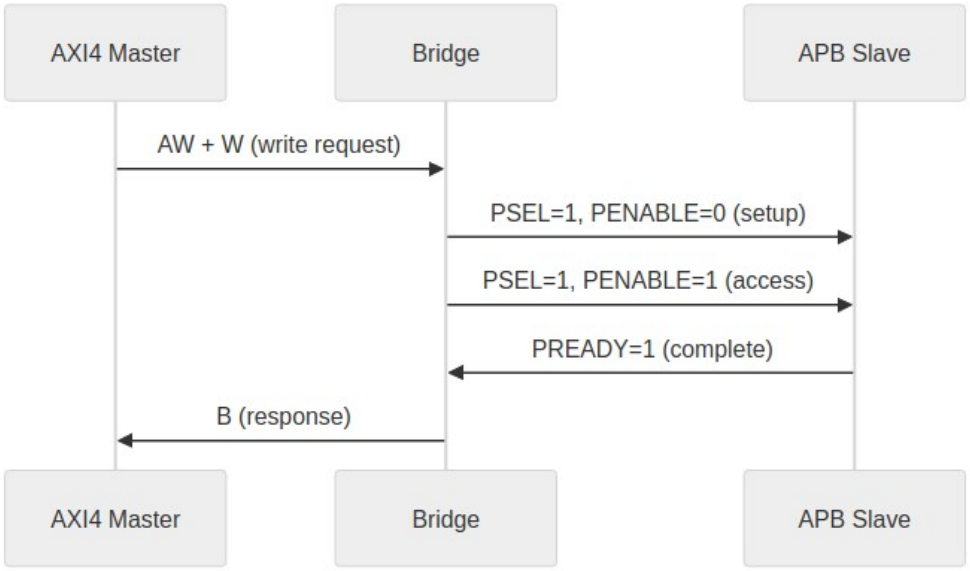
Rules the validator enforces on these ports: Lite masters have `id_width = 0` (no ID pins exist; the fabric ID is the master index, BRIDGE-016), APB and Wishbone masters have `addr_width = 32` (the requester addresses the whole fabric – an APB *slave* port's `PADDR` is a window offset, a

master's is not), and `axi5_features` on an `axil5` master may name only `user` and `exclusive`, the two groups with an AXI4 destination.

14.5 AXI4 to APB Conversion

14.5.1 Conversion Process

14.5.2 Figure 3.1: AXI4 to APB Conversion Sequence



AXI4 to APB Conversion

14.5.3 Burst Handling

- AXI4 bursts split into individual APB transfers
- WSTRB converted to per-byte enables
- B response generated after all beats complete

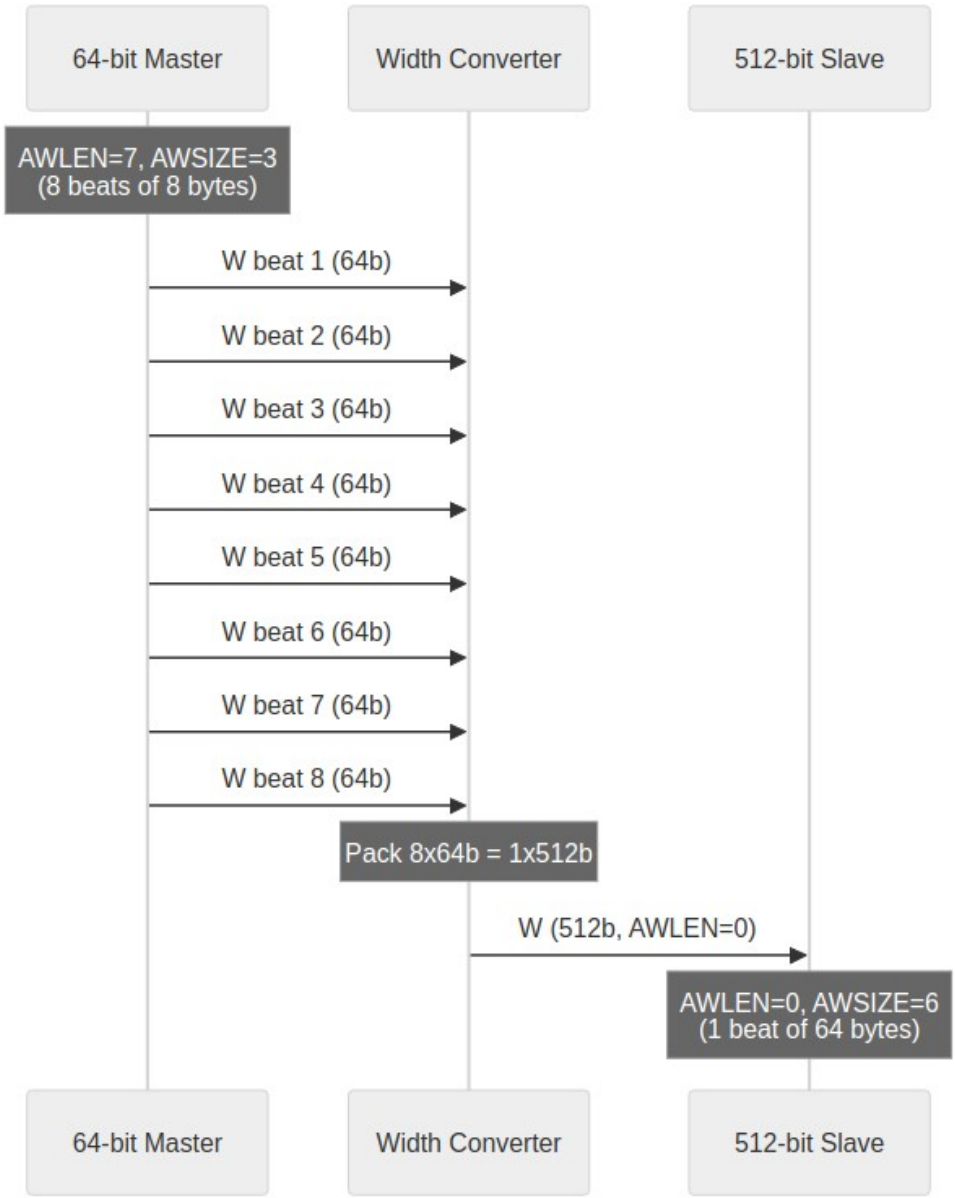
14.5.4 Timing Considerations

- APB is inherently slower (minimum 2 cycles per transfer)
- Pipeline stalls during APB access
- Consider separate APB crossbar for multiple slow peripherals

14.6 Width Conversion

14.6.1 Upsize (Narrow to Wide)

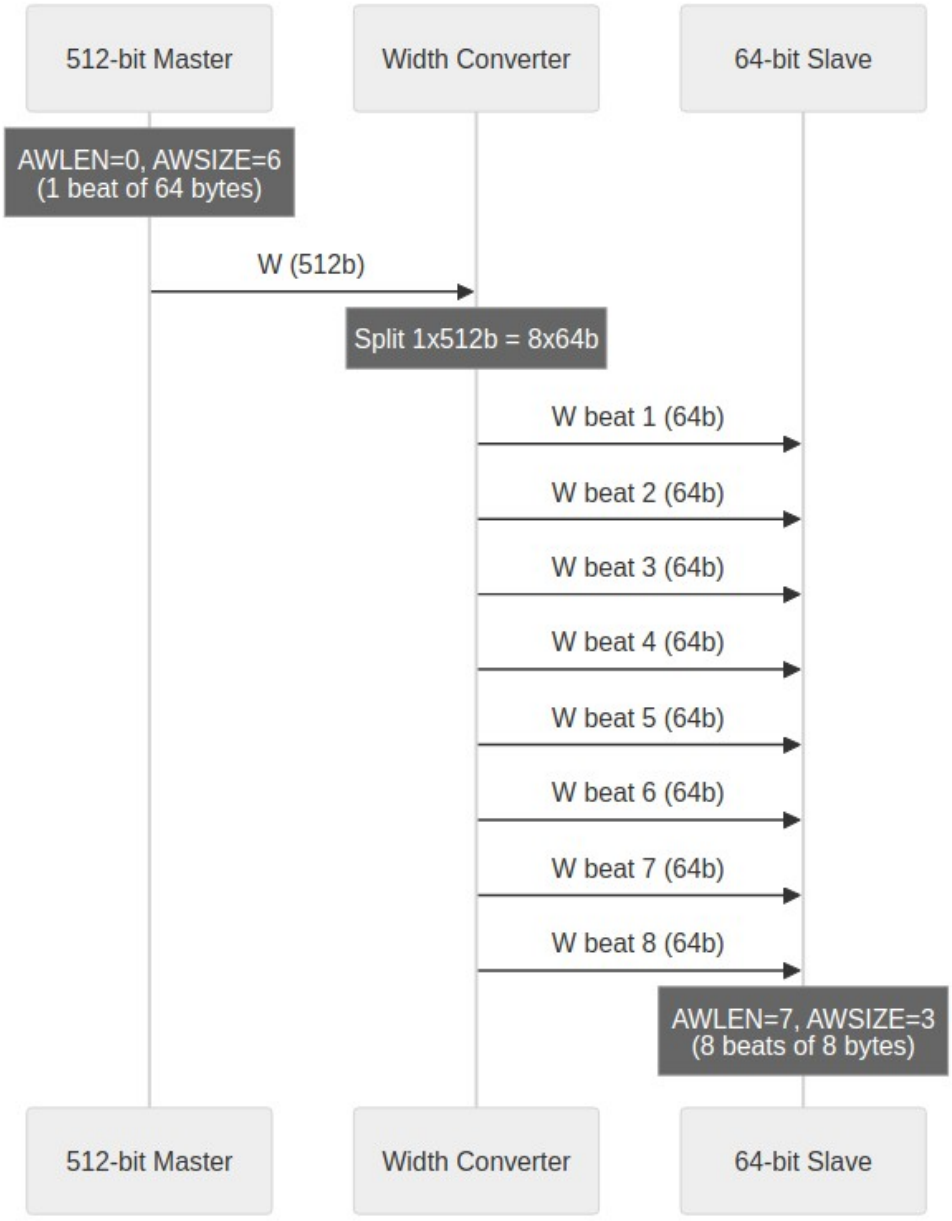
14.6.2 Figure 3.2: Width Upsize Conversion



Width Upsize

14.6.3 Downsize (Wide to Narrow)

14.6.4 Figure 3.3: Width Downsize Conversion



Width Downsize

14.7 Channel-Specific Protocol

14.7.1 Write-Only Masters

Only AW, W, B channels active:

- AR permanently deasserted
- R channel ignored (no routing logic)
- Reduced signal count and logic

14.7.2 Read-Only Masters

Only AR, R channels active:

- AW, W, B channels ignored
- No write arbitration for this master
- Reduced signal count and logic

15 AXI4 Interface Overview

15.1 Interface Organization

15.1.1 Master-Side Interfaces

Bridge presents **slave interfaces** to upstream masters. Each master port includes:

Table 4.9: Master-Side Interface Channels

Channel	Signals	Direction (to Bridge)
AW	AWVALID, AWREADY, AWADDR, AWID, AWLEN, AWSIZE, AWBURST, …	Input
W	WVALID, WREADY, WDATA, WSTRB, WLAST	Input
B	BVALID, BREADY, BID, BRESP	Output
AR	ARVALID, ARREADY, ARADDR, ARID, ARLEN, ARSIZE, ARBURST, …	Input
R	RVALID, RREADY, RDATA, RID, RRESP, RLAST	Output

15.1.2 Slave-Side Interfaces

Bridge presents **master interfaces** to downstream slaves. Each slave port includes:

Table 4.10: Slave-Side Interface Channels

Channel	Signals	Direction (from Bridge)
AW	AWVALID, AWREADY, AWADDR, AWID, AWLEN, AWSIZE, AWBURST, …	Output
W	WVALID, WREADY, WDATA, WSTRB, WLAST	Output
B	BVALID, BREADY, BID, BRESP	Input
AR	ARVALID, ARREADY, ARADDR, ARID, ARLEN, ARSIZE, ARBURST, …	Output
R	RVALID, RREADY, RDATA, RID, RRESP, RLAST	Input

15.2 Signal Naming Convention

15.2.1 Custom Prefixes

Each port uses a custom prefix for signal naming:

```
// Example: Master 0 with prefix "cpu_m_axi_"
input  logic [31:0] cpu_m_axi_awaddr,
input  logic [3:0]  cpu_m_axi_awid,
input  logic        cpu_m_axi_awvalid,
output logic        cpu_m_axi_awready,
// ... remaining AW signals

// Example: Slave 0 with prefix "ddr_s_axi_"
output logic [31:0] ddr_s_axi_awaddr,
output logic [3:0]  ddr_s_axi_awid,  // same width as the master port
output logic        ddr_s_axi_awvalid,
input  logic        ddr_s_axi_awready,
```

15.2.2 ID Width Extension

Slave-side IDs are the SAME width as master-side:

```
Master ID Width: ID_WIDTH (configuration parameter)
Bridge ID Width: clog2(NUM_MASTERS)
Slave ID Width: EQUAL to the master ID width -- IDs are pass-through.

There is no bridge_id prepended to the slave-side ID and no width
extension.
Responses are routed back to the originating master by an in-order
bridge_id
FIFO in each slave adapter, keyed on FIFO POSITION rather than on the
returned
BID/RID. That imposes a requirement the fabric does not check: each
slave port
must return B/R in request order across ALL IDs. See BRIDGE-010.
```

15.3 Channel-Specific Interfaces

15.3.1 Write-Only Master (wr)

Only AW, W, B channels generated:

```
// Write Address Channel
input  logic [ADDR_WIDTH-1:0] descr_m_axi_awaddr,
input  logic [ID_WIDTH-1:0]   descr_m_axi_awid,
// ... full AW channel

// Write Data Channel
input  logic [DATA_WIDTH-1:0] descr_m_axi_wdata,
input  logic [STRB_WIDTH-1:0] descr_m_axi_wstrb,
// ... full W channel

// Write Response Channel
output logic [ID_WIDTH-1:0]   descr_m_axi_bid,
output logic [1:0]            descr_m_axi_bresp,
// ... full B channel

// NO AR or R channels
```

15.3.2 Read-Only Master (rd)

Only AR, R channels generated:

```
// Read Address Channel
input  logic [ADDR_WIDTH-1:0] src_m_axi_araddr,
input  logic [ID_WIDTH-1:0]   src_m_axi_arid,
// ... full AR channel

// Read Data Channel
output logic [DATA_WIDTH-1:0] src_m_axi_rdata,
output logic [ID_WIDTH-1:0]   src_m_axi_rid,
// ... full R channel

// NO AW, W, or B channels
```

15.4 Monitor Bus Output (when `variants` includes `mon`)

When monitor collection is enabled (via the bridge TOML `variants` list including `"mon"`), each bridge instance exposes a unified monitor interface at the bridge top level:

Signal	Type	Width	Direction	Description
`s_mon_axil_*`	AXIL Slave	32-bit addr, 64-bit data	Input	Monitor packet read interface for CPU consumption
`m_axil_mon_*`	AXIL Master	32-bit addr, 64-bit data	Output	Monitor packet write interface for bulk trace capture
`stream_irq`	logic	1-bit	Output	Interrupt signal (asserted when error FIFO has records)

Per-port collection comes from the `axi4_master_{rd,wr}_mon` and `axi4_slave_{rd,wr}_mon` wrappers; a tree of `monbus_arbiter` instances funnels them into a single `monbus_axil4_axil4_group` at the bridge top. The group provides a 64-bit free-running timestamp counter, sampled at each packet arrival, and exposes both a slave interface (CPU read access) and a master interface (bulk DMA writes to system memory).

Packet Format: the canonical 128-bit packet layout and field definitions live in `docs/markdown/rtl-amba/includes/monitor_package_spec.md` — see that reference for bit-field descriptions, protocol-type enumerations, and packet-type codes.

15.5 Handshake Protocol

15.5.1 Valid/Ready Handshake

All channels use AXI4 valid/ready handshake:

```
Transfer occurs when: VALID && READY
```

15.5.2 Backpressure

- Bridge asserts READY when able to accept
- Masters must hold VALID until READY
- Bridge does not drop transactions

15.6 Burst Support

15.6.1 Supported Burst Types

Table 4.11: Supported Burst Types

AWBURST/ARBURST	Type	Description
2'b00	FIXED	Same address each beat
2'b01	INCR	Incrementing address
2'b10	WRAP	Wrapping at boundary
2'b11	Reserved	Not used

15.6.2 Burst Length

- AWLEN/ARLEN: 8-bit field
- Length = AXLEN + 1 (1 to 256 beats)
- Burst does not cross 4KB boundary (AXI4 rule)

16 APB Interface

16.1 Overview

APB is where the slow things live: UARTs, GPIO, configuration registers. This page covers the bridge's APB surface — the signal set, the port-naming convention, transaction timing, and what the AXI4-to-APB conversion actually does.

16.2 Ports

16.2.1 Signal Definition

Table 4.1: APB Signal Definitions

Signal	Width	Direction	Description
PSEL	1	Output	Slave select
PENABLE	1	Output	Transaction phase
PADDR	ADDR_WIDTH	Output	Address
PWRITE	1	Output	Write enable
PWDATA	DATA_WIDTH	Output	Write data
PSTRB	DATA_WIDTH/8	Output	Byte strobes
PPROT	3	Output	Protection
PRDATA	DATA_WIDTH	Input	Read data
PSLVERR	1	Input	Slave error
PREADY	1	Input	Slave ready

16.2.2 Signal Naming

APB slave ports use custom prefixes:

```
// Example: APB slave with prefix "uart_apb_"
output logic        uart_apb_psel,
output logic        uart_apb_penable,
output logic [31:0] uart_apb_paddr,
output logic        uart_apb_pwrite,
output logic [31:0] uart_apb_pwdata,
output logic [3:0]  uart_apb_pstrb,
output logic [2:0]  uart_apb_pprot,
input  logic [31:0] uart_apb_prdata,
input  logic        uart_apb_pslverr,
input  logic        uart_apb_pready
```

16.3 Functional Description

16.3.1 AXI4 to APB Conversion

When an AXI4 master accesses an APB slave, the bridge:

1. **Burst splitting** - AXI4 bursts become multiple APB transfers
2. **Channel mapping** - AW+W combined into PADDR+PWDATA
3. **Response generation** - APB PSLVERR maps to AXI4 BRESP/RRESP
4. **Timing adaptation** - Insert wait states as needed

16.3.2 Burst Handling

Table 4.2: AXI4 to APB Burst Conversion

AXI4 AWLEN	APB Transfers
0 (1 beat)	1 transfer
1 (2 beats)	2 transfers
N (N+1 beats)	N+1 transfers

16.3.3 Error Mapping

Table 4.3: APB to AXI4 Error Mapping

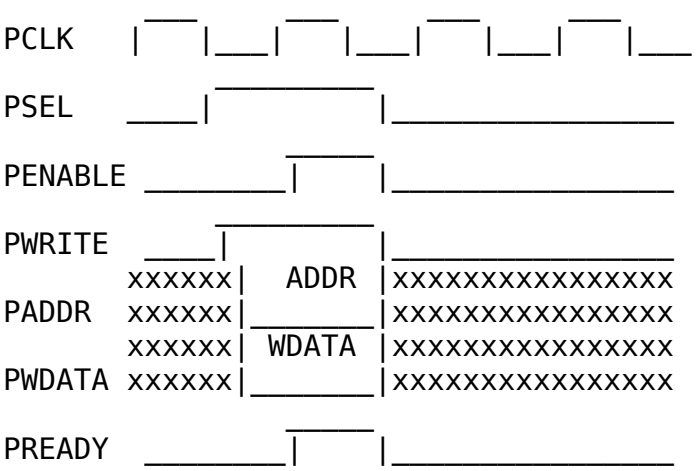
APB PSLVERR	AXI4 BRESP/RRESP
0 (OK)	2'b00 (OKAY)
1 (Error)	2'b10 (SLVERR)

16.4 Timing

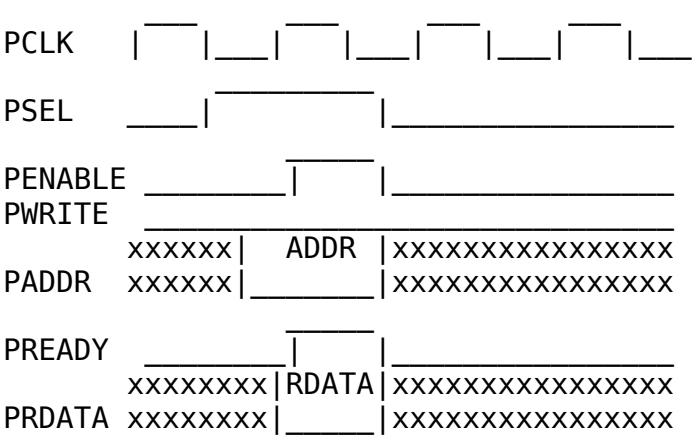
Every APB transfer pays for a setup phase plus an access phase, so the floor is two cycles per transfer. There is no pipelining — transfers run sequentially — and no bursts: each beat requires the full handshake.

16.5 Waveforms

16.5.1 Write Transaction



16.5.2 Read Transaction



16.6 APB Requester Ports

Since BRIDGE-014 an APB port can also be a *master*: `protocol = "apb"` or `"apb5"` in `[[bridge.masters]]`. The signal set is the same ten (fifteen for APB5) with the directions of the slave table reversed – the bridge is the completer, so `PSEL`, `PENABLE`, `PADDR[31:0]`, `PWRITE`, `PWDATA`, `PSTRB`, `PPROT` are inputs and `PREADY`, `PRDATA`, `PSLVERR` outputs. `PADDR` is the full 32-bit fabric address (the validator insists on `addr_width = 32`); each transfer becomes one single-beat AXI4 transaction through `apb4_to_axi4` / `apb5_to_axi4`, and both `SLVERR` and `DECERR` come back as `PSLVERR`. The requester is one-outstanding by nature of APB, so throughput is one transfer per fabric round trip.

16.7 Design Notes

- Use APB only for slow peripherals (UART, GPIO, config registers)
- Group APB slaves behind dedicated sub-crossbar
- Use AXI4/AXI4-Lite for higher-bandwidth slaves

17 Clock and Reset

17.1 Overview

One clock, one reset. The AXI fabric runs in a single synchronous domain — there is exactly one exception, at an APB slave boundary, and it gets its own discussion below — and the reset is asynchronous, active-low. This page covers what that means for your clock tree, your reset strategy, and your constraint file.

17.2 Ports

17.2.1 Clock

Table 4.4: Clock Requirements

Signal	Description	Requirements
aclk	System clock	All interfaces synchronous
Frequency	Operating frequency	Design-dependent
Duty cycle	Clock duty cycle	40-60% typical

17.2.2 Reset

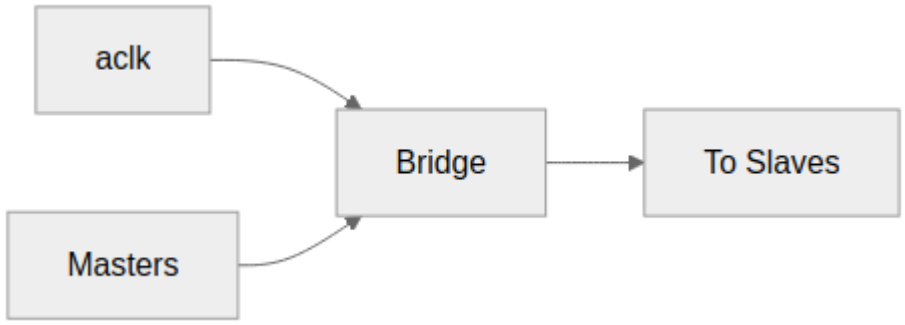
Table 4.5: Reset Signal

Signal	Description	Polarity
aresetn	Async reset	Active-low

17.3 Functional Description

17.3.1 Single Clock Domain

17.3.1.1 Figure 4.1: Clock Distribution



Clock Distribution

All masters and slaves must be in the same clock domain as Bridge.

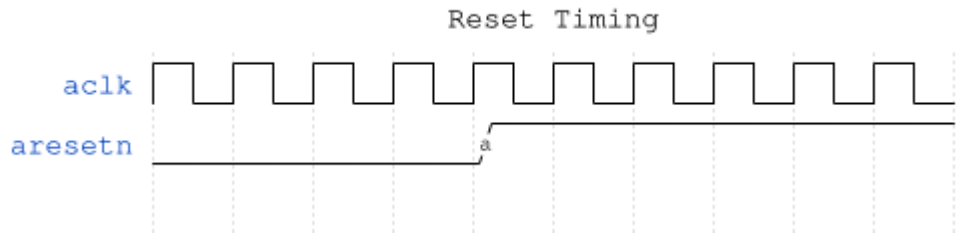
17.3.2 No Clock Domain Crossing

The bridge FABRIC does not include CDC logic. One exception matters: an `apb`/`apb5` slave gets `axi4_to_apb4_shim`, whose core contains two gray-pointer async FIFOs (`u_cmd_cdc_fifo`, `u_rsp_cdc_fifo`) and its own `pclk`/`presetn`. So a bridge WITH an APB slave does cross clock domains, at that slave boundary only. For the AXI fabric itself:

- All inputs sampled on aclk rising edge
- All outputs generated on aclk rising edge
- External CDC required if masters/slaves use different clocks

17.3.3 Reset Behavior

17.3.3.1 Figure 4.3: Reset Timing



Reset Timing

Reset is active-low: the bridge is in reset whenever `aresetn = 0`.

17.3.4 Reset Effects

During reset (aresetn = 0):

Table 4.6: Reset Effects

Component	State
Arbiters	Cleared (no grants)
bridge_id FIFOs	Pointers cleared (no outstanding)
FIFOs	Emptied
Outputs	Deasserted (VALID = 0)

17.3.5 Outstanding Transactions

Warning: reset discards transactions in flight:

- Master may see timeout (no response)
- Slave may receive partial transaction
- System must ensure quiescence before reset

17.3.6 Reset Release

After reset release (aresetn = 1):

1. Bridge accepts new transactions after 1 cycle
2. All state machines start in IDLE
3. No residual grants or locks

17.4 Timing

17.4.1 Setup and Hold

All inputs must meet setup/hold relative to aclk:

Table 4.7: Timing Constraints

Constraint	Typical Value
Setup time	Process-dependent
Hold time	Process-dependent

17.4.2 Clock-to-Output

All outputs appear after aclk rising edge:

Table 4.8: Clock-to-Output Delays

Path	Typical Delay
aclk to VALID	1 cycle (registered)
aclk to READY	1 cycle (registered)
aclk to DATA	1 cycle (registered)

READY is registered like the rest, which an earlier revision of this table gave as “Combinational”. Every external port terminates in a `gaxi_skid_buffer` inside its boundary wrapper – a master port’s `awready` is `axi4_slave_wr.s_axi_awready`, wired straight to the skid’s `wr_ready`, and that is assigned in an `ALWAYS_FF_RST` block. A master budgeting a combinational ready path against this bridge would be wrong by a register stage.

17.5 Usage Example

17.5.1 Recommended Constraints

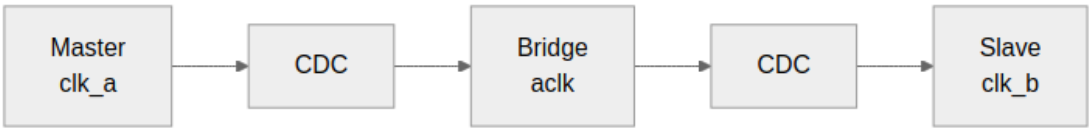
```
# Example SDC constraints
create_clock -period 10 [get_ports aclk]  ;# 100 MHz

# Reset is async - synchronize externally
set_false_path -from [get_ports aresetn]
```

17.6 Design Notes

17.6.1 External CDC Required

For systems with multiple clock domains:

17.6.1.1 Figure 4.2: Multi-Clock CDC



Multi-Clock CDC

17.6.2 CDC Recommendations

- Use AXI4 async FIFO bridges
- Consider AXI4 clock converter IP
- Ensure proper gray-coding for pointers

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

18 AXI5 and APB5 Interfaces (AMBA5 Support)

18.1 Overview

The bridge remains AMBA4-shaped internally — the crossbar fabric is always AXI4 — but any master or slave port can be declared AMBA5. This chapter covers the external surfaces; the mechanism lives in the MAS (AMBA5 Boundary and Native Sideband).

18.2 Parameters

18.2.1 Declaring AMBA5 Ports

AMBA5 is declared per port, in the TOML:

```
[[bridge.masters]]
name = "cpu_wr"
protocol = "axi5"
axi5_features = ["trace", "atomic"]   # optional; empty = base AXI5
# ... standard fields unchanged

[[bridge.slaves]]
name = "periph5"
protocol = "apb5"                     # APB5 peripheral via the apb5
shim
channels = "rw"                       # APB rules unchanged (rw-only,
32-bit)

[[bridge.masters]]
name = "lite5"
protocol = "axil5"                    # AXI5-Lite requester (BRIDGE-
014)
id_width = 0                          # Lite has no ID pins
axi5_features = ["user", "exclusive"]  # the two groups with an AXI4
destination

[[bridge.masters]]
name = "apb5m"
protocol = "apb5"                     # APB5 requester (BRIDGE-014)
id_width = 0
addr_width = 32                       # the requester addresses the
whole fabric
channels = "rw"
```

18.3 Ports

18.3.1 AXI5 Port Surface

An `axi5` port exposes the AXI4 signal set **minus AW/ARREGION** (AXI5 removed REGION) **plus** the enabled features' sideband signals. Disabled features' signals are not exposed at all — the boundary wrapper ties them off internally.

Feature	Signals added	Class
`nsaid`	`aw/arnsaid[3:0]`	Droppable sideband
`trace`	`aw/artrace`, `btrace`, `rtrace`	Droppable sideband
`mpam`	`aw/armpam[10:0]`	Droppable sideband
`mecid`	`aw/armecid[15:0]`	Droppable sideband
`unique`	`aw/arunique`	Droppable sideband
`poison`	`wpoison`, `rpoison`	**Connectivity-gated**
`atomic`	`awatop[5:0]`	**Connectivity-gated**; read-return classes native on rw ports, DECERR on write-only ports
`mte`, `chunking`	—	Rejected at config time (deferred)

18.3.2 APB5 Slave Surface

An `apb5` slave exposes the APB4 requester surface plus the APB5 sideband, mirroring `rtl/amba/apb5/apb5_slave.sv` pin-for-pin:

Direction	Signals
Requester → completer	APB4 set + `PAUSER`, `PWUSER` (driven `'0` — nothing upstream sources them)
Completer → requester	APB4 set + `PWAKEUP`, `PRUSER`, `PBUSER` (accepted and terminated)

The transfer protocol is unchanged from APB4, so the `axi4_to_apb5_shim` is a sideband wrapper over the APB4 conversion core; APB constraints (rw-only, 32-bit data) apply unchanged.

18.3.3 AXI5-Lite Master Surface

An `axil5` master port (BRIDGE-014) exposes the AXI4-Lite requester set plus the **whole** AXI5-Lite sideband, every group whether or not it is enabled – the same rule as the slave surface, so the boundary keeps one shape. The directions flip relative to the slave table: requester-driven groups are bridge inputs, completer-driven groups bridge outputs.

AXI5-Lite sideband groups at an `axil5` master boundary

Group	Signals	Behaviour
FORWARDED (when enabled)	`awlock`/`arlock` (`exclusive`); `awuser`, `wuser`, `aruser` (`user`)	reach the adapter's AXI4 face and ride the fabric to the slave
TERMINATED inputs	`aw/arloop`, `aw/armpam`, `aw/armecid`, `aw/arnsaid`, `aw/artrace`, `wpoison`; `awlock`/`awuser`/… when their feature is off	consumed at the bridge top (a reduction into an `_unused` wire), never floating
TERMINATED outputs	`bloop`, `btrace`, `rloop`, `rtrace`, `rpoison`; `buser`/`ruser` when `user` is off	driven `'0`
Response USER	`buser`, `ruser` with `user` on	connected, but read 0: the master adapters tie response-side USER (see the PRD)

18.3.4 APB5 Master Surface

An `apb5` master port makes the bridge the APB5 **completer**: the APB4 set with the requester-driven signals as inputs and `PREADY/PRDATA/PSLVERR` as outputs, plus `PAUSER`, `PWUSER`, `PWAKEUP` in and `PRUSER`, `PBUSER` out, one bit each (the fabric USER width). `PAUSER[0]`/`PWUSER[0]` become `awuser`/`aruser`/`wuser` on the fabric – observable at an `axil5` slave with `user` enabled. `PWAKEUP` is requester-driven and is accepted and terminated. Behind the surface sits `apb5_to_axi4` (`apb4_to_axi4` for `protocol = "apb"`), then the same AXI4 timing wrapper an AXI4 master port gets; from there the port is an AXI4-Lite-shaped single-beat requester and takes the wide-slave aligner toward wider slaves. An unmapped address answers `PSLVERR` (the subtractive slave's DECERR folds into APB's one error bit).

18.3.5 AXI5-Lite Slave Surface

An `axil5` slave gets `axi4_to_axil5_{rd,wr}` at the boundary: the AXI4-Lite conversion core plus the AXI5-Lite sideband. Because the master side is AXI4, the sideband splits three ways, and which group a signal falls in is the whole story of what an AXI4 front end can and cannot express.

AXI5-Lite sideband groups at an `axil5` slave boundary

Group	Signals	Behaviour
FORWARDED	`awlock`, `awuser`, `wuser`, `arlock`, `aruser` (request); `buser`, `ruser` (response)	AXI4 has an equivalent, passed straight through and returned to the master.
TIED	`awloop`, `awmecid`, `awmpam`, `awnsaid`, `awtrace`, `wpoison` (and the AR equivalents)	AXI5 additions with no AXI4 source. The port exists and is driven to `'0` — never left floating. There is deliberately no `ENABLE_` knob: there is nothing it could switch between.
TERMINATED	`bloop`, `btrace` (and the R equivalents)	Completer-driven, with nothing on the AXI4 side to return them to. Accepted and dropped.

18.4 Functional Description

Droppable sideband passes natively end-to-end when both ends of a path are AXI5, feature-enabled, and width-matched; on any other path it terminates at the fabric boundary with a generation-time warning.

Connectivity-gated features are a config **error** unless *every* connected path is native (AXI5 both ends, feature enabled both ends, data widths matched — dwidth converters cannot carry per-beat sideband). Dropping POISON silently would launder corrupted data; dropping ATOP would turn an atomic into a plain write.

Atomics depend on the port having a read path. `AWATOP = 01xxxx` (AtomicStore) and plain writes forward natively on any atomic-enabled port. Read-return classes (AtomicLoad `10xxxx`, AtomicSwap/Compare `11000x`) answer on the R channel with the AW's ID:

- On an **rw** master port they forward natively. The slave performs the operation and returns the location's original data on R; the bridge routes that beat back by ID (`axi5_atomic_rr_tracker` at the slave adapter, an extra AR->R tracking slot at the master adapter). Every connected atomic slave must be `rw` – the validator rejects a write-only one, since it could never return the data.
- On a **write-only** master port there is nowhere to deliver the R beat, so the boundary's `axi5_atomic_filter` answers those classes locally with **DECERR** – no slave-side AW handshake, no memory side effect.

The tied group is the honest limit of an AXI4 front end. MPAM partition IDs, MECID encryption contexts and NSAID security IDs are properties of the *originating* master; a bridge whose masters speak AXI4 has nowhere to learn them, so it drives zeros rather than inventing values. If a design needs real MPAM or MECID at the slave, the master port has to be `axi5`, not `axi4`.

Sideband is held for the whole burst, not just the address beat. AXI4 presents `AWUSER`/`AWLOCK` once, with AW; AXI5-Lite needs them stable across every beat that follows. The shims capture on the address handshake and hold:

```
wire w_aw_accept = s_axi_awvalid && s_axi_awready;
wire [AXI_USER_WIDTH-1:0] w_awuser_sel = w_aw_accept ? s_axi_awuser
                                                     : r_held_awuser;
```

A combinational passthrough here is a real defect rather than a style question: with two bursts in flight, burst A's later beats would carry burst B's `AWUSER`.

18.4.1 Interop Matrix

Master Slave	axi4	axi5	apb / apb5 / axil	axil5
axi4	native	base subset	via shim	via shim,

Master Slave	axi4	axi5	apb / apb5 / axil	axil5
				sideband TIED to `'0`
axi5	sideband drops (warning)	**native sideband** when width-matched	sideband drops (warning)	forwarded where AXI4-expressible
axil / axil5	single-beat AXI4	single-beat AXI4	via shim	`user`/ `exclusive` forwarded end to end (axil5 master); rest tied
apb / apb5	single-beat AXI4	single-beat AXI4	via shim (APB in, APB out)	`PAUSER[0]` visible as `awuser` (apb5 master)

Connectivity-gated features tighten the axi5→axi5 cell: they *require* it.

19 Unmapped-Address Handling

19.1 Overview

19.1.1 Why the bridge answers an address no slave owns

A positively-decoded fabric answers only addresses that fall inside some slave's range. An address in none of them selects nothing: the one-hot select is all-zero, no slave sees `AWVALID`/`ARVALID`, `READY` never rises, and the master waits forever.

A hang is the worst failure available here, because it destroys the evidence. There is no response to inspect, no error bit to read, and the offending address is whatever the master still has latched. A stray pointer in software becomes a wedged interconnect with nothing to point at the cause.

The bridge therefore ends its decode chain in an `else` – subtractive decode – and that `else` selects an internal slave that always answers.

19.2 Ports

19.2.1 Status, interrupt and clearing

The catch-all latches its status. Every bridge exposes:

Port	Dir	Meaning
`unmapped_irq`	out	**sticky**: set by the first unmapped access, held until cleared
`unmapped_addr[31:0]`	out	address of that **first** access
`unmapped_count[7:0]`	out	number of unmapped accesses, saturating at 255
`unmapped_clear`	in	pulse to clear the flag and the count

Three choices worth knowing about:

- **Sticky, not a pulse.** A one-cycle pulse is gone before software can look, and this is precisely the access nobody expected.
- **The first address wins.** A later fault does not overwrite it; the first one is usually what explains the rest.
- **The address survives the clear**, so a late reader still learns where the fault was after the flag is acknowledged. The count saturates rather than wrapping – “255+” is honest, a wrapped 3 is not.

19.3 Functional Description

19.3.1 Behaviour

Write	every `W` beat is accepted; one `B` per `AW`
Read	`AxLEN+1` beats, `RLAST` on the last
Response	**`DECERR`** on both, always
Read data	`0xDEADBEEF`, replicated to the bus width

`DECERR` rather than `SLVERR` is the AXI4 code for "no slave at this address": nothing failed to service the request, there was nothing there to service it. There is no `OKAY` mode and no parameter to select one – a mode switch here is where lint, simulation and synthesis end up disagreeing about what the fabric does.

The read data is a recognisable pattern rather than zeros because zeros are indistinguishable from real memory. `0xDEADBEEF` in a dump is a fault; `0` is a plausible value.

Write data is discarded. There is nowhere for it to go, and inventing a destination would be worse than saying so.

19.3.2 Register access (cfg regblock builds)

Bridges built with the cfg register block also expose the status over the cfg **AXI4-Lite** window. The port is `s_cfg_axil_*` – AXI4-Lite, not APB. An APB-attached CPU reaches it through an `axi4_to_apb4_shim` upstream of the bridge, which is an integration choice and not part of this module:

Unmapped-address status registers

Register	Field	Access	Meaning
`SUBTRACTIVE_STATUS`	`HIT[0]`	RO	mirrors `unmapped_irq`
	`CLEAR[1]`	W	write 1 to clear the flag and count
	`COUNT[15:8]`	RO	mirrors `unmapped_count`
`SUBTRACTIVE_ADDR`	`ADDR[31:0]`	RO	mirrors `unmapped_addr`

The sticky state lives in the RTL, not in the register block: one owner for one piece of state, so the two cannot disagree about whether a fault happened. These registers are a window onto it plus a clear. Clearing works from either the `unmapped_clear` pin or a `CLEAR` write; neither disables the other.

These registers are appended **after** every pre-existing register, so adding them did not move any existing offset. That is deliberate: a register map is an ABI. An earlier attempt inserted per-slave configuration for the catch-all midway through the map, which renumbered everything after it, silently moved the monitor-enable bits, and left the monitors switched off while every functional test still passed.

19.3.3 When it can never fire

The catch-all is the `else` of the decode chain, so it is reachable only if the slave ranges leave a **gap**. A map that tiles the whole address space has no unmapped address to catch, and the `else` is unreachable logic that synthesis removes – it costs nothing and reports nothing.

Most shipped variants are in exactly that position. Of the 22 generated bridges, **18 tile the space completely** and 4 leave gaps. `bridge_2x2_rw` is typical of the first group: `ddr` takes `<= 0x7FFF_FFFF` and `sram` takes `>= 0x8000_0000`, between them the entire 32-bit range.

This is worth knowing before wiring `unmapped_irq` to anything: on a fully-tiled bridge it is tied low by construction, and a test that expects it to assert will wait forever. Check your own address map, not this table – it changes with the TOML.

The insurance is still worth having. A map is tiled *today*; the next slave added, or the next range narrowed, opens a gap, and the difference between “reports it” and “hangs the master” is decided at that moment rather than at integration time.

19.3.4 What this does not do

The catch-all does not make an unmapped access *correct*. It makes it **visible and survivable**: the master gets an error instead of stalling, and software gets an address instead of a puzzle. An address that reaches it is still a configuration or software fault.

19.4 Navigation

Previous: AXI5 and APB5 Interfaces

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

20 Wishbone B4 Interface

20.1 Overview

Wishbone B4 is the open bus of the FPGA world – the fabric OpenCores cores speak and the one most soft CPUs ship with. Since BRIDGE-019 a bridge port can be Wishbone on either side: a **slave port** with `protocol = "wb4"` makes the bridge the Wishbone requester toward an external completer; a **master port** with `protocol = "wb4"` makes the bridge the completer for an external Wishbone requester. The fabric between stays AXI4.

The mode is **B4 pipelined** – a new STB every clock while STALL is low, in-order termination – which is what `rtl/amba/wb4` and the converters implement. B4 standard ("classic") mode exists in the converters as a parameter but is not selectable from the TOML; the two modes do not mix on one bus, so a classic peer needs a classic bridge build.

20.2 Ports

20.2.1 Signal Definition

Table 4.7: Wishbone B4 Signal Definitions

Signal	Width	Slave port (bridge requests)	Master port (bridge completes)	Description
CYC	1	Output	Input	Bus cycle in progress
STB	1	Output	Input	Transfer request
WE	1	Output	Input	1 = write, 0 = read
ADR	ADDR_WIDTH	Output	Input	Byte address (the fabric address, unwindowed)
DAT_W	DATA_WIDTH	Output	Input	Write data (the spec's DAT_O from the requester)
SEL	DATA_WIDTH/8	Output	Input	Byte lane select = AXI WSTRB
CTI	3	Output	Input	Burst hint (carried, not acted on; CLASSIC when the bridge drives it)
BTE	2	Output	Input	Burst type extension (LINEAR when the bridge drives it)
STALL	1	Input	Output	Pipelined back-pressure
ACK	1	Input	Output	Normal termination
ERR	1	Input	Output	Error termination
RTY	1	Input	Output	Retry termination (never driven by the bridge)
DAT_R	DATA_WIDTH	Input	Output	Read data (the spec's DAT_I at the requester)

20.2.2 Signal Naming

Wishbone ports use the TOML prefix followed by the uppercase B4 name, the convention `rtl/amba/wb4` and the RDS-DV BFMs share:

```
// Slave port, prefix "wbp_wb_" -- the bridge drives the bus
output logic        wbp_wb_CYC,
output logic        wbp_wb_STB,
output logic        wbp_wb_WE,
output logic [31:0] wbp_wb_ADR,
output logic [31:0] wbp_wb_DAT_W,
output logic [3:0]  wbp_wb_SEL,
output logic [2:0]  wbp_wb_CTI,
output logic [1:0]  wbp_wb_BTE,
input  logic        wbp_wb_STALL,
input  logic        wbp_wb_ACK,
input  logic        wbp_wb_ERR,
input  logic        wbp_wb_RTY,
input  logic [31:0] wbp_wb_DAT_R
```

A master port has the same thirteen with the directions reversed.

20.3 Functional Description

20.3.1 Slave port: AXI4 to Wishbone

`axi4_to_wb4` (converters MAS, chapter 3) sits in the slave adapter: the AXI4-Lite decomposers turn each AXI4 burst into one single-beat transfer per beat, and `axil4_to_wb4` turns those into Wishbone transfers, in order. `WSTRB` becomes `SEL`. Termination maps back as:

Table 4.8: Wishbone termination to AXI4 response

Wishbone	AXI4 response
ACK	OKAY
ERR	SLVERR
RTY	SLVERR (`RTY_RESP`; AXI has no retry)

The monitored (`mon`) variant places the `axi4_master_*_mon` wrappers between the crossbar and the converter, exactly as for APB slave ports.

20.3.2 Master port: Wishbone to AXI4

`wb4_to_axi4` sits in the master adapter in front of the ordinary AXI4 timing wrapper: `wb4_to_axil4` merges AXI's two response channels back into Wishbone's single in-order termination stream, and the wrapper promotes the AXI4-Lite face to AXI4 (`AxLEN = 0`, full-width `AxSIZE`, INCR, `WLAST = 1`, a constant ID – the fabric prepends the master index, BRIDGE-016). From the wrapper on, the port is an AXI4-Lite-shaped single-beat requester: decode, the width converters, the wide-slave aligner toward wider slaves and the response mux are the ones every other master uses. `SLVERR` and `DECERR` both terminate `ERR` – an unmapped address (the subtractive slave) is indistinguishable from a failing completer on this bus. `RTY` is never generated.

20.3.3 Rules the validator enforces

- `channels = "rw"` (one bus carries both directions), `id_width = 0`.
- `data_width` is 8, 16, 32 or 64.
- A master port has `addr_width = 32`: `ADR` is the full fabric address, where an APB or Wishbone *slave* port's address is whatever the fabric forwards for its window.

20.4 Design Notes

- **In-order by construction.** Wishbone terminates in issue order and the bridge's slave-side tracking is in-order for shim-converted slaves, so a Wishbone completer never needs the CAM. Two masters sharing a Wishbone completer are serialised by the crossbar's arbitration and answered in that order.
- **One transfer per beat.** A 16-beat AXI4 burst is sixteen Wishbone transfers; the pipelined bus takes one per clock when the completer does not stall, so the cost is latency, not bandwidth.
- **Burst hints are not generated.** `CTI`/`BTE` are CLASSIC/LINEAR on a slave port; a Wishbone completer that needs registered-feedback bursts to perform will not see them from the bridge.

Verified by `dv/tests/test_bridge_2x2_wb4_paths.py` on `bridge_2x2_wb4` (error folding both ways, SEL lanes at both a Wishbone and a 64-bit AXI4 completer, burst decomposition, both requesters in flight at the Wishbone completer) plus the generated tests of that fixture.

21 Throughput Characteristics

21.1 Overview

What the fabric can move when everything goes right, and what eats into that number when it doesn’t. The short version: bursts and matched widths win; conversions and contention cost you.

21.2 Functional Description

21.2.1 Peak Throughput

One master driving one slave — no contention, no conversion:

Table 5.1: Peak Throughput by Data Width

Data Width	Frequency	Peak Throughput
32-bit	100 MHz	400 MB/s
64-bit	100 MHz	800 MB/s
128-bit	100 MHz	1.6 GB/s
256-bit	100 MHz	3.2 GB/s
512-bit	100 MHz	6.4 GB/s

21.2.2 Formula

```
Peak Throughput = DATA_WIDTH (bits) × Frequency (Hz) / 8 bytes/bit
```

21.2.3 Factors Affecting Throughput

Table 5.2: Factors Affecting Throughput

Factor	Impact	Mitigation
Arbitration contention	Reduces per-master throughput	Increase slave ports
Width conversion	UNVERIFIED – see note	Match widths where possible
Protocol conversion	2+ cycles for APB	Use AXI4 for high-bandwidth
Response routing	Minimal (pipelined)	N/A

The width-conversion figure is not measured. “1-cycle penalty per direction” has no source: the converters (`axi_data_upsize`, `axi_data_dnsize`, `axi4_dwidth_converter_{rd,wr}`) decompose or combine beats, so the cost depends on the width RATIO and the burst length, and a single constant cannot be right for 256b->32b and 64b->32b alike.

It is left marked rather than replaced with a guess. Measuring it needs a propagation measurement on a converted path in `bridge_4x4_rw` (gpu 256b -> periph 32b) against a matched one (cpu 64b -> ddr0 64b), in the style of `test_bridge_2x2_rw_latency`. An attempt at that test could not

reliably observe the master-side reference and was withdrawn rather than committed half-working.

21.2.4 Multi-Master Scaling

With fair round-robin arbitration:

```
Per-Master Throughput = Peak Throughput / Active Masters (to same
slave)
```

21.2.4.1 Example: 4 Masters, 2 Slaves

```
All 4 masters accessing Slave 0:
  Per-master = Peak / 4

2 masters to Slave 0, 2 masters to Slave 1:
  Per-master = Peak / 2 (full parallelism)
```

21.2.5 Burst Efficiency

Table 5.3: Burst Efficiency Comparison

Transaction Type	Overhead	Efficiency
Single beat	1 cycle address + 1 cycle data	50%
4-beat burst	1 cycle address + 4 cycle data	80%
16-beat burst	1 cycle address + 16 cycle data	94%
256-beat burst	1 cycle address + 256 cycle data	99.6%

21.2.6 Width Conversion Impact

21.2.6.1 Upsize (Narrow to Wide)

```
64-bit to 512-bit (8:1 ratio):
  Input: 8 beats × 64 bits = 512 bits
  Output: 1 beat × 512 bits = 512 bits

Throughput: Preserved (same data, fewer beats)
Latency: +1 cycle (packing delay)
```

21.2.6.2 Downsize (Wide to Narrow)

```
512-bit to 64-bit (8:1 ratio):
  Input: 1 beat × 512 bits = 512 bits
```

```
  Output: 8 beats × 64 bits = 512 bits

Throughput: Preserved (same data, more beats)
Latency: +7 cycles (sequential output)
```

21.2.7 Protocol Conversion Impact

Table 5.4: AXI4 vs APB Protocol Impact

Metric	AXI4 Direct	AXI4 to APB
Min cycles/transfer	1	2
Burst support	Yes	No (split)
Pipeline depth	1+	1
Peak throughput	DATA_WIDTH/cycle	DATA_WIDTH/2 cycles

21.3 Design Notes

Bursts buy you efficiency:

- Use bursts whenever possible
- AXI4 supports up to 256 beats per burst
- APB does not support bursts (split internally)

And protocol choice matters more than most people expect:

- Keep high-bandwidth traffic on AXI4 paths
- Use APB only for low-bandwidth peripherals
- Group APB slaves to avoid impacting AXI4 traffic

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

22 Latency Analysis

22.1 Overview

Where the cycles go on the way through the bridge: address path, data path, response path, then the end-to-end best cases and the conversion costs. The headline numbers for a direct AXI4 path are measured, not estimated — the note after Table 5.7 explains how, and why an earlier version of that table was wrong.

22.2 Timing

22.2.1 Address Path Latency

Table 5.5: Address Path Latency

Stage	Cycles	Notes
Master Adapter (skid)	1	Pipeline registration
Address Decode	0	Combinational
Arbitration	0-1	0 if no contention, 1 if arbitrate
Slave Router	1	Pipeline registration
Total Address	**2-3**	Best/worst case

22.2.2 Data Path Latency

Table 5.6: Data Path Latency

Stage	Cycles	Notes
W follows AW	0	Same grant
Width conversion	0-1	0 if match, 1 if convert
Protocol conversion	0-2+	0 for AXI4, 2+ for APB
Total Data	**0-3+**	Best/worst case

22.2.3 Response Path Latency

Table 5.7: Response Path Latency

Stage	Cycles	Notes
Slave response	Variable	Slave-dependent
ID extraction	0	Combinational – routing is by FIFO position, no lookup
Response routing + master delivery	2	Registered; NOT a direct connection
Total Response	**2 + slave**	Plus slave latency

Measured. `bridge_2x2_rw`, cpu master to ddr slave, idle bridge:

Path	Cycles
AW accepted at the master -> AWVALID at the slave port	**2**
B accepted at the slave port -> BVALID at the master	**2**

Two skid stages each way: the master adapter's `axi4_slave_wr` AW skid, then the slave adapter's `axi4_master_wr` AW skid. The response path has the same structure, which is why the two figures match.

The response row previously read "Master delivery 0 (Direct connection)", total 1. It is registered, and the figure is 2.

What this measures, and what an earlier revision of this note got wrong. These are PROPAGATION times – how long a beat takes to appear on the far side. An earlier revision quoted 4 cycles for the request path, measured accept-to-accept. That metric also counts however long the far side held READY low, so it describes the attached slave's timing as much as the bridge's, and it was not even stable: successive runs gave 4/2 and then 2/6. Propagation is 2/2 on every run.

`test_bridge_2x2_rw_latency` asserts both values EXACTLY, so adding or removing a pipeline stage fails the test rather than silently ageing this table.

Other configurations differ – width converters and the APB/AXIL shims add stages – so this is the figure for a direct AXI4 path, not a universal constant.

22.2.4 End-to-End Latency

22.2.4.1 Write Transaction (Best Case)

```
Cycle 0: AW arrives at Bridge
Cycle 1: AW exits to Slave (skid buffer)
Cycle 2: Slave receives AW
Cycle 1: W arrives (same cycle as AW skid)
Cycle 2: W exits to Slave
Cycle 3: Slave receives W, generates B
Cycle 4: B routed back to Master

Total: 4 cycles (minimum)
```

22.2.4.2 Read Transaction (Best Case)

```
Cycle 0: AR arrives at Bridge
Cycle 1: AR exits to Slave (skid buffer)
Cycle 2: Slave receives AR, generates R
Cycle 3: R routed back to Master

Total: 3 cycles (minimum)
```

22.2.4.3 Contention Latency

When multiple masters contend for same slave:

```
Arbitration adds 1 cycle per contending master

Example: 4 masters requesting same slave
  Worst case: Wait for 3 other masters = 3 extra cycles
  Average case: Wait for 1.5 masters = 1-2 extra cycles
```

22.2.5 Width Conversion Latency

22.2.5.1 Upsize Latency

Table 5.8: Upsize Latency by Ratio

Ratio	Extra Cycles	Notes
1:1	0	No conversion
2:1	0	Pack on fly
4:1	0	Pack on fly
8:1	0-1	May need accumulation

22.2.5.2 Downsize Latency

Table 5.9: Downsize Latency by Ratio

Ratio	Extra Cycles	Notes
1:1	0	No conversion
1:2	+1	2 output beats
1:4	+3	4 output beats
1:8	+7	8 output beats

22.2.6 Protocol Conversion Latency

Table 5.10: AXI4 to APB Conversion Latency

Phase	Cycles
AXI4 AW decode	1
APB setup	1
APB access	1+ (PREADY)
B generation	1
Total	**4+ cycles**

22.2.6.1 APB Wait States

APB slaves may insert wait states:

```
PREADY deasserted: +1 cycle per wait
Typical UART/GPIO: 0-2 wait states
Slow peripherals: May have many wait states
```

22.3 Design Notes

22.3.1 Design Recommendations

1. **Match data widths** - Avoid conversion latency
2. **Use AXI4 for fast paths** - Avoid APB latency
3. **Minimize contention** - Spread traffic across slaves
4. **Use bursts** - Amortize address latency
5. **Pipeline responses** - Don't block on slow slaves

23 Resource Estimates

23.1 Overview

Rough silicon cost for the bridge and its converters — per master, per slave, per converter, and for a few complete configurations. Treat every number on this page as an order of magnitude, not a budget line: the note after Table 5.18 explains why, and it is worth reading before any of these figures make it into a spreadsheet.

23.2 Functional Description

23.2.1 Baseline Configuration (2x2, 64-bit)

Table 5.11: Baseline Resource Requirements

Resource	Count	Notes
LUTs	~2,000-3,000	Logic and routing
Registers	~1,500-2,000	Pipeline stages
Block RAM	0	No CAM or ID table is built
DSP	0	No arithmetic

23.2.2 Scaling Factors

Table 5.12: Resource Scaling Factors

Component	Scaling
Crossbar core	O(M x N)
Master adapters	O(M)
Slave routers	O(N)
ID tracking	O(M x Outstanding)
Width converters	O(per-path ratio)

23.2.3 Per-Master Resources

Table 5.13: Per-Master Resource Breakdown

Component	LUTs	Registers
Skid buffer	~50-100	~DATA_WIDTH
bridge_id FIFO (per slave)	~16 x BRIDGE_ID_WIDTH	in LUTs, not BRAM
Channel mux	~100	~50
Per Master Total	~200-300	~DATA_WIDTH + 100

23.2.4 Per-Slave Resources

Table 5.14: Per-Slave Resource Breakdown

Component	LUTs	Registers
Address decode	~50	0
Arbiter	~100-200	~50
Response mux	~100	~50
Per Slave Total	~250-350	~100

23.2.5 Crossbar Core

Table 5.15: Crossbar Core Resources

Component	LUTs	Registers
AW mux (N-way)	~100 x N	~50 x N
W mux (N-way)	~100 x N	~50 x N
B demux (M-way)	~50 x M	~50 x M
AR mux (N-way)	~100 x N	~50 x N
R demux (M-way)	~50 x M	~50 x M

23.2.6 Width Converter Resources

Table 5.16: Width Converter Resources

Ratio	LUTs	Registers	Notes
1:2 upsize	~200	~DATA_IN	Pack logic
1:4 upsize	~300	~DATA_IN	Pack logic
1:8 upsize	~400	~DATA_IN	Pack logic
2:1 downsize	~200	~DATA_OUT	Split logic
4:1 downsize	~300	~DATA_OUT	Split logic
8:1 downsize	~400	~DATA_OUT	Split logic

23.2.7 Protocol Converter Resources

23.2.7.1 AXI4 to APB

Table 5.17: AXI4 to APB Converter Resources

Component	LUTs	Registers
FSM	~100	~20

Component	LUTs	Registers
Burst counter	~50	~8
Data buffer	~100	~DATA_WIDTH
Response logic	~50	~10
Total	~300	~DATA_WIDTH + 50

23.2.8 Example Configurations

Table 5.18: Complete Bridge Resource Estimates

Config	Masters	Slaves	Data	LUTs	Registers
2x2 Basic	2	2	64	~2,500	~1,500
4x4 Standard	4	4	64	~5,000	~3,000
4x4 Wide	4	4	256	~8,000	~5,000
8x8 Large	8	8	128	~12,000	~8,000
RAPIDS (4x3)	4	3	512	~10,000	~6,000

23.2.8.1 Notes

- Estimates include all converters and adapters
- Actual results vary by synthesis tool and FPGA family
- Block RAM usage is zero: the per-slave bridge_id FIFOs are small LUT arrays
- DSP usage is zero (no multiplication/division)

These are hand estimates, and they do not reconcile. Summing the per-component tables above for the same configuration does not reproduce the summary figures in Table 5.18, and neither has been checked against a synthesis report. Treat every number on this page as an order-of-magnitude guide, not a budget: use it to tell “a few thousand LUTs” from “a few hundred”, and run synthesis for anything finer.

The honest fix is a synthesis run per shipped variant with the numbers replaced by measurements, which nobody has done. Saying so is better than leaving arithmetic that looks authoritative and is not.

23.3 Design Notes

23.3.1 Resource Reduction

1. **Use channel-specific masters** - 40-60% port reduction
2. **Match data widths** - Avoid converter logic
3. **Use APB sparingly** - Reduces AXI4 overhead
4. **Limit outstanding transactions** - shallower per-slave bridge_id FIFOs

23.3.2 Performance/Resource Trade-off

Table 5.19: Performance/Resource Trade-offs

Feature	Resource Impact	Performance Impact
Deeper skid buffers	+registers	+timing margin
Deeper bridge_id FIFOs	+LUTs	+outstanding depth (no OOO support)
Pipeline stages	+registers	+frequency
Wider data paths	+routing	+throughput

24 System Requirements

24.1 Overview

Everything your system owes the bridge before it will do its job: one clean clock, an asynchronous active-low reset, a single power domain, interfaces that follow the AXI4 rules, and an address map with no overlaps. Gaps in the map are allowed — the bridge answers those addresses itself — and the checklist at the end tells you how to prove all of it.

24.1.1 Clock Infrastructure

Table 6.1: Clock Requirements

Requirement	Specification
Clock source	Single clock domain (aclk)
Clock quality	Low jitter, stable frequency
Clock distribution	Must reach all Bridge ports

24.1.2 Reset Infrastructure

Table 6.2: Reset Requirements

Requirement	Specification
Reset type	Asynchronous, active-low (aresetn)
Reset synchronization	External synchronizer recommended
Reset distribution	Must reach all Bridge ports

24.1.3 Power

Table 6.3: Power Requirements

Requirement	Specification
Power domains	Single domain (no power gating)
Voltage	FPGA/ASIC process-dependent

24.2 Parameters

24.2.1 Address Map

Table 6.4: Address Map Requirements

Requirement	Specification
Slave ranges	Must not overlap
Range alignment	Power-of-2 recommended
Coverage	Gaps are permitted: an address in no slave range is claimed by the internal subtractive slave and answered with DECERR (see 4.5). It is still a fault – it is reported, not tolerated.

24.2.2 Connectivity

Table 6.5: Connectivity Requirements

Requirement	Specification
Matrix completeness	All masters must access at least one slave
Reachability	Consider traffic patterns for performance

24.3 Ports

24.3.1 Master Requirements

Masters connecting to Bridge must:

1. **Comply with AXI4 protocol** - Valid/ready handshake
2. **Use correct ID width** - Match configured ID_WIDTH
3. **Use correct data width** - Match configured DATA_WIDTH
4. **Use correct address width** - Match configured ADDR_WIDTH
5. **Stay within address range** - Target valid slave addresses

24.3.2 Slave Requirements

Slaves connecting to Bridge must:

1. **Comply with AXI4/APB protocol** - Based on configuration
2. **Accept the widened ID** - a slave sees `{master index, master id}`: the widest master's id_width plus `$clog2(NUM_MASTERS)` bits (none for a single master). Declare the slave's `id_width` at least that wide; the validator refuses less.
3. **Match configured data width** - Or use width conversion
4. **Respond to all transactions** - No hanging

24.4 Timing

24.4.1 Setup/Hold

Table 6.6: Timing Requirements

Constraint	Requirement
Input setup	Meet FPGA/ASIC requirements
Input hold	Meet FPGA/ASIC requirements
Clock-to-output	Per process characterization

24.4.2 Recommended Margins

Table 6.7: Recommended Timing Margins

Margin	Value	Purpose
Setup margin	10-20%	Variation tolerance
Hold margin	Positive	No hold violations
Clock uncertainty	Include in constraints	PLL/MMCM jitter

24.5 Testing

24.5.1 Pre-Integration Checks

1. **RTL lint clean** - No Verilator warnings
2. **Parameter validation** - All parameters in valid range
3. **Address map review** - no overlaps. Complete coverage is NOT required: a gap is claimed by the subtractive slave and answered with DECERR (4.5). Decide deliberately which you want – a gap that is meant to be a gap is fine, a gap you did not know about is a bug the fabric will now report.
4. **ID width calculation** - Sufficient for NUM_MASTERS

24.5.2 Integration Checks

1. **Signal connectivity** - All ports connected
2. **Width matching** - DATA_WIDTH, ADDR_WIDTH, ID_WIDTH
3. **Protocol matching** - AXI4 vs APB correctly configured
4. **Clock/reset connectivity** - All ports share same domain

24.5.3 Post-Integration Checks

1. **Basic transaction test** - Read/write to each slave
2. **Arbitration test** - Multi-master contention
3. **Error handling test** - unmapped-address response: DECERR + 0xDEADBEEF, `unmapped_irq` sticky, `SUBTRACTIVE_STATUS.CLEAR` clears it (see 4.5)
4. **Performance validation** - Meet throughput targets

---

25 Parameter Configuration

25.1 Overview

Every bridge the generator emits starts here: port counts, bus widths, per-port configuration, and the address map. This chapter collects the full parameter set — the core knobs, the widths the generator derives from them, the per-port fields, the monitor options, and the checks the generator runs before it accepts your configuration.

25.2 Parameters

25.2.1 Core Parameters

Table 6.8: Bridge Core Parameters

Parameter	Type	Range	Default	Description
NUM_MASTERS	int	1-32	2	Number of master ports
NUM_SLAVES	int	1-256	2	Number of slave ports
ADDR_WIDTH	int	12-64	32	Address bus width
DATA_WIDTH	int	32-512	64	Data bus width
ID_WIDTH	int	1-16	4	Master ID width
USER_WIDTH	int	1-16	1	User signal width

25.2.2 Derived Parameters

The generator works these out from the core set — you never write them yourself:

Table 6.9: Derived Parameters

Parameter	Formula	Example
BRIDGE_ID_WIDTH	clog2(NUM_MASTERS)	4 masters = 2 bits
STRB_WIDTH	DATA_WIDTH / 8	64b = 8 strobes

> `TOTAL_ID_WIDTH` was listed here as a derived parameter. It does not exist: IDs are not extended, so there is nothing to widen. `BRIDGE_ID_WIDTH` sizes the SIDEBAND master id carried beside the transaction, and is the only derived width the generated package defines.

25.2.3 Per-Port Configuration

Each master and slave port carries its own field set in the configuration file.

25.2.3.1 Master Port Configuration

Table 6.10: Master Port Configuration

Field	Type	Description
name	string	Unique identifier
prefix	string	Signal prefix
channels	enum	“rw”, “wr”, or “rd”
data_width	int	Port data width
id_width	int	Port ID width
use_monitor	bool	Optional, defaults to `true`. Per-port monitor wrappers. Only meaningful on a `mon` variant; on a `no` variant there is nothing to disable.

25.2.3.2 Slave Port Configuration

Table 6.11: Slave Port Configuration

Field	Type	Description
name	string	Unique identifier
prefix	string	Signal prefix
protocol	enum	`axi4`, `axi5`, `axil`, `axil5`, `apb`, `apb5` – the exact values `config_validator.valid_protocols` accepts. Note `axil`, not `axi4lite`.
data_width	int	Port data width
base_addr	hex	Base address (must be 4K-aligned)
addr_range	hex	Address range size (must be multiple of 4K)
use_monitor	bool	Optional, defaults to `true`. Per-port monitor wrappers. Only meaningful on a `mon` variant; on a `no` variant there is nothing to disable.

25.2.3.3 Slave Address Window Rules (MANDATORY)

Every slave must occupy a 4K-aligned address window that is a multiple of 4 KB:

```
Rule 1: Base Address Alignment
  base_addr & 0xFFF == 0x000

  Valid:   0x00000000, 0x00001000, 0x80000000, 0xFFFF0000
  Invalid: 0x00000001, 0x80000800, 0xC0000100

Rule 2: Range Alignment
  addr_range % 0x1000 == 0

  Valid:   0x1000 (4 KB), 0x2000 (8 KB), 0x10000 (64 KB)
  Invalid: 0x800 (2 KB), 0x1001, 0x7FFF

Rule 3: Non-Overlapping Windows
  All slaves must have non-overlapping address ranges

  Valid:   Slave0: 0x00000000-0x0FFFFFFF, Slave1: 0x10000000-
0x1FFFFFFF
  Invalid: Slave0: 0x00000000-0x10000000, Slave1: 0x0FFFFFFF-
0x1FFFFFFF
```

Why This Rule Exists:

- Real memory systems use 4K page boundaries (virtual memory, MMU)
- Linker scripts and memory maps assume 4K granularity
- Address decoders simplify to bit masks with 4K alignment
- Prevents ambiguity in address decode logic

There's nothing arbitrary about the 4K requirement — it's the granularity the rest of the system already assumes.

25.2.4 Top-Level Monitor Configuration

When monitor collection is desired, the bridge TOML configuration includes:

Field	Type	Description
variants	list	List of bridge variants to generate. Supported: `["no"]` (no monitor), `["mon"]` (monitor only), `["no", "mon"]` (both). Default: `["no"]`

25.2.4.1 Monitor Identifiers

Each monitored port gets a unique `(UNIT_ID, AGENT_ID)` pair for identification in monitor packets:

```
UNIT_ID Assignment:
  - UNIT_ID = 1: Master-side monitor wrappers
(axi4_master_{rd,wr}_mon)
  - UNIT_ID = 2: Slave-side monitor wrappers (axi4_slave_{rd,wr}_mon)

AGENT_ID Assignment (per port):
  - AGENT_ID = (port_index << 4) | channel_bit
  - channel_bit: 0 = read channel (AR/R), 1 = write channel (AW/W/B)
  - Example: Master port 2, write direction → AGENT_ID = (2 << 4) | 1
= 0x21
```

25.2.4.2 AXIL→Wider-Slave Master-Side Alignment

When an AXI4-Lite master connects to a wider AXI4 slave through the bridge (e.g., 32-bit AXIL master to 64-bit AXI4 slave), the generator emits a master-side alignment converter (the `axil_to_axi4_wide_align_{rd,wr}` modules) between the master adapter and the crossbar core. The converter handles partial-word alignment on the narrow side while preserving AXIL's single-beat semantics; the protocol stays AXIL — protocol shims are a slave-boundary concern, applied separately.

25.2.5 Parameter Validation

25.2.5.1 Generator Checks

The generator checks every configuration against these rules:

Table 6.12: Generator Validation Checks

Check	Error Condition
NUM_MASTERS	< 1 or > 32
NUM_SLAVES	< 1 or > 256
DATA_WIDTH	Not power of 2
Address overlap	Slave ranges intersect
Connectivity	Master with no slaves
ID width	Insufficient for masters

25.2.5.2 Runtime Validation

Bridge includes optional assertions for:

- Address alignment
- Burst boundary crossing
- Protocol violations
- ID mismatch

25.3 Usage Example

A complete configuration is two parts: the TOML file that describes the bridge, and the CSV that describes the connectivity.

25.3.1 TOML Configuration

```
[bridge]
  name = "my_bridge"
  description = "Example bridge configuration"

  defaults.skid_depths = {ar = 2, r = 4, aw = 2, w = 4, b = 2}

  masters = [
    {name = "cpu", prefix = "cpu_m_axi", channels = "rw",
     id_width = 4, addr_width = 32, data_width = 64, user_width = 1},
    {name = "dma", prefix = "dma_m_axi", channels = "rw",
     id_width = 4, addr_width = 32, data_width = 256, user_width = 1}
  ]

  slaves = [
    {name = "ddr", prefix = "ddr_s_axi", protocol = "axi4",
     id_width = 6, addr_width = 32, data_width = 512, user_width = 1,
     base_addr = 0x80000000, addr_range = 0x40000000},
    {name = "uart", prefix = "uart_apb", protocol = "apb",
     addr_width = 12, data_width = 32,
     base_addr = 0x00000000, addr_range = 0x00001000}
  ]
```

25.3.2 CSV Connectivity

```
master\slave,ddr,uart
cpu,1,1
dma,1,0
```

Two configurations to use as starting points:

25.3.3 Simple 2x2

```
[bridge]
  name = "bridge_simple"
  masters = [
    {name = "m0", prefix = "m0_axi", data_width = 64},
    {name = "m1", prefix = "m1_axi", data_width = 64}
  ]
  slaves = [
    {name = "s0", prefix = "s0_axi", base_addr = 0x0, addr_range =
0x10000000},
    {name = "s1", prefix = "s1_axi", base_addr = 0x10000000,
```

```
addr_range = 0x10000000}
  ]
```

25.3.4 Mixed Protocol

```
[bridge]
  name = "bridge_mixed"
  masters = [
    {name = "cpu", prefix = "cpu_axi", data_width = 64, channels =
"rw"}
  ]
  slaves = [
    {name = "mem",  prefix = "mem_axi",  protocol = "axi4", data_width
= 512,
     base_addr = 0x00000000, addr_range = 0x80000000},
    {name = "gpio", prefix = "gpio_apb", protocol = "apb",  data_width
= 32,
     base_addr = 0x80000000, addr_range = 0x00010000},
    {name = "uart", prefix = "uart_apb", protocol = "apb",  data_width
= 32,
     base_addr = 0x80010000, addr_range = 0x00010000}
  ]
```


---

26 Verification Strategy

26.1 Overview

Bridge gets verified at three levels — unit, integration, and system — and all of it runs on CocoTB with protocol BFMs doing the driving. This chapter lays out the test infrastructure, the execution model, the coverage targets, the debug hooks, and the regression schedule, then closes with the gaps we know about.

26.2 Testing

26.2.1 Verification Levels

26.2.1.1 Unit Level

Test individual Bridge components:

Table 6.13: Unit Level Test Focus

Component	Test Focus
Master Adapter	Skid buffer, sideband bridge_id (IDs pass through)
Slave Router	Address decode, arbitration
Width Converter	Upsize/downsize correctness
Protocol Converter	AXI4 to APB timing
Response Router	ID extraction, routing

26.2.1.2 Integration Level

Test complete Bridge functionality:

Table 6.14: Integration Test Coverage

Test Category	Coverage
Address routing	All master-slave paths
Arbitration	Contention, fairness
ID tracking	Response routing
Error handling	OOR, timeouts
Performance	Throughput, latency

26.2.1.3 System Level

Test Bridge in target system:

Table 6.15: System Level Tests

Test Category	Focus
End-to-end	CPU to memory
Multi-master	DMA + CPU
Mixed protocol	AXI4 + APB
Stress	Maximum traffic

Test Category	Focus

26.2.2 Test Infrastructure

26.2.2.1 CocoTB Framework with Protocol-BFM-Only Driving

Bridge verification runs on CocoTB with protocol BFMs only: **all test stimulus is driven through protocol BFMs (AXI4Master, AXI4Slave, etc.) with no direct DUT signal manipulation**. Each slave port is backed by an in-memory model (`MemoryModel`), and checking is assertion-based (readback matches write) rather than routing-based.

```
from CocoTBFramework.components.axi4 import AXI4Master, AXI4Slave
from TBClasses.shared.tbbase import TBBase

class BridgeTB(TBBase):
    def __init__(self, dut):
        super().__init__(dut)
        self.masters = [AXI4Master(dut, dut.aclk, f"m{i}_axi") for i
in range(NUM_MASTERS)]
        self.slaves = [MemoryModel(dut, dut.aclk, f"s{i}_axi") for i
in range(NUM_SLAVES)]

@cocotb.test()
async def cocotb_test_basic_write(dut):
    tb = BridgeTB(dut)
    await tb.setup_clocks_and_reset()

    # Write transaction via BFM only
    await tb.masters[0].write(addr=0x1000, data=0xDEADBEEF)

    # Verify: readback matches write (memory-model assertion)
    readback = await tb.masters[0].read(addr=0x1000)
    assert readback == 0xDEADBEEF
```

26.2.2.2 Test Categories

Table 6.16: Test Categories

Category	Description	Example
Basic	Single transactions	Read/write
Burst	Multi-beat transactions	16-beat burst
Concurrent	Multiple masters active	M0 + M1
Boundary Probe	Address window edges	Top/mid/bottom of each slave page

Category	Description	Example
Stress	Maximum utilization	Back-to-back
Error	Error conditions	SLVERR injection, timeout
Monitor	Monitor packet collection	Packet capture, IRQ assertion

26.2.3 Test Execution Model

26.2.3.1 Serial Execution

Bridge tests run **serially only** (no parallel execution via pytest-xdist). Each test function is named per-module to avoid cross-test cocotb function name pollution:

```
# Naming convention: test_bridge_{N}x{M}_{variant}_{scenario}
def test_bridge_1x2_rd_basic(request):        # Test name indicates 1
master, 2 slaves, read-only
def test_bridge_1x2_rd_monitor_smoke(request): # Monitor test on 1x2
def test_bridge_4x4_rw_concurrent(request):    # Concurrent test on
4x4
```

Cocotb test functions inside each file follow the same naming pattern but with `cocotb_test_*` prefix to avoid pytest collection conflicts.

26.2.3.2 Boundary Probe Testing

Address-window boundary testing probes the **top, middle, and bottom** of each slave's address window to catch address-decode logic errors at window edges:

```
for slave_idx, (base, window_size) in enumerate(slave_windows):
    # Probe bottom (base address)
    await master.write(base + 0x000, data)

    # Probe middle (arbitrary address within window)
    await master.write(base + window_size // 2, data)

    # Probe top (base + window_size - 1)
    await master.write(base + window_size - 1, data)
```

26.2.4 Coverage Goals

26.2.4.1 Functional Coverage

Table 6.17: Functional Coverage Goals

Category	Target
Address paths	100% master-slave

Category	Target
	combinations via boundary probes
Transaction types	Read, write, burst, error (SLVERR)
Monitor modes	Enabled and disabled
ID values	Full ID range (representative)

26.2.4.2 Code Coverage

Table 6.18: Code Coverage Goals

Metric	Target
Line coverage	>95%
Branch coverage	>90%
FSM coverage	100% state transitions
Toggle coverage	>80% signals

26.2.5 Debug Features

26.2.5.1 Waveform Capture

Bridge tests support VCD/FST waveform dumps:

```
pytest test_bridge.py --vcd=waves.vcd
gtkwave waves.vcd
```

26.2.5.2 Debug Signals

Bridge includes optional debug signals:

Table 6.19: Debug Signals

Signal	Description
dbg_aw_grant	Current AW grant
dbg_ar_grant	Current AR grant
dbg_outstanding	Outstanding transaction count
dbg_state	FSM states

26.2.6 Regression Testing

26.2.6.1 Continuous Integration

Table 6.20: CI Test Schedule

Stage	Tests	Frequency
Pre-commit	Basic sanity	Each commit
Nightly	Full regression	Daily
Weekly	Performance	Weekly

26.2.6.2 Test Matrix

Table 6.21: Configuration Test Matrix

Config	Data Width	Protocol	Tested
2x2	64	AXI4	Yes
4x4	64	AXI4	Yes
4x4	256	AXI4	Yes
2x2	64	AXI4+APB	Yes
RAPIDS	512	AXI4+APB	Yes

26.2.7 Known Limitations

26.2.7.1 Test Gaps

Table 6.22: Known Test Gaps

Gap	Status	Plan
APB burst stress	Partial	Phase 3
Width downsize stress	Partial	Planned
32-master config	Not tested	Low priority

26.2.7.2 Simulator Support

Table 6.23: Simulator Support Status

Simulator	Status
Verilator	Fully supported
Icarus Verilog	Supported
Commercial (VCS, Questa)	Not tested